Fondamenti dell'Informatica

Appunti di Michele Castenedoli

13 ottobre 2026

Indice

0 Prerequisiti matematici **1**
- 0.1 Basi di logica . . . 1
- 0.2 Insiemistica . . . 2

1 Introduzione **8**
- 1.1 Il problema dell'informatica . . . 8
- 1.2 Quante funzioni esistono? . . . 8
- 1.3 Quanti algoritmi esistono? . . . 9
- 1.4 Il teorema di Cantor . . . 10

2 Induzione **12**
- 2.1 Relazioni ben fondate . . . 12
- 2.2 Principio di induzione . . . 12

3 Linguaggi formali **14**
- 3.1 Alfabeti e stringhe . . . 14
- 3.2 Linguaggi . . . 15
- 3.3 Funzioni come linguaggi . . . 15
- 3.4 Automi a stati finiti deterministici . . . 16
- 3.5 Automi a stati finiti non deterministici . . . 21
- 3.6 Automi a stati finiti non deterministici con ε-transizioni . . . 25
- 3.7 Operazioni tra linguaggi . . . 29
- 3.8 Espressioni regolari . . . 30
- 3.9 Proprietà dei linguaggi regolari . . . 33
- 3.10 Problemi di decidibilità . . . 34
- 3.11 Relazioni di equivalenza e linguaggi . . . 34
- 3.12 Pumping lemma per i linguaggi regolari . . . 37

4 Linguaggi Context-Free **46**
- 4.1 Grammatiche libere dal contesto . . . 46
- 4.2 Linguaggio generato da $\mathcal{G}$ . . . 47
- 4.3 Alberi di derivazione . . . 50
- 4.4 Ambiguità . . . 51
- 4.5 Forme normali . . . 52
- 4.6 Eliminazione dei simboli inutili . . . 53
- 4.7 Eliminazione delle ε-produzioni . . . 55
- 4.8 Eliminazione delle produzioni unitarie . . . 57
- 4.9 Forma normale di Chomsky . . . 58

4.10 Pumping lemma per i linguaggi CF 59
4.11 Proprietà dei linguaggi CF 64
4.12 Proprietà di decisione 66

5 Esercizi **68**
5.1 Automi a stati finiti deterministici 68

0 Prerequisiti matematici

0.1 Basi di logica

0.1.1 Linguaggio del primo ordine

Un *linguaggio del primo ordine* è formato da quattro tipi di simboli:

1. Simboli di relazione, indicati con le metavariabili p, q, r;
2. Simboli di funzione, indicati con le metavariabili f, g, h;
3. Simboli di costante, indicati con le metavariabili c, d;
4. Simboli logici:
   - le parentesi (e) e la virgola;
   - un insieme numerabile di *variabili*, indicate con le metavariabili x, y, v;
   - i *connettivi logici* e i *quantificatori* $(\wedge, \vee, \neg, \implies, \iff, \forall, \exists)$;
   - il simbolo $\bot$, che indica il falso.

I simboli di relazione e di funzione hanno ciascuno la propria *arità*, ovvero il numero di argomenti.

A partire da questi simboli si costruiscono, in ordine, i termini, le formule atomiche e le formule.

Definizione (Termine). *I* termini *sono definiti per induzione:*

1. *ogni variabile è un termine;*
2. *ogni costante è un termine;*
3. *se f è un simbolo di funzione n-ario e $t_1, \dots, t_n$ sono termini, allora $f(t_1, \dots, t_n)$ è un termine.*

Definizione (Formula atomica). *Le* formule atomiche *sono:*

1. *$p(t_1, \dots, t_n)$, se p è un simbolo di relazione n-ario e $t_1, \dots, t_n$ sono termini;*
2. *il simbolo $\bot$.*

Definizione (Formula). *Le* formule *sono definite per induzione:*

1. *ogni formula atomica è una formula;*
2. *se ϕ è una formula, allora $\neg\phi$ è una formula;*
3. *se ϕ e ψ sono formule, allora $\phi \circ \psi$ è una formula, con $\circ \in \{\wedge, \vee, \implies, \iff\}$;*
4. *se ϕ è una formula e x è una variabile, allora $\forall x\,(\phi)$ e $\exists x\,(\phi)$ sono formule.*

0.1.2 Equivalenze

Si scrive $\phi \equiv \psi$ quando le formule ϕ e ψ sono *equivalenti*, ovvero assumono sempre lo stesso valore di verità.

Due negazioni consecutive si annullano (*doppia negazione*):

$$\neg(\neg\phi) \equiv \phi.$$

Le *leggi di De Morgan* descrivono la negazione di una congiunzione e di una disgiunzione:

$$\neg(\phi \wedge \psi) \equiv (\neg\phi) \vee (\neg\psi), \qquad \neg(\phi \vee \psi) \equiv (\neg\phi) \wedge (\neg\psi).$$

Un'*implicazione* può essere riscritta come disgiunzione, oppure come la sua *contronominale* (l'ultima forma):

$$(\phi \implies \psi) \equiv (\neg\phi) \vee \psi \equiv \big((\neg\psi) \implies (\neg\phi)\big).$$

Infine, la *negazione di un quantificatore* si ottiene scambiandolo con l'altro e negando la formula:

$$\neg\big(\exists x\,(\phi)\big) \equiv \forall x\,(\neg\phi), \qquad \neg\big(\forall x\,(\phi)\big) \equiv \exists x\,(\neg\phi).$$

0.1.3 Abbreviazioni

Per i quantificatori ristretti agli elementi di un insieme S si adottano le seguenti abbreviazioni:

$$(\exists x \in S.\ \phi) \equiv \exists x\,(x \in S \wedge \phi),$$
$$(\forall x \in S.\ \phi) \equiv \forall x\,(x \in S \implies \phi),$$

dove il punto si legge *tale che*.

0.2 Insiemistica

0.2.1 Definizione e rappresentazione

Un *insieme* è una collezione di elementi senza ripetizioni. Esso può essere rappresentato in due modi. La rappresentazione *intensionale* descrive gli elementi attraverso una loro caratteristica: ad esempio

$$\{x \mid p(x)\}$$

è l'insieme di tutti gli elementi x che rendono vero il predicato p. La rappresentazione *estensionale*, invece, elenca tutti gli elementi dell'insieme, come in $\{1, 2, 3\}$.

Un'altra definizione fondamentale per il resto del corso è quella di sottoinsieme.

Definizione (Sottoinsieme). *$A \subseteq B$, ovvero A è un* sottoinsieme *di B, se ogni elemento di A appartiene anche a B. A è invece un* sottoinsieme proprio *di B, $A \subset B$, se in più B contiene almeno un elemento che non appartiene ad A:*

$$A \subseteq B \iff \forall x\,(x \in A \implies x \in B),$$
$$A \subset B \iff A \subseteq B \wedge \exists x \in B.\ x \notin A.$$

0.2.2 Operazioni tra insiemi

Le operazioni classiche tra insiemi sono:

- Unione: $A \cup B = \{x \mid x \in A \vee x \in B\}$;
- Intersezione: $A \cap B = \{x \mid x \in A \wedge x \in B\}$;
- Differenza: $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$;
- Complemento: $\overline{A} = \{x \in U \mid x \notin A\}$, dove U è l'*insieme universo*, che contiene tutti gli elementi considerati.

Per ogni $x \in U$ vale quindi $x \in A \iff x \notin \overline{A}$ e, analogamente, $x \in \overline{A} \iff x \notin A$.

0.2.3 Relazioni

Per definire le relazioni è necessario il *prodotto cartesiano*. Dati gli insiemi $A_1, A_2, \ldots, A_n$, il loro prodotto cartesiano è definito come mostrato nella Figura 0.1.

$$A_1 \times A_2 \times \cdots \times A_n = \{\, \langle x_1, x_2, \ldots, x_n \rangle \mid \forall i.\ x_i \in A_i \,\}$$

Figura 0.1 *Prodotto cartesiano: la componente x_i di ogni tupla appartiene all'insieme A_i.*

Gli elementi del prodotto cartesiano sono le *tuple* $\langle x_1, x_2, \ldots, x_n \rangle$, in cui la componente i-esima x_i appartiene all'insieme A_i. A differenza degli insiemi, nelle tuple l'ordine è rilevante e gli elementi possono ripetersi: $\langle 1, 2 \rangle \neq \langle 2, 1 \rangle$, mentre $\{1, 2\} = \{2, 1\}$.

Definizione (Relazione binaria). *Dati due insiemi A e B, una* relazione binaria *tra A e B è un sottoinsieme del loro prodotto cartesiano:*

$$R \subseteq A \times B.$$

Se $x_1 \in A$ e $x_2 \in B$ sono in relazione si scrive

$$x_1 \ R \ x_2 \qquad \textit{oppure} \qquad (x_1, x_2) \in R.$$

Le coppie, ovvero le tuple di due elementi, si indicano anche con le parentesi tonde.

0.2.4 Proprietà delle relazioni

Le proprietà seguenti riguardano le relazioni su un insieme S, ovvero $R \subseteq S \times S$.

Definizione (Relazione riflessiva). *Una relazione R è* riflessiva *se ogni elemento è in relazione con se stesso:*

$$\forall x \in S.\ x \ R \ x.$$

Esempio. La relazione $\subseteq$ tra insiemi è riflessiva, poiché ogni insieme è sottoinsieme di se stesso:

$$\{1, 2, 5, 8\} \subseteq \{1, 2, 5, 8\}.$$

La riflessività deriva dal fatto che il simbolo $\subseteq$ ammette l'uguaglianza; il sottoinsieme proprio $\subset$, invece, non è riflessivo, poiché nessun insieme è sottoinsieme proprio di se stesso. ◇

Definizione (Relazione simmetrica). *Una relazione R è* simmetrica *se la posizione degli elementi nella relazione non è rilevante:*

$$\forall x, y \in S.\ x\ R\ y \implies y\ R\ x.$$

Esempio. L'uguaglianza tra insiemi è simmetrica: se $A = B$, allora anche $B = A$.

$$\underbrace{\{2, 5, 10\}}_{A} = \underbrace{\{2, 5, 10\}}_{B}$$

La relazione $\subseteq$, invece, non è simmetrica: $\{1\} \subseteq \{1, 2\}$, ma $\{1, 2\} \not\subseteq \{1\}$. ◇

Definizione (Relazione transitiva). *Una relazione R è* transitiva *se*

$$\forall x, y, z \in S.\ x\ R\ y \land y\ R\ z \implies x\ R\ z.$$

In altri termini, se x è in relazione con y e y è in relazione con z, allora anche x è in relazione con z, come illustrato nella Figura 0.2.

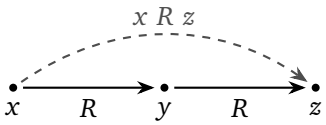


Figura 0.2 *Relazione transitiva: da $x\ R\ y$ e $y\ R\ z$ segue $x\ R\ z$.*

Definizione (Chiusura transitiva). *La* chiusura transitiva *di una relazione $R \subseteq S \times S$ è la relazione R^* definita come il più piccolo insieme $R^* \supseteq R$ tale che R^* sia transitiva: essa si ottiene aggiungendo a R le sole coppie necessarie a renderla transitiva.*

Esempio. Se $R = \{(1, 2), (2, 3)\}$, allora

$$R^* = \{(1, 2), (2, 3), (1, 3)\},$$

dove la coppia $(1, 3)$ è necessaria poiché $1\ R\ 2$ e $2\ R\ 3$. ◇

Definizione (Relazione antisimmetrica). *Una relazione R è* antisimmetrica *se*

$$\forall x, y \in S.\ x\ R\ y \land y\ R\ x \implies x = y.$$

In altri termini, elementi diversi non possono essere scambiati nella relazione: se $x \neq y$ e $x\ R\ y$, allora non può valere $y\ R\ x$.

Esempio. La relazione $\subseteq$ è antisimmetrica, poiché $A \subseteq B$ e $B \subseteq A$ implicano insieme $A = B$. ◇

0.2.5 Tipologie di relazione

Definizione (Tipi di relazione). *I tipi classici di relazione $R \subseteq S \times S$ sono:*

- *Equivalenza: R è riflessiva, simmetrica e transitiva.*
- *Pre-ordine: R è riflessiva e transitiva. Si tratta di un ordinamento molto debole, poiché manca l'antisimmetria: elementi diversi possono essere in relazione in entrambi i versi.*
- *Ordine parziale: R è un pre-ordine antisimmetrico.* Parziale *significa che non tutti gli elementi sono in relazione tra loro.*
- *Ordine totale: R è un ordine parziale in cui due elementi qualsiasi sono sempre in relazione:*

$$\forall x, y \in S.\ x\ R\ y \vee y\ R\ x.$$

Esempio. La relazione $=$ è di equivalenza, $\subseteq$ è un ordine parziale ($\{1\}$ e $\{2\}$ non sono in relazione in nessuno dei due versi) e $\leq$ sui numeri naturali è un ordine totale. ◇

Definizione (Partizione). *Una* partizione *di un insieme S è una famiglia di sottoinsiemi $\{S_1, S_2, \ldots, S_m\}$ di S, a due a due disgiunti, la cui unione è S:*

$$\bigcup_{i=1}^{m} S_i = S, \qquad \forall i \neq j.\ S_i \cap S_j = \emptyset.$$

Ogni elemento di S appartiene quindi a uno e un solo sottoinsieme della partizione, come mostra la Figura 0.3.

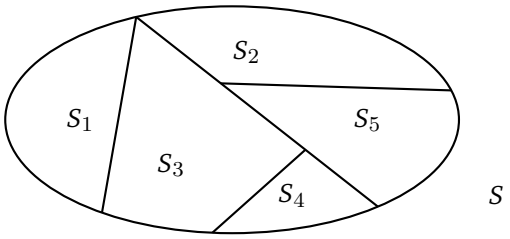


Figura 0.3 *Una partizione dell'insieme S in cinque sottoinsiemi disgiunti.*

Partizioni e relazioni di equivalenza sono strettamente legate: una relazione di equivalenza R su S e una partizione $\{S_1, \ldots, S_m\}$ di S si corrispondono quando

- due elementi dello stesso sottoinsieme sono sempre in relazione: se $a, b \in S_i$, allora $a\ R\ b$;
- due elementi di sottoinsiemi diversi non sono mai in relazione: se $a \in S_i$, $b \in S_j$ e $i \neq j$, allora $\neg(a\ R\ b)$.

Ogni partizione $\{S_1, \ldots, S_m\}$ definisce quindi una relazione di equivalenza, in cui due elementi sono in relazione se appartengono allo stesso sottoinsieme:

$$x\ R\ y \iff \exists i.\ x, y \in S_i.$$

Viceversa, ogni relazione di equivalenza R su S definisce una partizione di S, i cui sottoinsiemi raccolgono gli elementi in relazione tra loro.

Definizione (Classe di equivalenza). *Sia R una relazione di equivalenza su S e sia $y \in S$. La* classe di equivalenza *di y è l'insieme degli elementi in relazione con y:*

$$[y]_R = \{x \in S \mid x \; R \; y\}.$$

I sottoinsiemi della partizione definita da R sono esattamente le classi di equivalenza: ogni S_i è la classe $[y]_R$ di un qualsiasi suo elemento y, e

$$\{S_1, \ldots, S_m\} = \{[y]_R \mid y \in S\}.$$

0.2.6 Elementi minimali e massimali

Nel seguito una relazione d'ordine è indicata con $\leq$. La negazione di una relazione si indica barrandone il simbolo: $y \not\leq x$ significa che $y \leq x$ non vale.

Definizione (Elementi minimali e massimali). *Un elemento $x \in S$ è* minimale *rispetto a $\leq$ in S se nessun altro elemento di S è minore o uguale a x:*

$$\forall y \in S \setminus \{x\}.\ y \not\leq x.$$

Analogamente, $x \in S$ è massimale *rispetto a $\leq$ in S se nessun altro elemento di S è maggiore o uguale a x:*

$$\forall y \in S \setminus \{x\}.\ x \not\leq y.$$

In entrambe le definizioni è necessario escludere $y = x$: poiché $\leq$ è riflessiva, vale sempre $x \leq x$, e la condizione $\forall y \in S.\ y \not\leq x$ non sarebbe mai soddisfatta.

Se l'elemento minimale è unico si chiama *minimo*; se l'elemento massimale è unico si chiama *massimo*.[1]

Esempio. In $S = \{\{1\}, \{2\}, \{1, 2\}\}$ con la relazione $\subseteq$, gli elementi minimali sono $\{1\}$ e $\{2\}$, dunque non esiste un minimo, mentre l'unico elemento massimale $\{1, 2\}$ è il massimo. $\diamond$

0.2.7 Funzioni

Definizione (Funzione). *Una funzione è un caso particolare di relazione. Dati due insiemi D, il* dominio*, e C, il* codominio*, una relazione*

$$f \subseteq D \times C$$

è una funzione *da D a C, e si scrive $f\colon D \to C$, se*

$$\forall x \in D.\ \exists!\, y \in C \text{ tale che } f(x) = y.$$

Qui $f(x) = y$ significa $(x, y) \in f$ e il simbolo $\exists!$ si legge *esiste un unico*. Equivalentemente, a ogni elemento del dominio corrisponde uno e un solo elemento del codominio.

[1] Questo vale per gli insiemi finiti. In generale il minimo è l'elemento x tale che $\forall y \in S.\ x \leq y$ e il massimo è l'elemento x tale che $\forall y \in S.\ y \leq x$; in un insieme infinito può esistere un unico elemento massimale che non è il massimo.

Definizione (Immagine). *L'*immagine *di f, $\mathrm{range}(f) \subseteq C$, è l'insieme dei valori del codominio effettivamente assunti da f:*

$$\mathrm{range}(f) = \{\, c \in C \mid \exists d \in D.\ f(d) = c \,\} = \{\, f(d) \in C \mid d \in D \,\}.$$

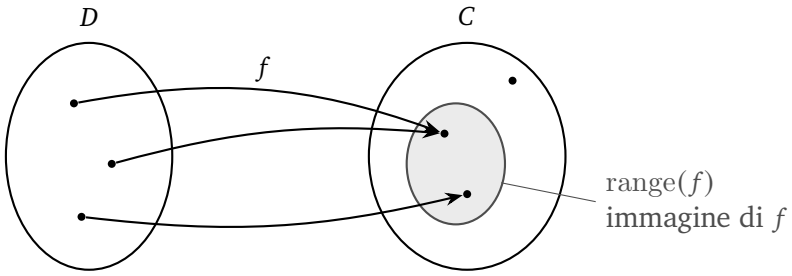


Figura 0.4 *Una funzione $f\colon D \to C$ e la sua immagine* $\mathrm{range}(f)$.

Nella Figura 0.4 ogni elemento di D è origine di esattamente una freccia, come richiesto dalla definizione di funzione, mentre un elemento di C può esserne destinazione più volte oppure mai; in quest'ultimo caso non appartiene all'immagine.

Definizione (Funzione iniettiva). *Una funzione $f\colon D \to C$ è* iniettiva *se elementi diversi del dominio hanno sempre immagini diverse:*

$$\forall x, y \in D.\ f(x) = f(y) \implies x = y.$$

Se f è iniettiva, dunque, due elementi distinti del dominio non possono avere la stessa immagine.

Definizione (Funzione suriettiva). *Una funzione $f\colon D \to C$ è* suriettiva *se ogni elemento del codominio è raggiunto da almeno un elemento del dominio:*

$$\forall y \in C.\ \exists x \in D.\ f(x) = y.$$

Definizione (Funzione biettiva). *Una funzione è* biettiva*, o* biezione*, se è sia iniettiva sia suriettiva: ogni elemento del codominio è raggiunto da* esattamente un *elemento del dominio.*

Due insiemi in biezione hanno la stessa cardinalità.

Definizione (Immagine inversa). *Data $f\colon D \to C$ e un sottoinsieme $Y \subseteq C$, l'*immagine inversa *di Y è l'insieme degli elementi del dominio che f manda in Y:*

$$f^{-1}(Y) = \{\, x \in D \mid f(x) \in Y \,\}.$$

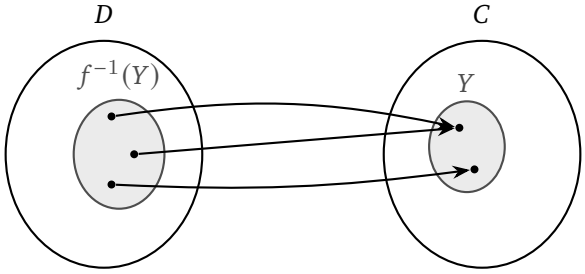


Figura 0.5 *Immagine inversa $f^{-1}(Y)$ di un sottoinsieme $Y \subseteq C$.*

1 Introduzione

1.1 Il problema dell'informatica

Si consideri una generica funzione matematica $f\colon \mathbb{N} \to \mathbb{N}$. Il problema di fondo dell'informatica consiste nello stabilire *quali funzioni siano calcolabili*.

Definizione (Funzione calcolabile).

$$f \textit{ è } \text{calcolabile} \iff \textit{esiste un algoritmo che la calcola},$$

dove per algoritmo *si intende una sequenza finita di passi di calcolo.*

Una stessa funzione calcolabile può essere calcolata da più algoritmi diversi, come illustrato nella Figura 1.1.

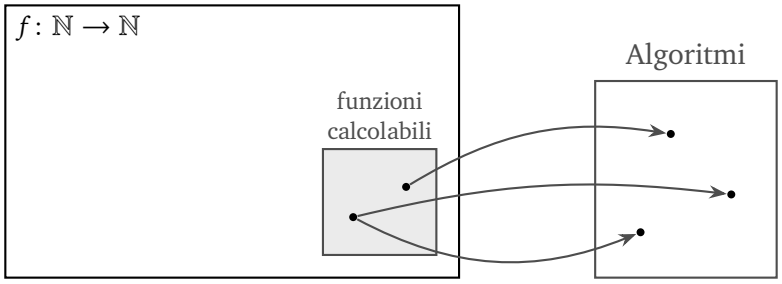


Figura 1.1 *Le funzioni calcolabili sono quelle per cui esiste almeno un algoritmo che le calcola.*

Per stabilire se tutte le funzioni siano calcolabili, si confronta la cardinalità dell'insieme delle funzioni con quella dell'insieme degli algoritmi.

1.2 Quante funzioni esistono?

Una funzione $f\colon \mathbb{N} \to \mathbb{N}$ può essere vista come un *insieme di coppie* in $\mathbb{N} \times \mathbb{N}$:

$$f(2) = 3 \;\mapsto\; (2,3) \qquad f(3) = 5 \;\mapsto\; (3,5) \qquad f(7) = 9 \;\mapsto\; (7,9)$$

Esempio. Una funzione è un insieme infinito di coppie, come

$$\{(0,3), (1,5), (2,2), (3,100), (4,1000), (5,20), (6,2), \dots\},$$

in cui ogni numero naturale compare come primo elemento di *esattamente una* coppia: non è possibile aggiungere un'altra coppia con primo elemento 2, poiché $f(2)$ è già fissato. ◇

Pertanto

$$f \subseteq \mathbb{N} \times \mathbb{N} \iff f \in \mathcal{P}(\mathbb{N} \times \mathbb{N}),$$

dove $\mathcal{P}(X)$ è l'*insieme delle parti* di X, ovvero l'insieme di tutti i suoi sottoinsiemi. Le funzioni da $\mathbb{N}$ in $\mathbb{N}$ sono quindi elementi di $\mathcal{P}(\mathbb{N} \times \mathbb{N})$.

1.2.1 $\mathbb{N} \times \mathbb{N}$ è numerabile

Vale $|\mathbb{N} \times \mathbb{N}| = |\mathbb{N}|$. Le seguenti affermazioni sono equivalenti:

- la cardinalità di $\mathbb{N} \times \mathbb{N}$ è uguale a quella di $\mathbb{N}$;
- l'insieme $\mathbb{N} \times \mathbb{N}$ è *numerabile*;
- esiste una *biezione* (mappatura univoca) tra $\mathbb{N} \times \mathbb{N}$ e $\mathbb{N}$.

Infatti, le coppie possono essere ordinate e numerate percorrendole per diagonali[1] (Figura 1.2):

$$(0,0) \mapsto 0 \quad (0,1) \mapsto 1 \quad (1,0) \mapsto 2 \quad (0,2) \mapsto 3 \quad (1,1) \mapsto 4 \quad (2,0) \mapsto 5 \quad \dots$$

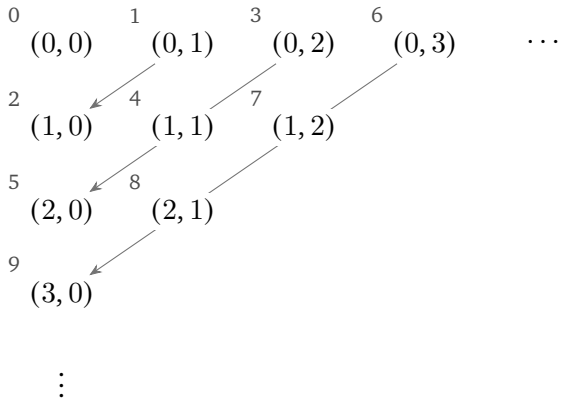


FIGURA 1.2 *Enumerazione delle coppie di $\mathbb{N} \times \mathbb{N}$ per diagonali.*

Poiché $\mathbb{N} \times \mathbb{N}$ e $\mathbb{N}$ sono in biezione, lo sono anche i loro insiemi delle parti. Quindi

$$|\{f \mid f \colon \mathbb{N} \to \mathbb{N}\}| = |\mathcal{P}(\mathbb{N} \times \mathbb{N})| = |\mathcal{P}(\mathbb{N})|,$$

ovvero esistono *tante funzioni quanti sono i sottoinsiemi di* $\mathbb{N}$.

1.3 QUANTI ALGORITMI ESISTONO?

Sia Σ un *alfabeto*, ovvero un insieme finito e non vuoto di simboli: quelli utilizzati per scrivere programmi e algoritmi. Un *programma* è una *stringa* (sequenza di simboli) su Σ, quindi

$$|\text{Programmi}| = |\text{stringhe su } \Sigma| = |\Sigma^*|,$$

[1] Non è invece possibile numerare le coppie riga per riga, $(0,0) \mapsto 0$, $(0,1) \mapsto 1$, $(0,2) \mapsto 2$, …: la riga delle coppie che iniziano con 0 è infinita, quindi la coppia $(1,0)$ non riceverebbe mai un numero.

dove l'operatore * è la *chiusura di Kleene*: Σ^* è l'insieme di tutte le stringhe ottenute concatenando zero o più simboli di Σ. Ogni stringa di Σ^* è quindi *finita*, anche se l'insieme è infinito:

$$\underbrace{\Sigma = \{a, b\}}_{\text{finito}} \qquad \underbrace{\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}}_{\text{infinito}}$$

dove ε è la *stringa vuota*.

Σ^* può essere messo in biezione con $\mathbb{N}$ poiché i suoi elementi sono ordinabili (prima per lunghezza, poi in ordine alfabetico, come nell'esempio): è sufficiente numerarli secondo tale ordine. Quindi Σ^* è numerabile, $|\Sigma^*| = |\mathbb{N}|$, e

$$|\text{Programmi}| = |\Sigma^*| = |\mathbb{N}|.$$

1.4 Il teorema di Cantor

Resta da confrontare $|\mathbb{N}|$, la cardinalità degli algoritmi, con $|\mathcal{P}(\mathbb{N})|$, la cardinalità delle funzioni.

Teorema 1 (di Cantor). *Per ogni insieme S si ha che*

$$|S| < |\mathcal{P}(S)|.$$

Per gli insiemi finiti il teorema è verificabile direttamente, come mostrato nella Tabella 1.1.

Insieme S	Insieme delle parti $\mathcal{P}(S)$	$\lvert S\rvert$	$\lvert\mathcal{P}(S)\rvert$
$\emptyset$	$\{\emptyset\}$	0	1
$\{a, b\}$	$\{\emptyset, \{a\}, \{b\}, \{a, b\}\}$	2	4
$\{a, b, c\}$	$\{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\}\}$	3	8

Tabella 1.1 *Cardinalità di alcuni insiemi finiti e dei loro insiemi delle parti.*

Dimostrazione del Teorema 1. Si suppone per assurdo che $|S| = |\mathcal{P}(S)|$. Allora esiste una funzione biettiva $f : S \to \mathcal{P}(S)$, che associa a ogni elemento di S un sottoinsieme di S stesso.

Si consideri l'insieme $A = \{x \in S \mid x \notin f(x)\} \subseteq S$. Poiché $\mathcal{P}(S)$ contiene ogni sottoinsieme di S, si ha $A \in \mathcal{P}(S)$ e, dato che f è suriettiva, esiste $a \in S$ tale che $f(a) = A$. Si hanno quindi due possibilità:

- $a \in A$: allora, per definizione di A, $a \notin f(a) = A$, assurdo ($\lightning$);
- $a \notin A$: allora, per definizione di A, $a \in f(a) = A$, assurdo ($\lightning$).

In entrambi i casi si giunge a una contraddizione, quindi non esiste $a \in S$ tale che $f(a) = A$: l'insieme A non ha controimmagine e f non può essere suriettiva. Di conseguenza $|S| \neq |\mathcal{P}(S)|$ e, poiché $x \mapsto \{x\}$ è una funzione iniettiva da S in $\mathcal{P}(S)$, $|S| < |\mathcal{P}(S)|$. □

In generale

$$|\mathcal{P}(S)| = 2^{|S|}.$$

Per $\mathbb{N}$ vale $|\mathcal{P}(\mathbb{N})| = |\mathbb{R}|$: i sottoinsiemi di $\mathbb{N}$ sono tanti quanti i numeri reali, che *non sono numerabili*. L'*ipotesi del continuo* afferma inoltre che non esiste alcun insieme con cardinalità strettamente compresa tra $|\mathbb{N}|$ e $|\mathbb{R}|$.

Riassumendo:

$$|\text{Programmi}| = |\mathbb{N}| < |\mathcal{P}(\mathbb{N})| = |\{f \mid f : \mathbb{N} \to \mathbb{N}\}|.$$

Poiché la cardinalità dei programmi è strettamente inferiore a quella delle funzioni, esistono *funzioni non calcolabili*.

2 Induzione

2.1 Relazioni ben fondate

Definizione (Relazione ben fondata). *Sia A un insieme e $\prec \subseteq A \times A$ una relazione su A, antisimmetrica e transitiva. La relazione $\prec$ è* ben fondata *se non esistono* catene discendenti infinite*, ovvero sequenze infinite $a_0 \succ a_1 \succ a_2 \succ \ldots$ di elementi di A.*

Ne segue che, se A non è vuoto, esistono *elementi minimali*, ovvero elementi di A privi di elementi più piccoli. L'insieme degli elementi minimali di A si indica con $\mathrm{Base}(A)$:

$$\mathrm{Base}(A) = \{\, a \in A \mid \nexists b \in A.\ b \prec a \,\}.$$

La situazione è illustrata nella Figura 2.1.

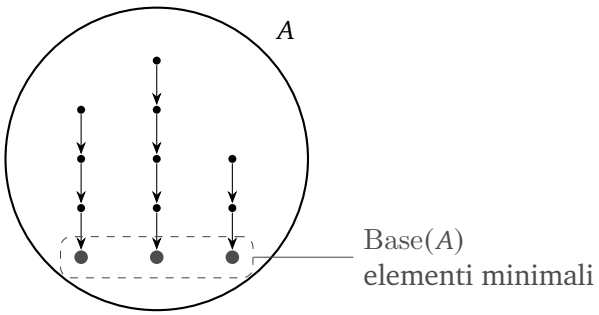


Figura 2.1 *Catene discendenti in un insieme ben fondato e suoi elementi minimali.*

2.2 Principio di induzione

Teorema 2 (Principio di induzione). *Sia $(A, \prec)$ ben fondato e sia Π una proprietà sugli elementi di A. Allora $\forall a \in A.\ \Pi(a)$ se e solo se valgono:*

(a) Base dell'induzione*: $\forall a \in \mathrm{Base}(A).\ \Pi(a)$;*
(b) Passo induttivo*: per ogni $a \in A \setminus \mathrm{Base}(A)$*

$$\underbrace{\big(\forall b \prec a.\ \Pi(b)\big)}_{\textit{ipotesi induttiva}} \implies \underbrace{\Pi(a)}_{\textit{tesi}}$$

Esempio. Si vuole dimostrare che

$$\forall n \geq 1. \quad \sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Sia $\Pi(n)$ tale uguaglianza. In questo caso $A = \mathbb{N} \setminus \{0\}$ con l'ordine usuale $<$, che è ben fondato, e $\mathrm{Base}(A) = \{1\}$.

Base. Per $n = 1$, $\Pi(1)$ diventa:

$$\sum_{i=1}^{1} i = 1 \qquad \frac{1 \cdot (1+1)}{2} = \frac{1 \cdot 2}{2} = 1.$$

I due membri coincidono, dunque $\Pi(1)$ è verificata.

Passo induttivo. Sia $n \in \mathbb{N} \setminus \{0, 1\}$ e si assuma l'*ipotesi induttiva*, ovvero che $\Pi(m)$ valga per ogni $m < n$. In particolare vale $\Pi(n-1)$, utilizzata nel secondo passaggio:

$$\begin{aligned}
\sum_{i=1}^{n} i &= \sum_{i=1}^{n-1} i + n \\
&= \frac{(n-1)(n-1+1)}{2} + n && \text{(ipotesi induttiva)} \\
&= \frac{(n-1)n + 2n}{2} \\
&= \frac{n(n-1+2)}{2} \\
&= \frac{n(n+1)}{2},
\end{aligned}$$

che è esattamente $\Pi(n)$. ◇

3 Linguaggi formali

3.1 Alfabeti e stringhe

Definizione (Alfabeto). *Un* alfabeto Σ *è un insieme finito e non vuoto di simboli, ovvero di entità primitive come lettere e caratteri.*

Definizione (Stringa). *Una* stringa*, o* parola*, è una sequenza finita di simboli su un alfabeto* Σ*.*

Come visto nella Sezione 1.3, l'insieme di tutte le stringhe su Σ si indica con Σ^*. Esso contiene sempre la *stringa vuota* ε, priva di simboli, ed è infinito numerabile.

Definizione (Lunghezza di una stringa). *La* lunghezza $|x|$ *di una stringa* x *è il numero di* occorrenze *di simboli al suo interno.*

Si contano le occorrenze e non i simboli: ad esempio $|abba| = 4$, anche se $abba$ contiene solo due simboli distinti. La stringa vuota ha lunghezza $|\varepsilon| = 0$.

Definizione (Prefisso, suffisso e sottostringa). *Sia* $x = a_1 a_2 \cdots a_n$ *una stringa su* Σ*, con* $a_1, \ldots, a_n \in \Sigma$*, e siano* $1 \leq i \leq j \leq n$*. Allora:*

- $a_1 \cdots a_i$*, la parte iniziale di* x*, è un* prefisso *di* x*;*
- $a_j \cdots a_n$*, la parte finale di* x*, è un* suffisso *di* x*;*
- $a_i \cdots a_j$*, una qualsiasi parte contigua di* x*, è una* sottostringa *di* x*.*

Anche la stringa vuota ε è prefisso, suffisso e sottostringa di ogni stringa. Come mostra la Figura 3.1, ogni prefisso e ogni suffisso di x sono anche sue sottostringhe.

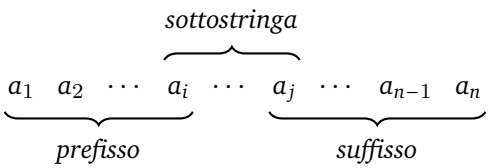


Figura 3.1 *Prefisso* $a_1 \cdots a_i$*, sottostringa* $a_i \cdots a_j$ *e suffisso* $a_j \cdots a_n$ *della stringa* $x = a_1 a_2 \cdots a_n$*.*

3.2 LINGUAGGI

DEFINIZIONE (Linguaggio). *Un* linguaggio $\mathcal{L}$ *su un alfabeto* Σ *è un sottoinsieme delle stringhe di* Σ^**, ovvero* $\mathcal{L} \subseteq \Sigma^*$.

Alcuni esempi di linguaggi sono $\emptyset$, dato che è sottoinsieme di qualsiasi insieme, e $\{\varepsilon\}$, il linguaggio che contiene solo la stringa vuota. I due non vanno confusi: $\emptyset$ non contiene alcuna stringa, mentre $\{\varepsilon\}$ ne contiene esattamente una, di lunghezza 0. Entrambi sono linguaggi su qualsiasi alfabeto, mentre Σ^* è il linguaggio di tutte le stringhe su Σ.

Poiché un linguaggio è, di fatto, un insieme, su di esso si possono eseguire le operazioni insiemistiche della Sezione 0.2.2, con Σ^* come insieme universo:

- COMPLEMENTO di $\mathcal{L} \subseteq \Sigma^*$: $\overline{\mathcal{L}} = \neg\mathcal{L} = \Sigma^* \setminus \mathcal{L}$;
- UNIONE di $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$: $\mathcal{L}_1 \cup \mathcal{L}_2 = \{x \in \Sigma^* \mid x \in \mathcal{L}_1 \vee x \in \mathcal{L}_2\}$;
- INTERSEZIONE di $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$: $\mathcal{L}_1 \cap \mathcal{L}_2 = \{x \in \Sigma^* \mid x \in \mathcal{L}_1 \wedge x \in \mathcal{L}_2\}$.

Il complemento è quindi sempre relativo a Σ^*. Ad esempio, il complemento del linguaggio vuoto è il linguaggio di tutte le stringhe, mentre quello di $\{\varepsilon\}$ è il linguaggio delle stringhe non vuote:

$$\overline{\emptyset} = \Sigma^* \setminus \emptyset = \Sigma^*, \qquad \overline{\{\varepsilon\}} = \{x \in \Sigma^* \mid x \neq \varepsilon\} = \Sigma^* \setminus \{\varepsilon\}.$$

Altre operazioni, come la concatenazione e la chiusura di Kleene, sono invece specifiche dei linguaggi e sono descritte nella Sezione 3.7.

3.3 FUNZIONI COME LINGUAGGI

Una funzione $f\colon \mathbb{N} \to \mathbb{N}$ può essere vista come un linguaggio $\mathcal{L}_f$ su un alfabeto Σ composto da un singolo simbolo.

Esempio. Si consideri l'alfabeto $\Sigma = \{1\}$ e una funzione $f\colon \mathbb{N} \to \mathbb{N}$. Allora

$$f\colon \mathbb{N} \to \mathbb{N} \quad \longleftrightarrow \quad \mathcal{L}_f = \{1^{f(x)} \mid x \in \mathbb{N}\},$$

dove 1^n indica la stringa formata da n occorrenze del simbolo 1, con $1^0 = \varepsilon$. ◇

Questo linguaggio permette di porre una domanda con risposta sì o no: data in input una stringa $\sigma \in \Sigma^*$, stabilire se essa appartenga o meno al linguaggio,

$$\sigma \in \Sigma^* \longmapsto \begin{cases} \sigma \in \mathcal{L}_f & \text{se } \sigma \text{ appartiene al linguaggio,} \\ \sigma \notin \mathcal{L}_f & \text{se } \sigma \text{ non appartiene al linguaggio.} \end{cases}$$

Si ragiona così su insiemi invece che su funzioni, dove gli elementi dell'insieme dipendono dal calcolo della funzione.

Esempio. Per $f(x) = 2x$ si ottiene

$$\mathcal{L}_f = \{1^{2x} \mid x \in \mathbb{N}\} = \{\varepsilon, 11, 1111, 111111, \dots\},$$

il linguaggio delle stringhe di lunghezza pari su $\{1\}$: ad esempio $1111 \in \mathcal{L}_f$, poiché $f(2) = 4$, mentre $111 \notin \mathcal{L}_f$, poiché nessun x ha $f(x) = 3$. ◇

3.4 Automi a stati finiti deterministici

I *linguaggi regolari* sono i linguaggi per cui esiste un automa a stati finiti deterministico che li riconosce.

Un *automa a stati finiti* è un modello matematico di un sistema con input, ed eventualmente output, a valori discreti. Il sistema si trova sempre in uno stato, scelto in un insieme finito di stati possibili, e l'essere in uno stato gli permette di tenere traccia della storia precedente.

Un automa a stati finiti si può rappresentare come una testina che scorre, sempre nella stessa direzione, un nastro di lunghezza illimitata contenente dei simboli. La testina si trova in un certo *stato* q e, a seconda del simbolo s_i che legge, passa in un altro stato oppure resta nello stesso. La situazione è illustrata nella Figura 3.2.

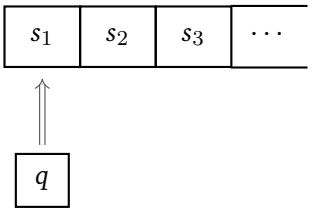


Figura 3.2 *Automa a stati finiti: la testina, nello stato q, legge il primo simbolo s_1 del nastro.*

Quando la lettura dei simboli termina, a seconda dello stato raggiunto dalla testina, l'automa fornisce un risultato di accettazione o di rifiuto della stringa letta.

Il comportamento dell'automa si definisce in maniera univoca mediante una tabella detta *matrice di transizione*.

Esempio. La matrice di transizione della Tabella 3.1 descrive un automa con stati q_0, q_1, q_2 sull'alfabeto $\{a, b\}$. Ogni riga corrisponde a uno stato e ogni colonna a un simbolo: la casella nella riga q e nella colonna a contiene lo stato in cui l'automa passa leggendo a dallo stato q. Ad esempio, dallo stato q_0, leggendo b, l'automa passa in q_2.

δ	a	b
q_0	q_1	q_2
q_1	q_1	q_0
q_2	q_1	q_0

Tabella 3.1 *Matrice di transizione di un automa con tre stati.*

$\diamond$

In alternativa, l'automa si rappresenta con un grafo: gli stati sono i nodi e ogni transizione è un arco etichettato con il simbolo letto, come negli esempi seguenti.

Definizione (Automa deterministico). *Un* automa a stati finiti deterministico *(DFA) è una quintupla $\langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$, dove:*

- Q *è un insieme finito di* stati*;*
- Σ *è un alfabeto;*
- $\delta \colon Q \times \Sigma \to Q$ *è la* funzione di transizione*;*
- $q_0 \in Q$ *è lo stato* iniziale*;*

- $\mathcal{F} \subseteq Q$ *è l'insieme degli stati* finali.

La funzione di transizione di un DFA è deterministica: per ogni stato e per ogni simbolo esiste sempre una e una sola transizione,

$$\forall q \in Q.\ \forall a \in \Sigma.\ \exists!\, p \in Q.\ \delta(q, a) = p.$$

L'unica forma di memoria dell'automa è lo stato in cui si trova.

Notazione. Si useranno p, q, r per denotare gli stati, P, Q, R, S per gli insiemi di stati, a, b per i simboli di Σ e x, y, z, u, v, w per le stringhe, tutti con o senza pedici.

Poiché la funzione δ riceve in ingresso un solo simbolo, è necessario estenderla per poter leggere stringhe. A questo scopo si definisce la funzione $\hat{\delta}$, che si ottiene in modo univoco da δ:

$$\hat{\delta}\colon Q \times \Sigma^* \to Q, \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = q, \\ \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a). \end{cases}$$

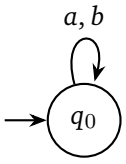
Una stringa x è *accettata*, o *riconosciuta*, da un DFA $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ se $\hat{\delta}(q_0, x) \in \mathcal{F}$. Il linguaggio riconosciuto (o accettato) da $\mathcal{M}$, indicato con $\mathcal{L}(\mathcal{M})$, è l'insieme delle stringhe accettate:

$$\mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \in \mathcal{F}\}.$$

Definizione (Linguaggio regolare). *Un linguaggio $\mathcal{L}$ è detto* regolare *se è accettato da qualche DFA, ovvero se esiste un DFA $\mathcal{M}$ tale che*

$$\mathcal{L} = \mathcal{L}(\mathcal{M}).$$

Esempio. Il linguaggio vuoto $\mathcal{L} = \emptyset$ è regolare. Lo riconosce il DFA $\mathcal{M}$ della Figura 3.3, con $Q = \{q_0\}$, $\Sigma = \{a, b\}$, stato iniziale q_0 e $\mathcal{F} = \emptyset$.



δ	a	b
q_0	q_0	q_0

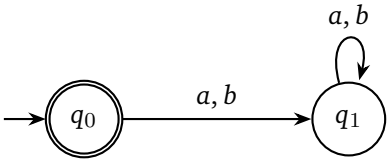
Figura 3.3 *DFA che riconosce il linguaggio vuoto e la sua matrice di transizione.*

Qualunque simbolo legga, l'automa resta in q_0, quindi $\hat{\delta}(q_0, x) = q_0$ per ogni $x \in \Sigma^*$. Poiché $\mathcal{F} = \emptyset$, però, la condizione di accettazione $\hat{\delta}(q_0, x) \in \mathcal{F}$ è sempre falsa: nessuno stato appartiene all'insieme vuoto. Nessuna stringa viene quindi accettata e

$$\mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \underbrace{\hat{\delta}(q_0, x) \in \mathcal{F}}_{\text{sempre falsa, poiché } \mathcal{F} = \emptyset}\} = \emptyset.$$

Riconoscere $\emptyset$ non significa che l'automa non esista, ma che non accetta alcuna stringa: in generale, ogni DFA con $\mathcal{F} = \emptyset$ riconosce il linguaggio vuoto. ◇

Esempio. Il linguaggio $\mathcal{L} = \{\varepsilon\}$ è regolare. Lo riconosce il DFA $\mathcal{M}$ della Figura 3.4, con $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, stato iniziale q_0 e $\mathcal{F} = \{q_0\}$.



δ	a	b
q_0	q_1	q_1
q_1	q_1	q_1

Figura 3.4 *DFA che riconosce il linguaggio $\{\varepsilon\}$ e la sua matrice di transizione: lo stato finale q_0 è indicato dal doppio cerchio.*

Lo stato iniziale è anche finale, quindi la stringa vuota, che non fa compiere alcuna transizione, viene accettata. Il primo simbolo letto porta invece in q_1, da cui l'automa non esce più:

$$\hat{\delta}(q_0, \varepsilon) = q_0 \in \mathcal{F}, \qquad \hat{\delta}(q_0, x) = q_1 \notin \mathcal{F} \quad \text{per ogni } x \neq \varepsilon.$$

Quindi $\mathcal{L}(\mathcal{M}) = \{\varepsilon\}$. ◇

3.4.1 Come dimostrare che un linguaggio è regolare

Un modo per dimostrare che un linguaggio è regolare è costruire un DFA che lo riconosca. L'uguaglianza $\mathcal{L} = \mathcal{L}(\mathcal{M})$ verrà dimostrata mediante la duplice inclusione $\mathcal{L} \subseteq \mathcal{L}(\mathcal{M})$ e $\mathcal{L}(\mathcal{M}) \subseteq \mathcal{L}$. In particolare:

$$\begin{aligned}
& \mathcal{L}(\mathcal{M}) \subseteq \mathcal{L} \\
\iff & \forall x \in \Sigma^*.\ \big(x \in \mathcal{L}(\mathcal{M}) \implies x \in \mathcal{L}\big) \\
\iff & \hat{\delta}(q_0, x) \in \mathcal{F} \implies x \in \mathcal{L} && \text{(definizione di } \mathcal{L}(\mathcal{M}), \text{ con } x \text{ arbitrario)} \\
\iff & x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F} && \text{(per contrapposizione)}
\end{aligned}$$

e

$$\begin{aligned}
& \mathcal{L} \subseteq \mathcal{L}(\mathcal{M}) \\
\iff & \forall x \in \Sigma^*.\ \big(x \in \mathcal{L} \implies x \in \mathcal{L}(\mathcal{M})\big) \\
\iff & x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F} && \text{(definizione di } \mathcal{L}(\mathcal{M}), \text{ con } x \text{ arbitrario)}
\end{aligned}$$

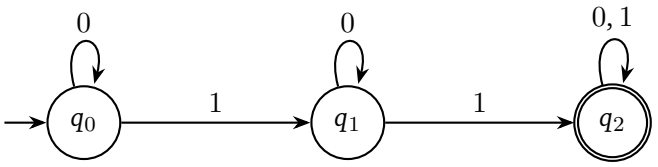
Riassumendo:

$$\begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F}, \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F}. \end{cases}$$

Dal momento che le condizioni da dimostrare sono analoghe, verranno portate avanti insieme ed è quindi necessario che la base dell'induzione sia la stessa per entrambe. In particolare, è richiesto di trovare una lunghezza tale per cui esistano stringhe in $\mathcal{L}$ e stringhe non in $\mathcal{L}$. I passi sono dunque:

- Base: si considera la lunghezza minima di x e, per tutte le lunghezze inferiori o uguali a quella, si dimostra che valgono entrambe le condizioni;
- Passo induttivo: si suppone che entrambe le condizioni valgano per ogni x tale che $|x| \leq n$ e si dimostra che valgono anche per $|x| = n + 1$.

Esempio. Si consideri il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid \text{“}x \text{ contiene almeno due } 1\text{”}\}$ definito sull'alfabeto binario $\Sigma = \{0, 1\}$. L'automa a stati finiti che riconosce $\mathcal{L}$ è quello della Figura 3.5, con stato iniziale q_0 e $\mathcal{F} = \{q_2\}$.



δ	0	1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_2

FIGURA 3.5 *DFA che riconosce le stringhe binarie con almeno due* 1 *e la sua matrice di transizione.*

È comunque necessario dimostrare che vengono riconosciute tutte e sole le stringhe di $\mathcal{L}$. In particolare, bisogna dimostrare che

$$\mathcal{L} = \mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \in \{q_2\}\}.$$

La dimostrazione avviene per induzione. La base non può riguardare *solo* stringhe con meno di due simboli: servono almeno due simboli per comporre una stringa $z \in \mathcal{L}$.[1] Quindi:

$$\forall x \in \Sigma^*.\ \big(|x| < 2 \implies x \notin \mathcal{L}\big)$$

Allora:

$$\left.\begin{array}{lll}
|x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} \\
|x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 \notin \mathcal{F} \\
 & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 \notin \mathcal{F} \\
|x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 \notin \mathcal{F} \\
 & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 \notin \mathcal{F} \\
 & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 \notin \mathcal{F} \\
 & x = 11 \in \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 \in \mathcal{F}
\end{array}\right\} \text{ BASE}$$

PASSO INDUTTIVO. *Ipotesi induttiva*: si suppone che per tutte le stringhe x, con $|x| \geq 2$, di lunghezza $|x| \leq n$, per un certo $n \in \mathbb{N}$, valga che

$$\begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F} \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F} \end{cases}$$

dove si è usato che $\mathcal{F} = \{q_2\}$. La seconda condizione, però, non basta: dice solo che l'automa è in q_0 oppure in q_1, mentre per leggere il simbolo successivo serve sapere in quale dei due. La si scorpora quindi in due condizioni più precise, ottenendo l'ipotesi

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) = q_2, \\ x \notin \mathcal{L} \wedge \text{“}x \text{ non contiene } 1\text{”} \implies \hat{\delta}(q_0, x) = q_0, \\ x \notin \mathcal{L} \wedge \text{“}x \text{ contiene un solo } 1\text{”} \implies \hat{\delta}(q_0, x) = q_1. \end{cases}$$

[1] In realtà, non è mai possibile considerare $|x| = 0$ come unico caso base, perché l'unica stringa di tale lunghezza, ε, o appartiene a un linguaggio o non vi appartiene. Servono, invece, lunghezze tali per cui esistono stringhe sia appartenenti che non.

È lecito farlo perché ogni nuova condizione implica quella di partenza, dato che q_0 e q_1 sono diversi da q_2, e la base verifica già gli stati esatti. In cambio, nel passo induttivo va dimostrata per x l'intera $\Psi(x)$.

Si dimostri allora il caso base per Ψ:

$$\left.\begin{array}{lllll}
|x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 & \text{(caso 2 di } \Psi) & \\
|x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 & \text{(caso 2 di } \Psi) & \\
 & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 & \text{(caso 3 di } \Psi) & \\
|x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 & \text{(caso 2 di } \Psi) & \\
 & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 & \text{(caso 3 di } \Psi) & \\
 & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 & \text{(caso 3 di } \Psi) & \\
 & x = 11 \in \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 & \text{(caso 1 di } \Psi) &
\end{array}\right\} \text{Base}$$

Passo induttivo. Per verificare $\Psi(x)$ su tutte le stringhe di Σ^* si applica l'ipotesi induttiva

$$\forall x \in \Sigma^*.\ \big(|x| \leq n \implies \Psi(x)\big)$$

Sia x una stringa con $|x| = n + 1$: allora $x = x'0$ oppure $x = x'1$, con $|x'| = n$, e a x' si può applicare l'ipotesi induttiva. Si distingue un caso per ciascuna condizione di $\Psi(x)$. In ogni caso si ragiona prima sul linguaggio, contando gli 1 di x'; poi si applica l'ipotesi induttiva a x' e infine si legge l'ultimo simbolo. Nei calcoli, *def* indica la definizione di $\hat{\delta}$ e *Hp* il punto in cui si sostituisce lo stato dato dall'ipotesi induttiva.

1. $x \in \mathcal{L}$, quindi x contiene almeno due 1.

   1a. $x = x'0$. Se x' contenesse un solo 1 (o nessuno), anche $x = x'0$ ne conterrebbe meno di due e quindi $x \notin \mathcal{L}$: assurdo. Quindi x' contiene almeno due 1, cioè $x' \in \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_2$. Allora

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_2, 0) = q_2 \in \mathcal{F}.$$

   1b. $x = x'1$. Se x' non contenesse 1, $x = x'1$ ne conterrebbe uno solo e quindi $x \notin \mathcal{L}$: assurdo. Quindi x' contiene almeno due 1, e quindi $x' \in \mathcal{L}$, oppure ne contiene esattamente uno, e quindi $x' \notin \mathcal{L}$.
   Se $x' \in \mathcal{L}$, per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$, quindi

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_2, 1) = q_2 \in \mathcal{F}.$$

   Se $x' \notin \mathcal{L}$, x' contiene un solo 1 e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_1$: è qui che serve lo scorporo, perché altrimenti si saprebbe solo che $\hat{\delta}(q_0, x') \neq q_2$. Quindi

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_1, 1) = q_2 \in \mathcal{F}.$$

2. $x \notin \mathcal{L}$ e x non contiene 1. Il caso $x = x'1$ è assurdo, perché x conterrebbe un 1, quindi $x = x'0$. Se x' contenesse almeno un 1, lo conterrebbe anche x: assurdo. Perciò x' non contiene 1: $x' \notin \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_0$. Allora

   $$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_0, 0) = q_0.$$

3. $x \notin \mathcal{L}$ e x contiene un solo 1.

3a. $x = x'1$. Se x' contenesse almeno un 1, $x = x'1$ ne conterrebbe almeno due e quindi $x \in \mathcal{L}$: assurdo. Perciò x' non contiene 1: $x' \notin \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_0$. Allora

$$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_0, 1) = q_1.$$

3b. $x = x'0$. Se x' non contenesse 1, nemmeno $x = x'0$ ne conterrebbe; se ne contenesse almeno due, $x \in \mathcal{L}$. In entrambi i casi è assurdo, perciò x' contiene un solo 1: $x' \notin \mathcal{L}$ e, per ipotesi induttiva, $\hat{\delta}(q_0, x') = q_1$. Allora

$$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_1, 0) = q_1.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$, quindi $\Psi(x)$ vale anche per $|x| = n + 1$. Per induzione vale per ogni $x \in \Sigma^*$: in particolare $\hat{\delta}(q_0, x) \in \mathcal{F}$ se e solo se $x \in \mathcal{L}$, ovvero $\mathcal{L}(\mathcal{M}) = \mathcal{L}$. ◇

3.5 Automi a stati finiti non deterministici

In un *automa a stati finiti non deterministico* (NFA) l'unico elemento che cambia rispetto ai DFA è la funzione di transizione. L'obiettivo è stabilire se il non determinismo aumenti la potenza di calcolo.

Anche in un NFA l'unica forma di memoria è lo stato in cui l'automa si trova, ma la funzione di transizione è non deterministica: da uno stato, leggendo un simbolo, l'automa può eseguire zero, una o più transizioni, cioè raggiungere un qualsiasi insieme di stati, anche vuoto.

Definizione (Automa non deterministico). *Un* automa a stati finiti non deterministico *(NFA) è una quintupla $\langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$, dove:*

- Q *è un insieme finito di stati;*
- Σ *è un alfabeto;*
- $\delta \colon Q \times \Sigma \to \mathcal{P}(Q)$ *è la* funzione di transizione*, che associa a ogni stato e a ogni simbolo l'insieme degli stati raggiungibili;*
- $q_0 \in Q$ *è lo stato* iniziale*;*
- $\mathcal{F} \subseteq Q$ *è l'insieme degli stati* finali.

Come per i DFA, la funzione δ va estesa alle stringhe. La funzione $\hat{\delta}$, che si ottiene in modo univoco da δ, è definita come segue:

$$\hat{\delta} \colon Q \times \Sigma^* \to \mathcal{P}(Q), \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = \{q\}, \\ \hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \delta(p, a). \end{cases}$$

Ad esempio, si supponga che, leggendo w a partire da q, l'automa possa arrivare negli stati p_1, p_2 e p_3, ovvero $\hat{\delta}(q, w) = \{p_1, p_2, p_3\}$. Leggendo poi il simbolo a, da ciascuno di essi si raggiunge l'insieme $\delta(p_i, a)$, e $\hat{\delta}(q, wa)$ è l'unione di questi insiemi, come mostrato nella Figura 3.6:

$$\hat{\delta}(q, wa) = \delta(p_1, a) \cup \delta(p_2, a) \cup \delta(p_3, a).$$

Nella figura $\delta(p_1, a) = \{r_1, r_2, r_3\}$, $\delta(p_2, a) = \{r_4, r_5, r_6\}$ e $\delta(p_3, a) = \{r_7, r_8\}$, quindi

$$\hat{\delta}(q, wa) = \{r_1, r_2, r_3\} \cup \{r_4, r_5, r_6\} \cup \{r_7, r_8\} = \{r_1, \ldots, r_8\}.$$

In generale gli insiemi $\delta(p_i, a)$ possono avere stati in comune, che nell'unione compaiono una volta sola. Leggendo wa un simbolo alla volta, l'automa attraversa quindi gli insiemi di stati

$$\hat{\delta}(q, \varepsilon) = \{q\} \xrightarrow{w} \hat{\delta}(q, w) = \{p_1, p_2, p_3\} \xrightarrow{a} \hat{\delta}(q, wa) = \{r_1, \ldots, r_8\}.$$

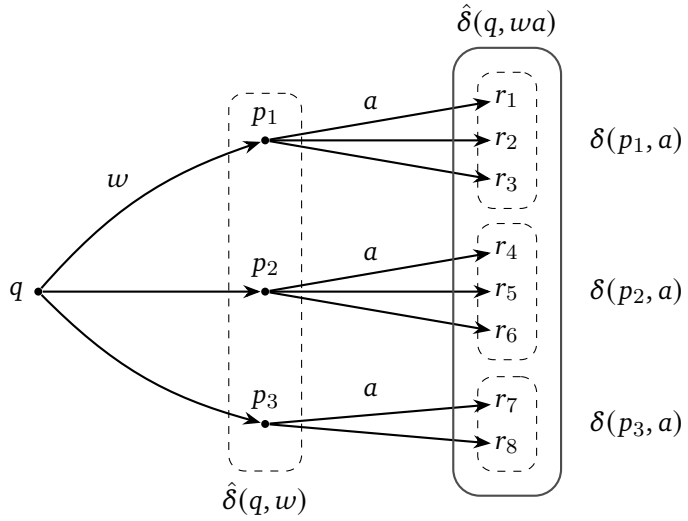


Figura 3.6 *Calcolo di $\hat{\delta}(q, wa)$ in un NFA: l'unione degli insiemi $\delta(p_i, a)$, per ogni $p_i \in \hat{\delta}(q, w)$.*

Il linguaggio accettato da un NFA $\mathcal{N} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ è l'insieme delle stringhe per cui almeno uno degli stati raggiunti è finale:

$$\mathcal{L}(\mathcal{N}) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \cap \mathcal{F} \neq \emptyset\}.$$

Teorema 3 (di Rabin e Scott, 1959).

(a) *Sia $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ un DFA. Allora esiste un NFA $\mathcal{N} = \langle Q', \Sigma, \delta', q_0, \mathcal{F} \rangle$ che riconosce lo stesso linguaggio, ovvero $\mathcal{L}(\mathcal{M}) = \mathcal{L}(\mathcal{N})$.*

(b) *Sia $\mathcal{N} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ un NFA. Allora esiste un DFA $\mathcal{M} = \langle Q', \Sigma, \delta', q_0', \mathcal{F}' \rangle$ che riconosce lo stesso linguaggio, ovvero $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{M})$.*

Dimostrazione del Teorema 3. L'obiettivo è dimostrare che NFA e DFA sono equivalenti.

Punto (a). Un DFA è un caso particolare di NFA, in cui ogni transizione porta in un solo stato. Si definisce quindi $\mathcal{N}$ con $Q' = Q$, gli stessi q_0 e $\mathcal{F}$, e

$$\delta'(q, a) = \{\delta(q, a)\}.$$

Per induzione su $|x|$ si verifica che $\hat{\delta}'(q_0, x) = \{\hat{\delta}(q_0, x)\}$. Per $x = \varepsilon$ entrambi i membri valgono $\{q_0\}$; per $x = wa$, con $|w| = n$, l'ipotesi induttiva dice che $\hat{\delta}'(q_0, w) = \{\hat{\delta}(q_0, w)\}$,

quindi

$$\begin{aligned}
\hat{\delta}'(q_0, wa) &= \bigcup_{p \in \hat{\delta}'(q_0,w)} \delta'(p,a) && \text{(definizione di } \hat{\delta}' \text{ per gli NFA)} \\
&= \bigcup_{p \in \{\hat{\delta}(q_0,w)\}} \delta'(p,a) && \text{(ipotesi induttiva)} \\
&= \delta'(\hat{\delta}(q_0,w),a) && \text{(unione di un solo stato)} \\
&= \{\delta(\hat{\delta}(q_0,w),a)\} && \text{(definizione di } \delta') \\
&= \{\hat{\delta}(q_0,wa)\} && \text{(definizione di } \hat{\delta}).
\end{aligned}$$

Quindi $\hat{\delta}'(q_0,x) \cap \mathcal{F} \neq \emptyset$ se e solo se $\hat{\delta}(q_0,x) \in \mathcal{F}$, ovvero $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{M})$.

Punto (b). L'idea è che ogni stato di $\mathcal{M}$ rappresenti l'insieme degli stati in cui $\mathcal{N}$ può trovarsi dopo aver letto una stringa. Si definisce quindi $\mathcal{M}$ come segue:

- $Q' = \mathcal{P}(Q)$: gli stati di $\mathcal{M}$ sono i sottoinsiemi di Q, che sono $2^{|Q|}$ e quindi in numero finito[2];
- $q_0' = \{q_0\}$: all'inizio $\mathcal{N}$ si trova solo in q_0;
- $\delta' : Q' \times \Sigma \to Q'$, con $\delta'(S,a) = \bigcup_{p \in S} \delta(p,a)$ per ogni $S \in Q'$ e $a \in \Sigma$: da S, leggendo a, si raggiungono tutti gli stati raggiungibili da almeno uno degli stati di S. Il risultato è un sottoinsieme di Q, cioè un solo stato di Q', quindi $\mathcal{M}$ è deterministico;
- $\mathcal{F}' = \{S \in Q' \mid S \cap \mathcal{F} \neq \emptyset\}$: un insieme di stati è finale se contiene almeno uno stato finale di $\mathcal{N}$.

Si dimostra per induzione su $|x|$ che, per ogni $x \in \Sigma^*$,

$$\hat{\delta}'(q_0', x) = \hat{\delta}(q_0, x),$$

cioè che dopo aver letto x l'automa $\mathcal{M}$ si trova nello stato che corrisponde esattamente all'insieme degli stati raggiunti da $\mathcal{N}$.

Base. Per $|x| = 0$ si ha $x = \varepsilon$ e

$$\hat{\delta}'(q_0', \varepsilon) = q_0' = \{q_0\} = \hat{\delta}(q_0, \varepsilon).$$

Passo induttivo. Si suppone che l'uguaglianza valga per tutte le stringhe di lunghezza n e sia $x = wa$, con $|w| = n$ e $a \in \Sigma$. Allora

$$\begin{aligned}
\hat{\delta}'(q_0', wa) &= \delta'(\hat{\delta}'(q_0', w), a) && \text{(definizione di } \hat{\delta}') \\
&= \delta'(\hat{\delta}(q_0,w),a) && \text{(ipotesi induttiva)} \\
&= \bigcup_{p \in \hat{\delta}(q_0,w)} \delta(p,a) && \text{(definizione di } \delta') \\
&= \hat{\delta}(q_0, wa) && \text{(definizione di } \hat{\delta} \text{ per gli NFA)}.
\end{aligned}$$

Infine, per ogni $x \in \Sigma^*$,

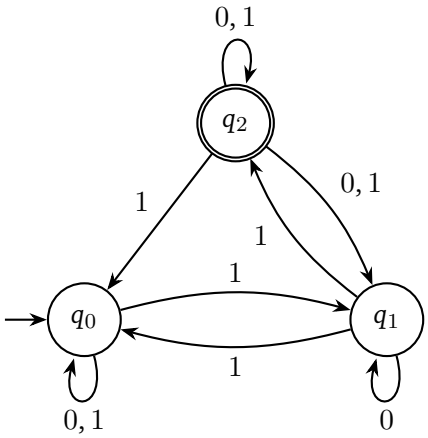
$$\begin{aligned}
x \in \mathcal{L}(\mathcal{M}) &\iff \hat{\delta}'(q_0', x) \in \mathcal{F}' \\
&\iff \hat{\delta}(q_0,x) \in \mathcal{F}' && \text{(uguaglianza appena dimostrata)} \\
&\iff \hat{\delta}(q_0,x) \cap \mathcal{F} \neq \emptyset && \text{(definizione di } \mathcal{F}') \\
&\iff x \in \mathcal{L}(\mathcal{N}),
\end{aligned}$$

[2]Sarebbe più preciso definire $Q' = \{q_1, \ldots, q_{2^{|Q|}}\}$ e stabilire una corrispondenza biunivoca tra questi stati e gli elementi di $\mathcal{P}(Q)$; usare direttamente i sottoinsiemi come stati rende però la dimostrazione molto più snella.

quindi $\mathcal{L}(\mathcal{M}) = \mathcal{L}(\mathcal{N})$. □

DFA e NFA riconoscono quindi la stessa classe di linguaggi, i linguaggi regolari: il non determinismo non aumenta la potenza di calcolo. In altre parole, la possibilità di raggiungere più stati diversi leggendo lo stesso simbolo *non* cambia la classe dei linguaggi riconoscibili.

Esempio. Si consideri l'NFA della Figura 3.7, con stato iniziale q_0 e unico stato finale q_2.



δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_1\}$	$\{q_0, q_2\}$
q_2	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$

Figura 3.7 *Un NFA e la sua matrice di transizione.*

Si applica la costruzione del punto (b) della dimostrazione del Teorema 3, che è del tutto meccanica: ogni sottoinsieme di $Q = \{q_0, q_1, q_2\}$ è uno stato del DFA, e la sua riga nella matrice di transizione si ottiene unendo le righe dell'NFA dei suoi elementi. Ad esempio,

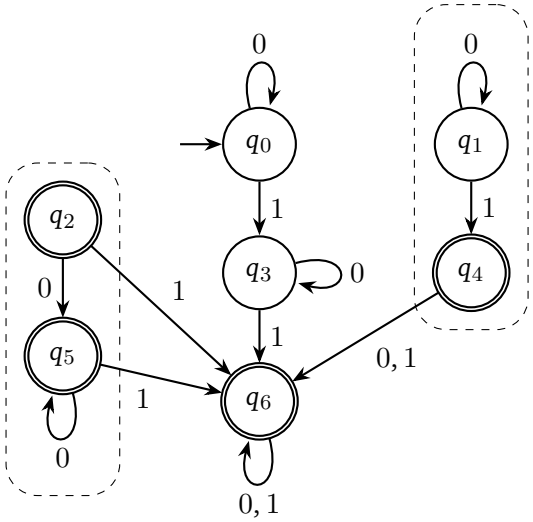
$$\delta'(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0, q_1\} \cup \{q_0, q_2\} = \{q_0, q_1, q_2\}.$$

Gli stati finali sono i sottoinsiemi che contengono q_2, l'unico stato finale dell'NFA. Si ottiene la Tabella 3.2, in cui a ogni sottoinsieme non vuoto si dà un nome q'_i.

Stato	Sottoinsieme	0	1	Finale
	$\emptyset$	$\emptyset$	$\emptyset$	
q'_0	$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$	
q'_1	$\{q_1\}$	$\{q_1\}$	$\{q_0, q_2\}$	
q'_2	$\{q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓
q'_3	$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	
q'_4	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓
q'_5	$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓
q'_6	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	✓

Tabella 3.2 *Costruzione per sottoinsiemi del DFA equivalente all'NFA della Figura 3.7.*

Sostituendo a ogni sottoinsieme il suo nome, e chiamando per semplicità q_i lo stato q'_i, si ottiene il DFA della Figura 3.8, in cui lo stato $\emptyset$ è omesso.



δ'	0	1
q_0	q_0	q_3
q_1	q_1	q_4
q_2	q_5	q_6
q_3	q_3	q_6
q_4	q_6	q_6
q_5	q_5	q_6
q_6	q_6	q_6

Figura 3.8 *Il DFA ottenuto dalla Tabella 3.2; i riquadri tratteggiati racchiudono gli stati non raggiungibili da q_0.*

È interessante notare che alcuni stati non sono raggiungibili da q_0. Gli stati q_1 e q_2 non hanno archi entranti da altri stati, mentre q_4 e q_5 ne hanno, ma solo da q_1 e da q_2. Quello che conta, infatti, non è avere archi entranti, ma l'esistenza di un cammino da q_0: gli unici stati raggiungibili sono q_0, q_3 e q_6.

Gli stati non raggiungibili non cambiano il linguaggio riconosciuto e si possono eliminare, ottenendo il DFA minimo della Figura 3.9.[3] È lo stesso automa dell'esempio sulle stringhe con almeno due 1 (Figura 3.5), con q_3 e q_6 al posto di q_1 e q_2: l'NFA di partenza riconosce quindi proprio quel linguaggio.

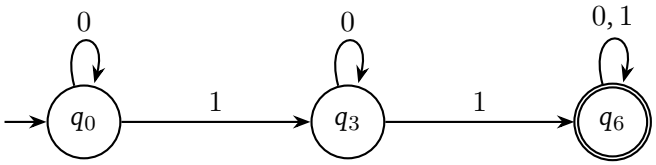


Figura 3.9 *Il DFA minimo, ottenuto eliminando gli stati non raggiungibili.*

◇

3.6 Automi a stati finiti non deterministici con ε-transizioni

Ci si chiede ora se la possibilità di cambiare stato *senza* leggere simboli cambi la classe dei linguaggi riconoscibili.

[3] In questo caso basta eliminare gli stati non raggiungibili; in generale, per ottenere il DFA minimo può servire anche unire stati equivalenti.

Definizione (Automa non deterministico con ε-transizioni). *Un* automa a stati finiti non deterministico con ε-transizioni *(ε-NFA) è una quintupla $\langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ che differisce da un NFA solo per la funzione di transizione:*

- *Q è un insieme finito di stati;*
- *Σ è un alfabeto;*
- *$\delta\colon Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ è la* funzione di transizione*: come negli NFA, da uno stato si raggiunge un insieme di stati, ma lo si può fare anche senza leggere alcun simbolo, seguendo un arco etichettato con ε (ε-*transizione*);*
- *$q_0 \in Q$ è lo stato* iniziale*;*
- *$\mathcal{F} \subseteq Q$ è l'insieme degli stati* finali*.*

Prima di estendere δ alle stringhe è necessario definire la funzione ε-closure$\colon Q \to \mathcal{P}(Q)$, che associa a ogni stato q l'insieme degli stati raggiungibili da q senza leggere simboli, ovvero seguendo zero o più archi etichettati con ε:

$$\varepsilon\text{-closure}(q) = \{p \in Q \mid p \text{ è raggiungibile da } q \text{ con zero o più } \varepsilon\text{-transizioni}\}.$$

Equivalentemente, ε-closure(q) è il più piccolo insieme che contiene q e che, per ogni suo stato p, contiene anche $\delta(p, \varepsilon)$. In particolare $q \in \varepsilon$-closure(q), perché q si raggiunge con zero ε-transizioni, e $\delta(q, \varepsilon) \subseteq \varepsilon$-closure$(q)$. L'inclusione può essere stretta: se $q \xrightarrow{\varepsilon} p \xrightarrow{\varepsilon} r$, allora $r \in \varepsilon$-closure(q), anche se r non appartiene necessariamente a $\delta(q, \varepsilon)$. Per un insieme di stati $S \subseteq Q$ si pone

$$\varepsilon\text{-closure}(S) = \bigcup_{s \in S} \varepsilon\text{-closure}(s).$$

Si può ora definire la funzione estesa $\hat{\delta}$:

$$\hat{\delta}\colon Q \times \Sigma^* \to \mathcal{P}(Q), \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = \varepsilon\text{-closure}(q), \\ \hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q,x)} \varepsilon\text{-closure}(\delta(p, a)). \end{cases}$$

La ricorsione avviene sulla stringa e non sugli stati: $\hat{\delta}(q, xa)$ richiama $\hat{\delta}(q, x)$ con lo stesso stato di partenza q e la stringa privata dell'ultimo simbolo, fino al caso base $\hat{\delta}(q, \varepsilon)$.

Esempio. Si consideri l'automa della Figura 3.10 e si calcoli $\hat{\delta}(q, a)$. Poiché $a = \varepsilon a$, si applica il secondo caso della definizione con $x = \varepsilon$.

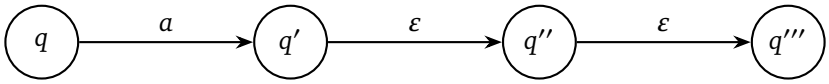


Figura 3.10 *Un automa con due ε-transizioni consecutive.*

Da q non escono archi etichettati con ε, quindi $\hat{\delta}(q, \varepsilon) = \varepsilon$-closure$(q) = \{q\}$ e l'unione è su un solo stato. Leggendo a si ottiene $\delta(q, a) = \{q'\}$, e infine si seguono tutte le catene di archi ε a partire da q':

$$\begin{aligned} \hat{\delta}(q, a) &= \bigcup_{p \in \hat{\delta}(q,\varepsilon)} \varepsilon\text{-closure}(\delta(p, a)) && \text{(definizione di } \hat{\delta}) \\ &= \varepsilon\text{-closure}(\{q'\}) && \text{(estensione di } \varepsilon\text{-closure agli insiemi)} \\ &= \{q', q'', q'''\}. \end{aligned}$$

Per calcolare la chiusura si parte da $\{q'\}$ e si aggiungono gli stati raggiungibili con un arco ε da quelli già presenti, finché non se ne trovano di nuovi: prima q'', poi q'''. Da q''' non escono archi ε, quindi ci si ferma. A differenza di DFA e NFA, $\hat{\delta}(q, a)$ può quindi essere diverso da $\delta(q, a) = \{q'\}$.

Le ε-transizioni contano anche prima di leggere un simbolo: in un automa $q \xrightarrow{\varepsilon} r \xrightarrow{a} s$ si ha $\delta(q, a) = \emptyset$, ma $\hat{\delta}(q, \varepsilon) = \{q, r\}$ e quindi $s \in \hat{\delta}(q, a)$. ◇

Esempio. Si calcoli $\hat{\delta}(q, ab)$ per l'automa della Figura 3.11, svolgendo per intero la ricorsione. Le transizioni dell'automa sono $\delta(q, a) = \{q'\}$, $\delta(q', \varepsilon) = \{q''\}$, $\delta(q'', b) = \{s\}$ e $\delta(s, \varepsilon) = \{t\}$; tutte le altre danno $\emptyset$.

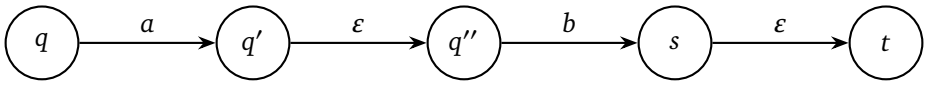


Figura 3.11 *Un automa in cui il simbolo b si può leggere solo dopo una ε-transizione.*

Discesa. Ogni chiamata toglie l'ultimo simbolo della stringa, fino al caso base:

$$\begin{aligned} \hat{\delta}(q, ab) &\qquad ab = a \cdot b\text{: si richiama } \hat{\delta}(q, a), \\ \hat{\delta}(q, a) &\qquad a = \varepsilon \cdot a\text{: si richiama } \hat{\delta}(q, \varepsilon), \\ \hat{\delta}(q, \varepsilon) &\qquad \text{caso base.} \end{aligned}$$

Risalita. Ogni livello legge il proprio simbolo e chiude con le ε-transizioni. Il caso base restituisce

$$\hat{\delta}(q, \varepsilon) = \varepsilon\text{-closure}(q) = \{q\}.$$

Il livello intermedio legge a:

$$\begin{aligned} \hat{\delta}(q, a) &= \bigcup_{p \in \{q\}} \varepsilon\text{-closure}(\delta(p, a)) && \text{(unione su un solo stato)} \\ &= \varepsilon\text{-closure}(\delta(q, a)) = \varepsilon\text{-closure}(\{q'\}) \\ &= \{q', q''\} && (\delta(q', \varepsilon) = \{q''\},\ \delta(q'', \varepsilon) = \emptyset). \end{aligned}$$

Il livello più esterno legge b, partendo dall'insieme appena restituito:

$$\begin{aligned} \hat{\delta}(q, ab) &= \bigcup_{p \in \{q', q''\}} \varepsilon\text{-closure}(\delta(p, b)) \\ &= \varepsilon\text{-closure}(\delta(q', b)) \cup \varepsilon\text{-closure}(\delta(q'', b)) && \text{(unione su due stati)} \\ &= \varepsilon\text{-closure}(\emptyset) \cup \varepsilon\text{-closure}(\{s\}) \\ &= \emptyset \cup \{s, t\} = \{s, t\} && (\delta(s, \varepsilon) = \{t\},\ \delta(t, \varepsilon) = \emptyset). \end{aligned}$$

A ogni livello l'insieme su cui si fa l'unione è il valore restituito dal livello sottostante: $\{q\}$ per la lettura di a, $\{q', q''\}$ per la lettura di b. Il ramo $p = q'$ non contribuisce, perché da q' non si legge b; è il ramo $p = q''$, raggiunto solo grazie alla ε-closure applicata dopo a, a portare in s. ◇

Teorema 4 (Equivalenza tra NFA ed ε-NFA).

(a) *Sia $\mathcal{N}_\varepsilon = \langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ un ε-NFA. Allora esiste un NFA $\mathcal{N} = \langle Q', \Sigma, \delta', q_0', \mathcal{F}'\rangle$ tale che $\mathcal{L}(\mathcal{N}_\varepsilon) = \mathcal{L}(\mathcal{N})$.*

(b) *Sia $\mathcal{N} = \langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ un NFA. Allora esiste un ε-NFA $\mathcal{N}_\varepsilon = \langle Q', \Sigma, \delta', q_0', \mathcal{F}'\rangle$ tale che $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{N}_\varepsilon)$.*

Dimostrazione del Teorema 4. Anche in questo caso si deve dimostrare un'equivalenza, quindi servono entrambi i versi.

Punto (a). La dimostrazione si articola in tre passi:

1. costruire un NFA $\mathcal{N}$ a partire da $\mathcal{N}_\varepsilon$;
2. dimostrare che, dopo aver letto una stringa non vuota, $\mathcal{N}$ e $\mathcal{N}_\varepsilon$ si trovano nello stesso insieme di stati;
3. dedurne che $\mathcal{N}$ e $\mathcal{N}_\varepsilon$ accettano le stesse stringhe.

Passo 1: costruzione di $\mathcal{N}$. Gli stati, l'alfabeto e lo stato iniziale restano quelli di $\mathcal{N}_\varepsilon$, cioè $Q' = Q$ e $q_0' = q_0$; cambiano solo la funzione di transizione e gli stati finali.

- La nuova funzione di transizione ingloba le ε-transizioni dell'automa originale, sia quelle che precedono sia quelle che seguono la lettura del simbolo:

$$\delta'(q,a) := \hat{\delta}(q,a) = \bigcup_{p \in \varepsilon\text{-closure}(q)} \varepsilon\text{-closure}(\delta(p,a)).$$

- Lo stato iniziale diventa finale se da esso si può raggiungere uno stato finale con sole ε-transizioni, perché in questo caso $\mathcal{N}_\varepsilon$ accetta ε:

$$\mathcal{F}' = \begin{cases} \mathcal{F} \cup \{q_0\} & \text{se } \varepsilon\text{-closure}(q_0) \cap \mathcal{F} \neq \emptyset, \\ \mathcal{F} & \text{altrimenti.} \end{cases}$$

Passo 2: stessi stati raggiunti. Si dimostra per induzione su $|x|$ che[4]

$$\hat{\delta}'(q_0, x) = \hat{\delta}(q_0, x) \qquad \text{per ogni } x \in \Sigma^* \text{ con } x \neq \varepsilon.$$

L'ipotesi $x \neq \varepsilon$ è necessaria, perché per la stringa vuota l'uguaglianza non vale: $\hat{\delta}'(q_0, \varepsilon) = \{q_0\}$, mentre $\hat{\delta}(q_0, \varepsilon) = \varepsilon\text{-closure}(q_0)$.

Passo 3: stesse stringhe accettate. Bisogna dimostrare che, per ogni $x \in \Sigma^*$,[5]

$$\hat{\delta}'(q_0, x) \cap \mathcal{F}' \neq \emptyset \iff \hat{\delta}(q_0, x) \cap \mathcal{F} \neq \emptyset.$$

Questo passo usa il passo 2, che va quindi dimostrato prima. Poiché il passo 2 vale solo per le stringhe non vuote, si distinguono due casi.

Caso $x = \varepsilon$. $\mathcal{N}$ accetta ε se e solo se $q_0 \in \mathcal{F}'$, mentre $\mathcal{N}_\varepsilon$ accetta ε se e solo se $\varepsilon\text{-closure}(q_0) \cap \mathcal{F} \neq \emptyset$. Basta quindi dimostrare che

$$q_0 \in \mathcal{F}' \iff \varepsilon\text{-closure}(q_0) \cap \mathcal{F} \neq \emptyset,$$

che segue dalla definizione di $\mathcal{F}'$.

($\Leftarrow$) Se $\varepsilon\text{-closure}(q_0) \cap \mathcal{F} \neq \emptyset$, si applica il primo caso della definizione: $\mathcal{F}' = \mathcal{F} \cup \{q_0\}$, quindi $q_0 \in \mathcal{F}'$.

[4] La dimostrazione per induzione di questo passo non è stata svolta a lezione.

[5] La dimostrazione di questo passo non è stata svolta a lezione.

($\Rightarrow$) Se $q_0 \in \mathcal{F}'$, ci sono due possibilità:

- è stato applicato il primo caso della definizione, perché $\varepsilon\text{-closure}(q_0) \cap \mathcal{F} \neq \emptyset$, e la condizione vale già;
- è stato applicato il secondo caso, $\mathcal{F}' = \mathcal{F}$: allora $q_0 \in \mathcal{F}$, cioè q_0 era già finale. Poiché $q_0 \in \varepsilon\text{-closure}(q_0)$ sempre, dato che con zero ε-transizioni si resta in q_0, si ha $q_0 \in \varepsilon\text{-closure}(q_0) \cap \mathcal{F}$, che quindi non è vuoto.

Caso $x \neq \varepsilon$. Per il passo 2 si può porre $S = \hat{\delta}'(q_0, x) = \hat{\delta}(q_0, x)$, e bisogna dimostrare che

$$S \cap \mathcal{F}' \neq \emptyset \iff S \cap \mathcal{F} \neq \emptyset.$$

($\Leftarrow$) $\mathcal{F}'$ è un sovrainsieme di $\mathcal{F}$, quindi se esiste $f \in S \cap \mathcal{F}$ allora $f \in S \cap \mathcal{F}'$.

($\Rightarrow$) Se $S \cap \mathcal{F}'$ contiene uno stato diverso da q_0, questo appartiene anche a $\mathcal{F}$ e non c'è altro da dimostrare. Resta il caso in cui l'unico stato in comune sia q_0, con $q_0 \notin \mathcal{F}$. Allora valgono sia $q_0 \in \mathcal{F}'$ sia $q_0 \in S$, e da ciascuna delle due condizioni segue un fatto:

- da $q_0 \in \mathcal{F}'$, per la definizione di $\mathcal{F}'$, esiste uno stato finale $f \in \varepsilon\text{-closure}(q_0) \cap \mathcal{F}$, cioè uno stato finale raggiungibile da q_0 con sole ε-transizioni;
- da $q_0 \in S$ segue $\varepsilon\text{-closure}(q_0) \subseteq S$. Infatti $q_0 \in S$ significa che, dopo aver letto x, l'automa può trovarsi in q_0. Poiché $x \neq \varepsilon$, l'ultimo passo di $\hat{\delta}$ legge un simbolo e termina con una ε-closure, che da q_0 segue tutte le ε-transizioni possibili e non si ferma finché trova stati nuovi: tutti gli stati raggiungibili da q_0 con sole ε-transizioni entrano quindi in S.

Messi insieme, i due fatti danno la tesi:

$$q_0 \in S \cap \mathcal{F}' \implies \big(\exists f \in \varepsilon\text{-closure}(q_0) \cap \mathcal{F}\big) \wedge \big(\varepsilon\text{-closure}(q_0) \subseteq S\big) \implies f \in S \cap \mathcal{F}.$$

In entrambi i casi $\mathcal{N}$ e $\mathcal{N}_\varepsilon$ accettano le stesse stringhe, quindi $\mathcal{L}(\mathcal{N}) = \mathcal{L}(\mathcal{N}_\varepsilon)$.

Punto (b). Questo verso è banale: un NFA è un ε-NFA privo di archi etichettati con ε, ovvero con $\delta(q, \varepsilon) = \emptyset$ per ogni $q \in Q$. □

3.7 Operazioni tra linguaggi

Siano $\mathcal{L}, \mathcal{L}_1, \mathcal{L}_2$ linguaggi sull'alfabeto Σ. Oltre alle operazioni insiemistiche della Sezione 0.2.2, come l'unione

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{x \mid x \in \mathcal{L}_1 \vee x \in \mathcal{L}_2\},$$

sui linguaggi sono definite due operazioni specifiche: la concatenazione e la chiusura di Kleene.

Definizione (Concatenazione). *La* concatenazione *di $\mathcal{L}_1$ e $\mathcal{L}_2$ è il linguaggio delle stringhe ottenute facendo seguire a una stringa di $\mathcal{L}_1$ una stringa di $\mathcal{L}_2$:*

$$\mathcal{L}_1 \cdot \mathcal{L}_2 = \{x \cdot y \mid x \in \mathcal{L}_1 \wedge y \in \mathcal{L}_2\}.$$

Se uno dei due linguaggi contiene la stringa vuota, la concatenazione contiene l'altro. Ad esempio, se $\varepsilon \in \mathcal{L}_1$, ogni $y \in \mathcal{L}_2$ si scrive come $\varepsilon \cdot y$, quindi

$$\varepsilon \in \mathcal{L}_1 \implies \mathcal{L}_2 \subseteq \mathcal{L}_1 \cdot \mathcal{L}_2, \qquad \varepsilon \in \mathcal{L}_2 \implies \mathcal{L}_1 \subseteq \mathcal{L}_1 \cdot \mathcal{L}_2.$$

Definizione (Chiusura di Kleene). *Le* potenze *di $\mathcal{L}$ si definiscono per concatenazione iterata:*

$$\mathcal{L}^0 = \{\varepsilon\}, \qquad \mathcal{L}^{i+1} = \mathcal{L} \cdot \mathcal{L}^i.$$

La chiusura di Kleene *$\mathcal{L}^*$ è l'unione di tutte le potenze di $\mathcal{L}$, mentre $\mathcal{L}^+$ esclude la potenza $\mathcal{L}^0$:*

$$\mathcal{L}^* = \bigcup_{i \geq 0} \mathcal{L}^i, \qquad \mathcal{L}^+ = \bigcup_{i \geq 1} \mathcal{L}^i.$$

Esempio. Sia $\mathcal{L} = \{ab\}$. Allora

$$\begin{aligned}
\mathcal{L}^0 &= \{\varepsilon\}, \\
\mathcal{L}^1 &= \{ab\} \cdot \{\varepsilon\} = \{ab\}, \\
\mathcal{L}^2 &= \{ab\} \cdot \mathcal{L}^1 = \{ab\} \cdot \{ab\} = \{abab\},
\end{aligned}$$

e così via, quindi $\mathcal{L}^* = \{\varepsilon, ab, abab, ababab, \dots\}$. Con $\mathcal{L} = \{ab, cd\}$, invece,

$$\begin{aligned}
\mathcal{L}^0 &= \{\varepsilon\}, \\
\mathcal{L}^1 &= \{ab, cd\} \cdot \{\varepsilon\} = \{ab, cd\}, \\
\mathcal{L}^2 &= \{ab, cd\} \cdot \mathcal{L}^1 = \{abab, abcd, cdab, cdcd\},
\end{aligned}$$

e così via. ◇

3.8 Espressioni regolari

Definizione (Espressione regolare). *Un'*espressione regolare *(RE) è uno strumento formale utilizzato per descrivere e generare in modo compatto le stringhe che appartengono a un linguaggio regolare. Le espressioni regolari su un alfabeto Σ sono definite per induzione:*

- *$\emptyset$ è un'espressione regolare, che denota l'insieme vuoto;*
- *ε è un'espressione regolare, che denota l'insieme $\{\varepsilon\}$;*
- *per ogni simbolo $a \in \Sigma$, a è un'espressione regolare, che denota l'insieme $\{a\}$;*
- *se r e s sono espressioni regolari che denotano rispettivamente gli insiemi R e S, allora $(r + s)$, $(r \cdot s)$ e (r^*) sono espressioni regolari, che denotano rispettivamente gli insiemi $R \cup S$, $R \cdot S$ e R^*.*

Se r è un'espressione regolare, si indica con $\mathcal{L}(r)$ il linguaggio denotato da r. Due espressioni regolari sono uguali se denotano lo stesso linguaggio,

$$r = s \iff \mathcal{L}(r) = \mathcal{L}(s),$$

e tra espressioni regolari valgono uguaglianze che ne permettono la manipolazione algebrica.

Esempio. Sia $\Sigma = \{0, 1\}$ e si consideri l'espressione regolare $r = 1(0 + 1)^*00$. Il linguaggio associato è

$$\mathcal{L}(r) = \mathcal{L}(1)\big(\mathcal{L}(0) \cup \mathcal{L}(1)\big)^*\mathcal{L}(0)\mathcal{L}(0) = \{1x00 \mid x \in \Sigma^*\},$$

ovvero l'insieme delle stringhe che rappresentano in binario, senza zeri iniziali, i multipli di 4 maggiori di zero. ◇

Il teorema seguente mostra che ogni linguaggio descritto da un'espressione regolare è riconosciuto da un ε-NFA.

Teorema 5 (di McNaughton e Yamada, 1960). *Sia r un'espressione regolare. Allora esiste un ε-NFA $\mathcal{M}$ tale che $\mathcal{L}(\mathcal{M}) = \mathcal{L}(r)$.*

Dimostrazione del Teorema 5. Si dimostra, più precisamente, che esiste un ε-NFA con un unico stato finale che riconosce $\mathcal{L}(r)$. Si procede per induzione sulla struttura di r, costruendo un automa per ciascun caso base e per ciascuna regola induttiva.

Base. I tre casi base sono riconosciuti dagli automi della Figura 3.12:

- se $r = \varepsilon$, l'automa riconosce $\mathcal{L}(r) = \{\varepsilon\}$, perché lo stato iniziale è anche finale e non ci sono transizioni;
- se $r = \emptyset$, l'automa riconosce $\mathcal{L}(r) = \emptyset$, perché lo stato finale non è raggiungibile;
- se $r = a$, con $a \in \Sigma$, l'automa riconosce $\mathcal{L}(r) = \{a\}$.

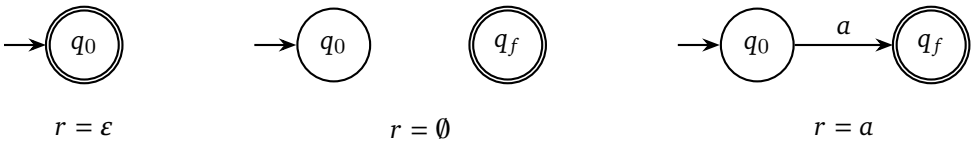


Figura 3.12 *Gli ε-NFA dei casi base.*

Passo induttivo. Per ipotesi induttiva, per ogni sottoespressione r_i di r esiste un ε-NFA $\mathcal{M}_i$, con stato iniziale q_0^i e unico stato finale q_f^i, che riconosce $\mathcal{L}(r_i)$. Nelle figure seguenti ogni $\mathcal{M}_i$ è racchiuso in un riquadro tratteggiato, di cui si mostrano solo lo stato iniziale e quello finale. Il nuovo automa ha stato iniziale q_0 e unico stato finale q_f: gli stati q_f^i non sono più finali.

Unione. Se $r = r_1 + r_2$, l'automa della Figura 3.13 riconosce $\mathcal{L}(r) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)$: da q_0 si sceglie, con una ε-transizione, quale dei due automi percorrere.

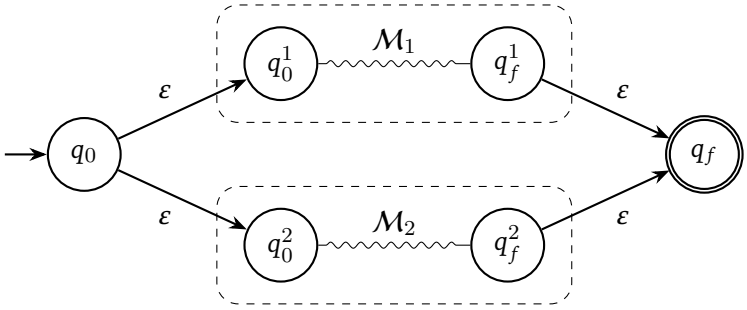


Figura 3.13 *Costruzione per l'unione $r_1 + r_2$.*

Concatenazione. Se $r = r_1 r_2$, l'automa della Figura 3.14 riconosce $\mathcal{L}(r) = \mathcal{L}(r_1) \cdot \mathcal{L}(r_2)$: dopo aver letto una stringa di $\mathcal{L}(r_1)$ si passa, senza leggere simboli, all'inizio di $\mathcal{M}_2$.

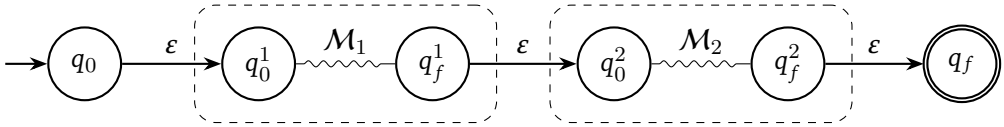


Figura 3.14 *Costruzione per la concatenazione $r_1 r_2$.*

Chiusura di Kleene. Se $r = r_1^*$, l'automa della Figura 3.15 riconosce $\mathcal{L}(r) = \mathcal{L}(r_1)^*$: l'arco da q_f^1 a q_0^1 permette di ripetere $\mathcal{M}_1$ quante volte si vuole, mentre quello da q_0 a q_f accetta la stringa vuota.

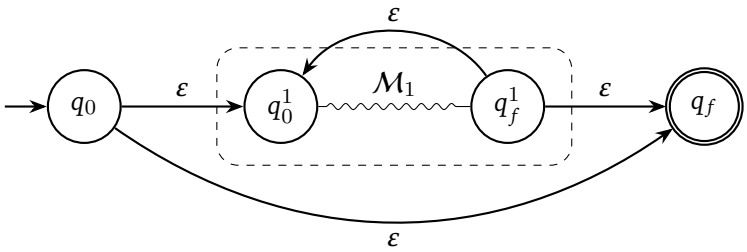


Figura 3.15 *Costruzione per la chiusura di Kleene* r_1^*.

□

Esempio. Si consideri l'espressione regolare $r = 0^* + 1^* + (01)^*$, che denota il linguaggio

$$\mathcal{L}(r) = \{x \in \{0,1\}^* \mid \exists m \in \mathbb{N}.\ x = 0^m \vee x = 1^m \vee x = (01)^m\}.$$

L'automa della Figura 3.16 riconosce correttamente il linguaggio, ma è costruito ad hoc e non segue la costruzione vista nella dimostrazione del Teorema 5.

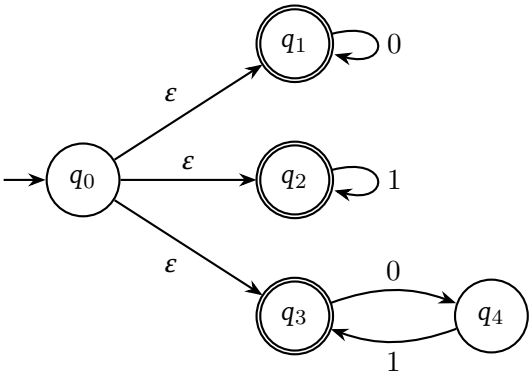


Figura 3.16 *Un ε-NFA costruito ad hoc per* $0^* + 1^* + (01)^*$.

L'automa della Figura 3.17, invece, ricalca la costruzione dell'unione: un nuovo stato iniziale q_0 e un unico stato finale q_f, collegati con ε-transizioni agli automi di 0^*, 1^* e $(01)^*$. Per leggibilità, questi ultimi sono semplificati rispetto alla costruzione completa, che userebbe più stati e più ε-transizioni.

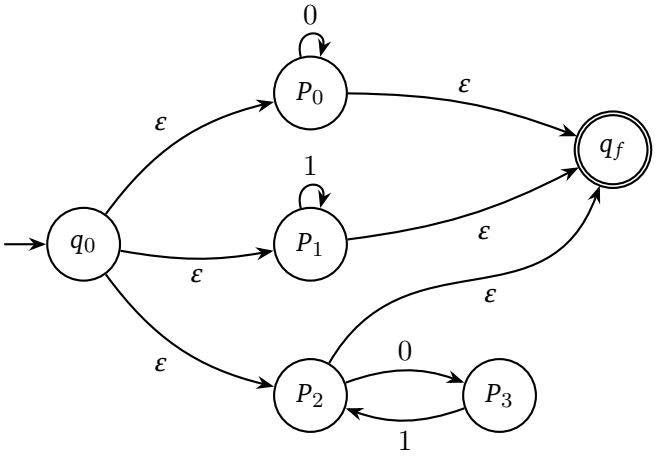


Figura 3.17 *Un ε-NFA per* $0^* + 1^* + (01)^*$ *con la struttura della costruzione dell'unione.*

◇

Ricapitolando, tutti i modelli visti descrivono la stessa classe di linguaggi:

$$\text{linguaggi regolari} \iff \text{DFA} \iff \text{NFA} \iff \varepsilon\text{-NFA} \iff \text{RE}.$$

3.9 Proprietà dei linguaggi regolari

Si indica con LR l'insieme dei linguaggi regolari:

$$\text{LR} = \{\mathcal{L} \subseteq \Sigma^* \mid \text{“}\mathcal{L} \text{ è un linguaggio regolare”}\}.$$

I linguaggi regolari godono di alcune proprietà di chiusura.

Teorema 6 (Proprietà di chiusura). *Siano $\mathcal{L}, \mathcal{L}_1, \mathcal{L}_2 \in \text{LR}$. Allora:*

(a) LR *è chiuso rispetto all'unione, alla concatenazione e alla chiusura di Kleene:*

$$\mathcal{L}_1 \cup \mathcal{L}_2 \in \text{LR}, \qquad \mathcal{L}_1 \cdot \mathcal{L}_2 \in \text{LR}, \qquad \mathcal{L}^* \in \text{LR};$$

(b) LR *è chiuso rispetto al complemento:* $\overline{\mathcal{L}} = \Sigma^* \setminus \mathcal{L} \in \text{LR}$*;*

(c) LR *è chiuso rispetto all'intersezione:* $\mathcal{L}_1 \cap \mathcal{L}_2 \in \text{LR}$.

Dimostrazione del Teorema 6. Punto (a). Deriva direttamente dall'equivalenza con le espressioni regolari: se $\mathcal{L}_1 = \mathcal{L}(r_1)$, $\mathcal{L}_2 = \mathcal{L}(r_2)$ e $\mathcal{L} = \mathcal{L}(r)$, i tre linguaggi sono denotati rispettivamente dalle espressioni regolari $r_1 + r_2$, $r_1 r_2$ e r^*.

Punto (b). Sia $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F}\rangle$ un DFA che riconosce $\mathcal{L}$. Basta ridefinire l'insieme degli stati finali come $\mathcal{F}' = Q \setminus \mathcal{F}$: il DFA ottenuto accetta esattamente le stringhe rifiutate da $\mathcal{M}$, cioè $\overline{\mathcal{L}}$.

Punto (c). Per le leggi di De Morgan,

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \overline{\overline{\mathcal{L}_1} \cup \overline{\mathcal{L}_2}},$$

e il membro destro è regolare per i punti (a) e (b). □

La chiusura rispetto all'unione e all'intersezione vale solo per un numero *finito* di linguaggi: anche se ogni $\mathcal{L}_m$ è regolare, l'unione o l'intersezione di infiniti linguaggi regolari può non esserlo. In simboli,

$$\left(\forall m \in \mathbb{N}.\ \mathcal{L}_m \in \text{LR}\right) \not\Rightarrow \bigcup_{m\in\mathbb{N}} \mathcal{L}_m \in \text{LR},$$
$$\left(\forall m \in \mathbb{N}.\ \mathcal{L}_m \in \text{LR}\right) \not\Rightarrow \bigcap_{m\in\mathbb{N}} \mathcal{L}_m \in \text{LR}.$$

3.10 Problemi di decidibilità

Definizione (Problema decidibile). *Un problema è* decidibile *se esiste un algoritmo che, ricevuta in input un'istanza qualsiasi del problema, termina sempre in un numero finito di passi e fornisce la risposta corretta (sì o no).*

Teorema 7 (Problemi decidibili sui linguaggi regolari). *Sia $\mathcal{L}$ un linguaggio regolare e sia $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ un DFA tale che $\mathcal{L} = \mathcal{L}(\mathcal{M})$. I seguenti problemi sono decidibili:*

(a) Vacuità*: stabilire se $\mathcal{L} \neq \emptyset$. Vale*

$$\mathcal{L} \neq \emptyset \iff \exists x \in \mathcal{L}(\mathcal{M}).\ |x| < |Q|;$$

(b) Infinità*: stabilire se $|\mathcal{L}| = \omega$, ovvero se $\mathcal{L}$ è infinito. Detto $n = |Q|$ il numero di stati di $\mathcal{M}$, vale*

$$|\mathcal{L}| = \omega \iff \exists x \in \mathcal{L}(\mathcal{M}).\ n \leq |x| < 2n;$$

(c) Equivalenza*: dati due DFA $\mathcal{M}_1$ e $\mathcal{M}_2$, stabilire se $\mathcal{L}(\mathcal{M}_1) = \mathcal{L}(\mathcal{M}_2)$.*

Per i primi due problemi basta quindi controllare se $\mathcal{M}$ accetta almeno una delle stringhe di lunghezza limitata, che sono in numero finito.

3.11 Relazioni di equivalenza e linguaggi

Definizione (Relazione indotta da un linguaggio). *Sia $\mathcal{L} \subseteq \Sigma^*$. La relazione $R_\mathcal{L} \subseteq \Sigma^* \times \Sigma^*$ è definita da*

$$x\ R_\mathcal{L}\ y \iff \forall z \in \Sigma^*.\ (xz \in \mathcal{L} \iff yz \in \mathcal{L}).$$

Due stringhe sono quindi in relazione se, prolungandole con uno stesso suffisso z qualsiasi, si ottengono stringhe che stanno entrambe in $\mathcal{L}$ o entrambe fuori.

Definizione (Relazione indotta da un DFA). *Sia $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ un DFA. La relazione $R_\mathcal{M} \subseteq \Sigma^* \times \Sigma^*$ è definita da*

$$x\ R_\mathcal{M}\ y \iff \hat{\delta}(q_0, x) = \hat{\delta}(q_0, y).$$

Due stringhe sono quindi in relazione se portano l'automa nello stesso stato. $R_\mathcal{L}$ e $R_\mathcal{M}$ sono entrambe relazioni di equivalenza.

Definizione (Relazione invariante a destra). *Una relazione di equivalenza $R \subseteq \Sigma^* \times \Sigma^*$ è* invariante a destra *se*

$$x\ R\ y \implies \forall z \in \Sigma^*.\ xz\ R\ yz.$$

L'invarianza a destra è una proprietà che una relazione può avere o non avere, e la condizione è una sola implicazione, non un'equivalenza: da $xz\ R\ yz$ non segue, in generale, $x\ R\ y$.

$R_\mathcal{M}$ è invariante a destra: se due stringhe portano l'automa nello stesso stato, leggendo poi uno stesso suffisso z si arriva ancora nello stesso stato. Poiché $\hat{\delta}(q_0, xz) = \hat{\delta}(\hat{\delta}(q_0, x), z)$, si ha

$$\hat{\delta}(q_0, x) = \hat{\delta}(q_0, y) \implies \forall z \in \Sigma^*.\ \hat{\delta}(q_0, xz) = \hat{\delta}(q_0, yz) \implies \forall z \in \Sigma^*.\ xz\ R_\mathcal{M}\ yz.$$

Anche $R_\mathcal{L}$ è invariante a destra. Se $x\ R_\mathcal{L}\ y$ e $w \in \Sigma^*$, per ogni $z \in \Sigma^*$ si applica la definizione di $R_\mathcal{L}$ al suffisso wz, ottenendo

$$(xw)z \in \mathcal{L} \iff (yw)z \in \mathcal{L},$$

cioè $xw\ R_\mathcal{L}\ yw$.

Definizione (Linguaggio indotto da uno stato). *Sia $\mathcal{M}$ un DFA e sia q un suo stato. Il linguaggio* indotto *da q è*

$$\mathcal{L}_q = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) = q\},$$

ovvero il linguaggio "accettato" dallo stato q.

Le stringhe accettate da $\mathcal{M}$ sono quelle che portano in uno stato finale, quindi

$$\mathcal{L}(\mathcal{M}) = \bigcup_{q \in \mathcal{F}} \mathcal{L}_q.$$

Definizione (Indice). *L'*indice *di una relazione di equivalenza è il numero delle sue classi di equivalenza. La relazione è* di indice finito *se le sue classi di equivalenza sono in numero finito.*

Definizione (Raffinamento). *Siano R e R' relazioni di equivalenza sullo stesso insieme. R è un* raffinamento *di R' se*

$$x \, R \, y \implies x \, R' \, y.$$

Le seguenti affermazioni sono equivalenti alla definizione:

- ogni classe di equivalenza di R è contenuta in una classe di R', cioè $[x]_R \subseteq [x]_{R'}$ per ogni x;
- ogni classe di equivalenza di R' è unione di classi di equivalenza di R.

Ne segue che l'indice di R è maggiore o uguale a quello di R' e, in particolare, che se R è di indice finito anche R' lo è. Queste ultime sono solo conseguenze: un indice maggiore o uguale non basta perché R raffini R'.

Teorema 8 (di Myhill e Nerode). *Sia $\mathcal{L} \subseteq \Sigma^*$. I seguenti enunciati sono equivalenti:*

(1) *$\mathcal{L}$ è riconosciuto da un DFA, cioè è regolare;*
(2) *$\mathcal{L}$ è unione di classi di equivalenza di una relazione di equivalenza su Σ^* invariante a destra e di indice finito;*
(3) *$R_{\mathcal{L}}$ è di indice finito.*

Dimostrazione del Teorema 8. Si dimostrano le implicazioni in catena,

$$(1) \implies (2) \implies (3) \implies (1),$$

da cui segue che i tre enunciati sono equivalenti.

(1) $\Rightarrow$ (2). Si suppone che valga (1), cioè che $\mathcal{L}$ sia regolare: esiste un DFA $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ tale che $\mathcal{L} = \mathcal{L}(\mathcal{M})$. Si dimostra che vale (2) con la relazione $R_{\mathcal{M}}$. Come visto,

$$\mathcal{L} = \bigcup_{q \in \mathcal{F}} \mathcal{L}_q = \bigcup_{q \in \mathcal{F}} \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) = q\},$$

quindi basta verificare che:

- $R_{\mathcal{M}}$ è invariante a destra, come mostrato sopra;
- i linguaggi indotti $\mathcal{L}_q$ sono le classi di equivalenza di $R_{\mathcal{M}}$. Infatti, se $x \in \mathcal{L}_q$, per ogni $y \in \Sigma^*$ si ha

$$x \, R_{\mathcal{M}} \, y \iff \hat{\delta}(q_0, y) = \hat{\delta}(q_0, x) = q \iff y \in \mathcal{L}_q,$$

cioè $\mathcal{L}_q = [x]_{R_{\mathcal{M}}}$;

- $R_{\mathcal{M}}$ è di indice finito: $\mathcal{M}$ è a stati finiti, quindi $|Q|$ è finito, e le classi di equivalenza sono una per ogni stato raggiungibile da q_0, quindi al più $|Q|$.

Quindi $\mathcal{L}$ è unione delle classi di equivalenza $\mathcal{L}_q$, con $q \in \mathcal{F}$, di una relazione invariante a destra e di indice finito, ovvero vale (2).

(2) $\Rightarrow$ (3). Si suppone che valga (2): $\mathcal{L}$ è unione di classi di equivalenza di una relazione R invariante a destra e di indice finito (quindi $\mathcal{L}$ non taglia le classi: non è possibile che, di due elementi in relazione tra loro, uno stia in $\mathcal{L}$ e l'altro fuori). Si dimostra che R è un raffinamento di $R_{\mathcal{L}}$, cioè che

$$\forall x \in \Sigma^*.\ [x]_R \subseteq [x]_{R_{\mathcal{L}}}.$$

Siano $x \in \Sigma^*$ e $y \in [x]_R$, cioè $x\ R\ y$. Allora

$$\begin{aligned}
x\ R\ y &\implies \forall z \in \Sigma^*.\ xz\ R\ yz && (R \text{ è invariante a destra}) \\
&\implies \forall z \in \Sigma^*.\ (xz \in \mathcal{L} \iff yz \in \mathcal{L}) && (\mathcal{L} \text{ non taglia le classi di } R) \\
&\iff x\ R_{\mathcal{L}}\ y && (\text{definizione di } R_{\mathcal{L}}) \\
&\iff y \in [x]_{R_{\mathcal{L}}} && (\text{definizione di classe di equivalenza}).
\end{aligned}$$

Si è partiti da $y \in [x]_R$ e si è arrivati a $y \in [x]_{R_{\mathcal{L}}}$, quindi $[x]_R \subseteq [x]_{R_{\mathcal{L}}}$. Dunque R è un raffinamento di $R_{\mathcal{L}}$ e, poiché R è di indice finito, anche $R_{\mathcal{L}}$ è di indice finito: vale (3).

(3) $\Rightarrow$ (1). Si suppone che valga (3): $R_{\mathcal{L}}$ è di indice finito. La dimostrazione si articola in tre passi:

1. costruire un DFA $\mathcal{M}'$ i cui stati sono le classi di equivalenza di $R_{\mathcal{L}}$;
2. dimostrare che, leggendo una stringa y a partire dalla classe di x, $\mathcal{M}'$ arriva nella classe di xy;
3. dedurne che $\mathcal{L} = \mathcal{L}(\mathcal{M}')$.

Passo 1: costruzione di $\mathcal{M}'$. Si pone $\mathcal{M}' = \langle Q', \Sigma', \delta', q_0', \mathcal{F}' \rangle$ con:

- $Q' = \{[x]_{R_{\mathcal{L}}} \mid x \in \Sigma^*\}$, che è un insieme finito per ipotesi;
- $\Sigma' = \Sigma$, l'alfabeto di $\mathcal{L}$;
- $\delta' : Q' \times \Sigma \to Q'$, con $\delta'([x]_{R_{\mathcal{L}}}, a) \coloneqq [xa]_{R_{\mathcal{L}}}$;
- $q_0' = [\varepsilon]_{R_{\mathcal{L}}}$;
- $\mathcal{F}' = \{[x]_{R_{\mathcal{L}}} \mid x \in \mathcal{L}\}$.

Quella di δ' è una buona definizione: il risultato non dipende dall'elemento x scelto per rappresentare la classe. Infatti, se $[x]_{R_{\mathcal{L}}} = [y]_{R_{\mathcal{L}}}$, cioè $x\ R_{\mathcal{L}}\ y$, allora

$$\begin{aligned}
x\ R_{\mathcal{L}}\ y &\implies \forall z \in \Sigma^*.\ xz\ R_{\mathcal{L}}\ yz && (R_{\mathcal{L}} \text{ è invariante a destra}) \\
&\implies xa\ R_{\mathcal{L}}\ ya && (\text{caso } z = a) \\
&\iff [xa]_{R_{\mathcal{L}}} = [ya]_{R_{\mathcal{L}}}.
\end{aligned}$$

Passo 2: stato raggiunto leggendo y. Si dimostra, per induzione sulla lunghezza $|y|$ della stringa letta, che

$$\forall [x]_{R_{\mathcal{L}}} \in Q'.\ \hat{\delta}'([x]_{R_{\mathcal{L}}}, y) = [xy]_{R_{\mathcal{L}}}.$$

- *Base* ($y = \varepsilon$). Leggendo ε non si cambia stato e $x = x\varepsilon$ per ogni x, quindi

$$\hat{\delta}'([x]_{R_{\mathcal{L}}}, \varepsilon) = [x]_{R_{\mathcal{L}}} = [x\varepsilon]_{R_{\mathcal{L}}}.$$

- *Passo induttivo*. Si suppone che l'uguaglianza valga per ogni y con $|y| \leq m$ e sia ya una stringa di lunghezza $m+1$. Allora

$$\begin{aligned}
\hat{\delta}'([x]_{R_\mathcal{L}}, ya) &= \delta'(\hat{\delta}'([x]_{R_\mathcal{L}}, y), a) && \text{(definizione di } \hat{\delta}') \\
&= \delta'([xy]_{R_\mathcal{L}}, a) && \text{(ipotesi induttiva)} \\
&= [xya]_{R_\mathcal{L}} && \text{(definizione di } \delta').
\end{aligned}$$

Passo 3: $\mathcal{L} = \mathcal{L}(\mathcal{M}')$. Bisogna dimostrare che

$$\forall y \in \Sigma^*.\ y \in \mathcal{L}(\mathcal{M}') \iff y \in \mathcal{L}.$$

Si usa il passo 2 con $x = \varepsilon$, cioè partendo dallo stato iniziale $q_0' = [\varepsilon]_{R_\mathcal{L}}$: la stringa y resta qualsiasi, quindi serve il passo 2 per intero e non solo il suo caso base. Poiché ε concatenata a sinistra non cambia la stringa, $[\varepsilon y]_{R_\mathcal{L}} = [y]_{R_\mathcal{L}}$, e si ottiene

$$\begin{aligned}
y \in \mathcal{L}(\mathcal{M}') &\iff \hat{\delta}'(q_0', y) \in \mathcal{F}' && \text{(definizione di } \mathcal{L}(\mathcal{M}')) \\
&\iff \hat{\delta}'([\varepsilon]_{R_\mathcal{L}}, y) \in \mathcal{F}' && \text{(definizione di } q_0') \\
&\iff [\varepsilon y]_{R_\mathcal{L}} = [y]_{R_\mathcal{L}} \in \mathcal{F}' && \text{(passo 2 con } x = \varepsilon) \\
&\iff y \in \mathcal{L} && \text{(definizione di } \mathcal{F}').
\end{aligned}$$

Nell'ultimo passaggio, $[y]_{R_\mathcal{L}} \in \mathcal{F}' \iff y \in \mathcal{L}$, il verso ($\Leftarrow$) è immediato, mentre il verso ($\Rightarrow$) richiede un argomento in più:

($\Leftarrow$) se $y \in \mathcal{L}$, allora $[y]_{R_\mathcal{L}} \in \mathcal{F}'$ per definizione di $\mathcal{F}'$;

($\Rightarrow$) se $[y]_{R_\mathcal{L}} \in \mathcal{F}'$, allora $[y]_{R_\mathcal{L}} = [x]_{R_\mathcal{L}}$ per qualche $x \in \mathcal{L}$, non necessariamente y stesso, quindi $y\ R_\mathcal{L}\ x$. Prendendo $z = \varepsilon$ nella definizione di $R_\mathcal{L}$ si ottiene $y \in \mathcal{L} \iff x \in \mathcal{L}$ e, poiché $x \in \mathcal{L}$, anche $y \in \mathcal{L}$.

Quindi $\mathcal{L} = \mathcal{L}(\mathcal{M}')$: $\mathcal{L}$ è regolare e vale (1). □

3.12 PUMPING LEMMA PER I LINGUAGGI REGOLARI

Fornisce una condizione necessaria alla regolarità, ovvero una condizione sufficiente alla non **non** regolarità.

TEOREMA 9 (Pumping lemma). *Sia $\mathcal{L}$ un linguaggio regolare. Allora $\exists k \in \mathbb{N}$ tale che $\forall z \in \mathcal{L}$ con $|z| \geq k$ esiste una suddivisione in tre parti, $z = uvw$, con*

$$|uv| \leq k, \qquad |v| > 0,$$

in modo che

$$\forall i \in \mathbb{N}.\ uv^i w \in \mathcal{L}.$$

In altre parole, ogni stringa abbastanza lunga di $\mathcal{L}$ contiene, entro i primi k simboli, una parte non vuota v che si può ripetere quante volte si vuole, o togliere del tutto ($i = 0$), ottenendo sempre stringhe di $\mathcal{L}$, come illustrato nella Figura 3.18.

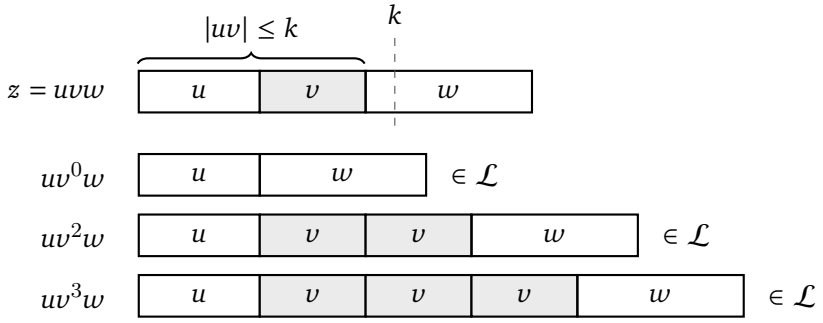


FIGURA 3.18 *Suddivisione $z = uvw$ con $|uv| \le k$ e $|v| > 0$: ripetendo la parte v, in grigio, zero o più volte si ottengono ancora stringhe di $\mathcal{L}$.*

La Figura 3.19 mostra la struttura dei quantificatori del lemma, dove $\mathcal{M}$ è un DFA che riconosce $\mathcal{L}$. Tra le stringhe di $\mathcal{L} = \mathcal{L}(\mathcal{M})$ si considerano quelle con $|z| \ge k$; per ciascuna di esse, tra tutte le suddivisioni $z = uvw$, ne esiste almeno una che rispetta entrambi i vincoli, $|uv| \le k$ e $|v| > 0$, e per questa vale $uv^iw \in \mathcal{L}$ per ogni $i \in \mathbb{N}$.

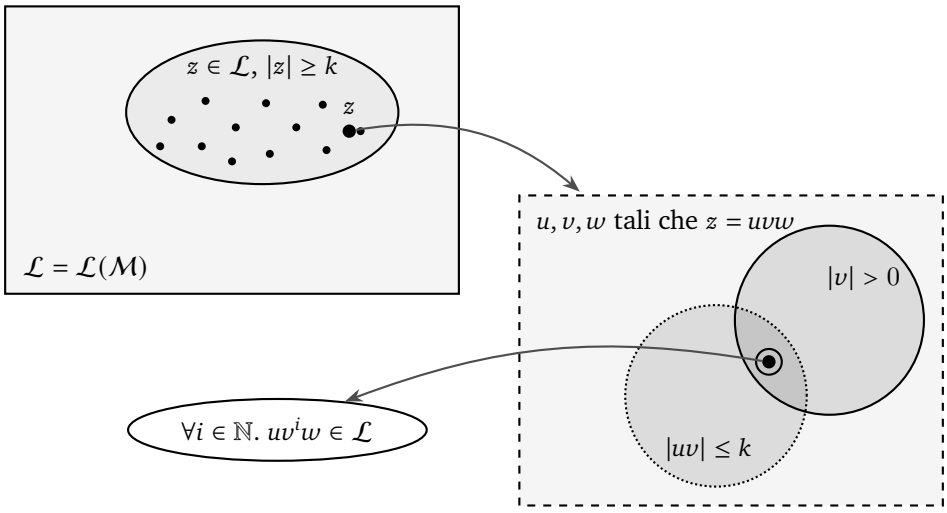


FIGURA 3.19 *Il pumping lemma visto con gli insiemi: per ogni $z \in \mathcal{L}$ con $|z| \ge k$, tra tutte le suddivisioni $z = uvw$ (riquadro tratteggiato) ne esiste una con $|uv| \le k$ e $|v| > 0$, nell'intersezione dei due cerchi, per cui $uv^iw \in \mathcal{L}$ per ogni i.*

Dimostrazione del Teorema 9. Poiché $\mathcal{L}$ è regolare, esiste un DFA $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ tale che $\mathcal{L} = \mathcal{L}(\mathcal{M})$. Sia $n = |Q| \in \mathbb{N}$ il numero di stati di $\mathcal{M}$: si sceglie $k = n$, cioè la costante k del lemma è il numero di stati dell'automa.

Sia $z \in \mathcal{L}$ con $|z| \ge k$, che esiste perché si considera $\mathcal{L}$ infinito,[6] e sia

$$z = a_1 a_2 \cdots a_m, \qquad m \ge k, \qquad a_1, a_2, \ldots, a_m \in \Sigma.$$

Per essere riconosciuta, z fa attraversare all'automa $m + 1$ stati: quello iniziale e uno per ogni simbolo letto. Poiché $m \ge k = n$, si ha $m + 1 > n$: si attraversano più di n stati, ma gli stati sono solo n. Quindi, nel cammino che riconosce z, almeno uno stato viene attraversato due volte.

[6] Se $\mathcal{L}$ è finito può non esistere alcuna z con $|z| \ge k$: in quel caso la tesi vale banalmente, perché non c'è nessuna stringa da considerare. Se $\mathcal{L}$ è infinito, invece, una tale z esiste sempre, perché le stringhe di lunghezza minore di k sono in numero finito.

Sia q il *primo* stato che si ripete. Come mostra la Figura 3.20, z si suddivide in tre parti:

- u, che porta da q_0 alla prima visita di q;
- v, che percorre un ciclo e riporta da q di nuovo in q;
- w, che porta da q allo stato finale q_f raggiunto leggendo z.

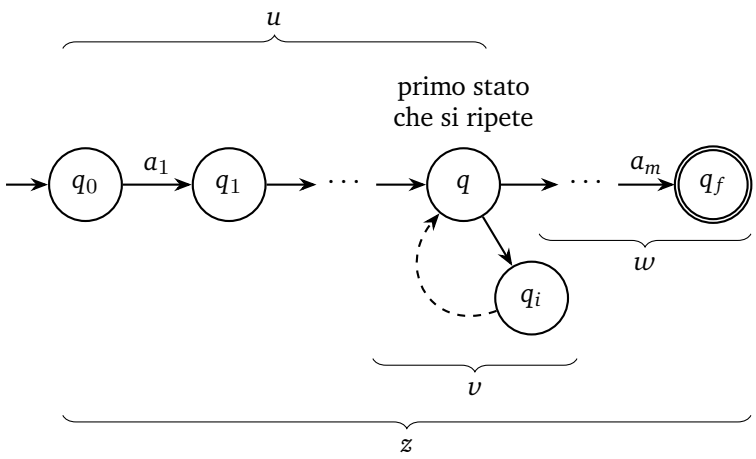


FIGURA 3.20 *Cammino di $\mathcal{M}$ su z: q è il primo stato che si ripete, u porta da q_0 a q, v è il ciclo che riporta in q e w porta da q allo stato finale q_f.*

La suddivisione rispetta i due vincoli del lemma.

- $|v| > 0$, perché il ciclo da q a q legge almeno un simbolo.
- $|uv| \leq k = n$, perché q è il primo stato che si ripete. Fino alla seconda visita di q gli stati attraversati sono tutti distinti, tranne q che compare due volte, quindi sono al più $n + 1$:

$$\underbrace{q_0, q_1, \ldots, q_{n-1}, q_n}_{n+1 \text{ stati}} \implies n \text{ simboli letti.}$$

  La stringa uv, che porta alla seconda visita di q, ha quindi al più n simboli. Il caso limite, $|uv| = n$, si ha quando $q_0, \ldots, q_{n-1}$ sono tutti gli n stati, diversi tra loro, e q_n coincide con uno di essi. Ad esempio, se si ripete l'ultimo, $q_n = q_{n-1}$, allora $|u| = n - 1 = k - 1$ e $|v| = 1$: u attraversa tutti gli n stati e v, con un solo simbolo, riporta in q_{n-1}.

Infine, poiché v riporta da q in q, il ciclo si può percorrere un numero qualsiasi di volte, o saltare: dopo aver letto uv^i l'automa è sempre in q, e da lì w porta in $q_f \in \mathcal{F}$. Quindi

$$|uv| \leq k \ \wedge \ |v| > 0 \implies \forall i \in \mathbb{N}.\ uv^i w \in \mathcal{L}. \qquad \square$$

Esempio. Si consideri un DFA con $n = 6$ stati, $Q = \{q_0, q_1, \ldots, q_5\}$, quindi $k = 6$, e una stringa $z \in \mathcal{L}$ con $|z| \geq k$:

$$z = a_1 a_2 a_3 a_4 a_5 a_6 a_7.$$

Leggendo i 7 simboli di z, l'automa attraversa 8 stati:

$$\underbrace{q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} q_3 \xrightarrow{a_4} q_4 \xrightarrow{a_5} q_5 \xrightarrow{a_6} q_5 \xrightarrow{a_7} q_f,}_{8 \text{ stati attraversati}}$$

dove q_f, uno dei sei stati, è lo stato finale raggiunto. Gli stati sono solo 6, quindi qualcuno si ripete: il primo è q_5, raggiunto dopo a_5 e di nuovo dopo a_6, attraverso un arco auto-entrante. È il caso limite descritto nella dimostrazione, in cui si ripete l'ultimo stato:

$$u = a_1a_2a_3a_4a_5, \qquad v = a_6, \qquad w = a_7,$$

con $|u| = 5 = k - 1$, $|v| = 1$ e $|uv| = 6 = k$, come mostra la Figura 3.21.

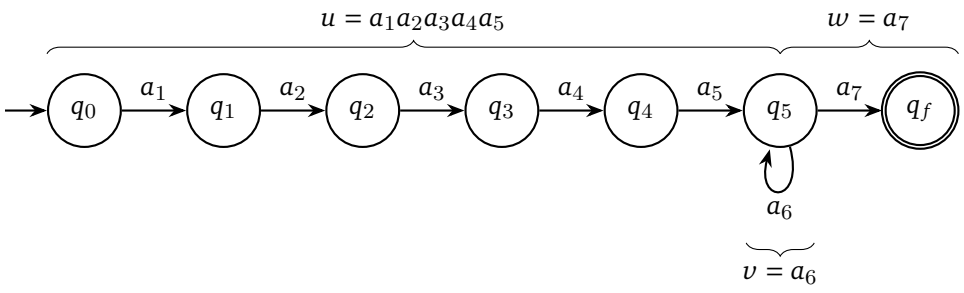


FIGURA 3.21 *La suddivisione di $z = a_1 \cdots a_7$: il primo stato che si ripete è q_5, e l'arco auto-entrante a_6 è la parte v.*

Iterando un numero qualsiasi di volte sull'arco auto-entrante si arriva sempre in q_5 e poi, con a_7, in q_f. Quindi $uv^iw = a_1a_2a_3a_4a_5\, a_6^i\, a_7 \in \mathcal{L}$ per ogni $i \in \mathbb{N}$. ◇

Il lemma ha la forma di un'implicazione,

$$\mathcal{L} \text{ regolare} \implies \text{vale la proprietà descritta},$$

e si usa soprattutto nella forma contronominale (Sezione 0.1.2),

$$(A \implies B) \equiv (\neg B \implies \neg A),$$

che in questo caso diventa

$$\text{la proprietà descritta non vale} \implies \mathcal{L} \text{ non è regolare}.$$

Il pumping lemma fornisce una condizione necessaria alla regolarità di un linguaggio, ossia una condizione sufficiente alla non regolarità.

L'utilizzo del pumping lemma per dimostrare la non regolarità richiede la negazione della condizione necessaria. Se

$$\begin{aligned}
&\forall k \in \mathbb{N}.\\
&\exists z \in \mathcal{L} \text{ con } |z| \geq k.\\
&\forall z = uvw \text{ con } |uv| \leq k \text{ e } |v| > 0.\\
&\exists i \in \mathbb{N}.\ uv^iw \notin \mathcal{L},
\end{aligned}$$

allora $\mathcal{L}$ *non* è regolare.

Esempio. Si consideri il linguaggio

$$\mathcal{L} = \{a^n b^n \mid n \in \mathbb{N}\},$$

che contiene ε, ab, $aabb$ e così via. Per riconoscere la sola stringa ab basta il primo automa della Figura 3.22. Viene naturale generalizzarlo con il secondo automa, in cui le frecce ondulate indicano i cammini che leggono a^n e b^n.

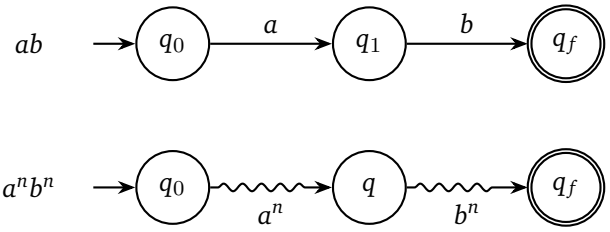


FIGURA 3.22 *L'automa che riconosce ab e, con le frecce ondulate, quello per $a^n b^n$, che esiste solo se n è fissato a priori.*

Se n è fissato a priori, il secondo automa esiste, perché il linguaggio contiene una sola stringa. Se invece n non è fornito a priori, come in $\mathcal{L}$, tale automa *non* esiste. Il numero di a ripetute consecutivamente potrebbe infatti essere diverso dal numero di b, e l'automa dovrebbe accettare soltanto le stringhe in cui i due numeri coincidono. L'unica forma di memoria di un DFA, però, è lo stato in cui si trova: gli stati sono in numero finito, mentre n può essere arbitrariamente grande, quindi non c'è alcun modo di tenere traccia di quante a sono state lette.

Si dimostra ora che $\mathcal{L}$ non è regolare usando il pumping lemma, cioè verificando la negazione della condizione necessaria, un quantificatore alla volta.

- $\forall k \in \mathbb{N}$: si sceglie un valore generico k, la costante del pumping lemma, sul quale non si può porre *alcuna* condizione: la dimostrazione deve valere qualunque sia k.
- $\exists z \in \mathcal{L}$ con $|z| \geq k$: basta individuare una stringa che appartenga a $\mathcal{L}$ e abbia lunghezza almeno k. Si sceglie
$$z = a^k b^k,$$
in cui il numero di a è uguale al numero di b, quindi $z \in \mathcal{L}$, e $|z| = 2k \geq k$.
- $\forall z = uvw$ con $|uv| \leq k$ e $|v| > 0$: si devono considerare *tutte* le suddivisioni di z che rispettano i due vincoli. I primi k simboli di z sono tutti a, quindi il vincolo $|uv| \leq k$ impone che uv sia contenuta in a^k. Infatti, se v contenesse almeno una b, allora uv conterrebbe tutte le k occorrenze di a e almeno una b, e quindi sarebbe $|uv| > k$, come mostra la Figura 3.23.

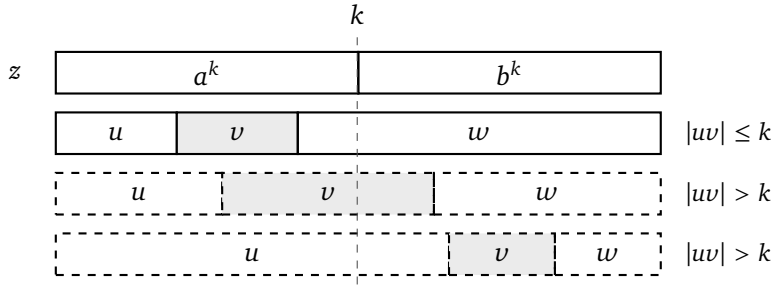


FIGURA 3.23 *La stringa $z = a^k b^k$ e alcune sue suddivisioni: se uv è contenuta in a^k il vincolo è rispettato, mentre le suddivisioni tratteggiate, in cui v contiene almeno una b, hanno $|uv| > k$.*

Dunque $z = uvw$ con uv contenuta in a^k: v è formata solo da a e, poiché $|v| > 0$, ne contiene almeno una.

- $\exists i \in \mathbb{N}.\ uv^i w \notin \mathcal{L}$: ripetendo v cambia solo il numero di a, perché rispetto a z ci sono $i-1$ copie di v in più:

$$z_i = uv^i w = a^{k+(i-1)|v|}b^k,$$

come mostra la Figura 3.24.

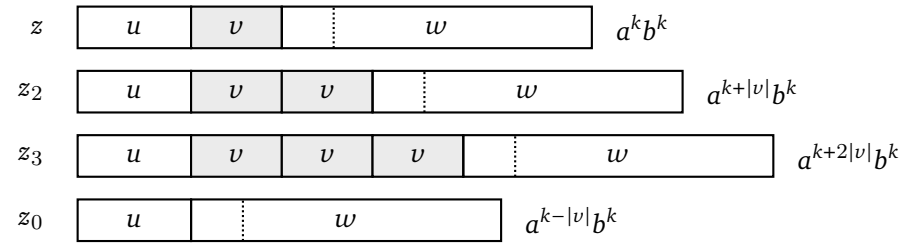


FIGURA 3.24 *Le stringhe $z_i = uv^i w$: ogni copia di v in più aggiunge $|v|$ simboli a, mentre togliendo v ne restano $k - |v|$. La linea punteggiata segna il punto in cui finiscono le a e iniziano le b.*

La condizione di appartenenza a $\mathcal{L}$ è che il numero di a sia uguale al numero di b, cioè $a^n b^m \in \mathcal{L}$ se e solo se $n = m$. Quindi

$$z_i \in \mathcal{L} \iff k + (i-1)|v| = k \iff (i-1)|v| = 0.$$

Basta trovare un solo valore di i: si sceglie $i = 2$. Se fosse $z_2 \in \mathcal{L}$ si avrebbe $|v| = 0$, assurdo per l'ipotesi $|v| > 0$ fatta sulla suddivisione, quindi $z_2 \notin \mathcal{L}$.

Il ragionamento vale per ogni k e per ogni suddivisione che rispetta i vincoli, quindi la negazione della condizione necessaria è verificata: $\mathcal{L}$ non è regolare. ◇

Esempio. Si consideri il linguaggio

$$\mathcal{L} = \{0^n \mid n \text{ è un quadrato perfetto}\} = \{0^n \mid \exists m.\ n = m^2\}.$$

Si dimostra che $\mathcal{L}$ non è regolare. La condizione di appartenenza a $\mathcal{L}$ è

$$0^n \in \mathcal{L} \iff \exists m \in \mathbb{N}.\ n = m^2.$$

Come nell'esempio precedente, si verifica la negazione della condizione necessaria.

- $\forall k \in \mathbb{N}$: si sceglie un valore generico k, sul quale non si può porre alcuna condizione.
- $\exists z \in \mathcal{L}$ con $|z| \geq k$: la scelta $z = 0^k$ non andrebbe bene, perché apparterrebbe a $\mathcal{L}$ solo se k fosse un quadrato perfetto. Si sceglie invece $n = k^2$, cioè

$$z = 0^{k^2},$$

che appartiene a $\mathcal{L}$, perché k^2 è il quadrato di k, e ha lunghezza $|z| = k^2 \geq k$.
- $\forall z = uvw$ con $|uv| \leq k$ e $|v| > 0$: poiché z è formata solo da 0, anche v lo è. Inoltre $|v| \leq |uv| \leq k$, quindi $0 < |v| \leq k$.
- $\exists i \in \mathbb{N}.\ uv^i w \notin \mathcal{L}$: rispetto a z ci sono $i-1$ copie di v in più, quindi

$$z_i = uv^i w = 0^{k^2+(i-1)|v|}.$$

Si sceglie $i = 2$: si ha $z_2 = 0^{k^2+|v|}$, e $z_2 \in \mathcal{L}$ se e solo se $k^2 + |v|$ è un quadrato perfetto.

Che $k^2 + |v|$ non sia un quadrato perfetto si può dimostrare in due modi.

Prima dimostrazione, per assurdo. Se $k^2 + |v|$ fosse un quadrato perfetto, esisterebbe $j \in \mathbb{N}$ tale che $k^2 + |v| = j^2$. Poiché $|v| \geq 1$ si ha $j^2 > k^2$ e, poiché il quadrato è una funzione crescente, $j > k$. Esiste quindi $m \geq 1$ tale che $j = k + m$, da cui

$$k^2 + |v| = (k+m)^2 = k^2 + 2km + m^2 \implies |v| = 2km + m^2 > 2k \geq k.$$

Allora $|uv| = |u| + |v| \geq |v| > k$, assurdo per l'ipotesi $|uv| \leq k$ fatta sulla suddivisione.

Seconda dimostrazione, tra due quadrati consecutivi. Da $0 < |v| \leq k$ si ottiene

$$k^2 < k^2 + |v| \leq k^2 + k < k^2 + 2k + 1 = (k+1)^2.$$

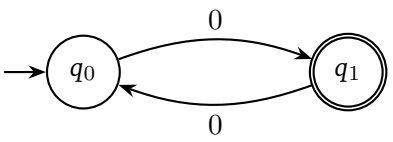
Il numero $k^2 + |v|$ cade quindi strettamente tra due quadrati consecutivi, e tra due quadrati consecutivi non ci sono altri quadrati.

In entrambi i casi $z_2 \notin \mathcal{L}$. Il ragionamento vale per ogni k e per ogni suddivisione che rispetta i vincoli, quindi $\mathcal{L}$ non è regolare. ◇

Esempio. Si consideri il linguaggio

$$\mathcal{L} = \{0^n 0^{n+1} \mid n \in \mathbb{N}\} = \{0^{2n+1} \mid n \in \mathbb{N}\},$$

ovvero l'insieme delle stringhe di 0 di lunghezza dispari: 0, 000, 00000 e così via. Nonostante la somiglianza con $a^n b^n$, $\mathcal{L}$ è regolare: lo riconosce il DFA della Figura 3.25, che ricorda soltanto se il numero di simboli letti è pari (q_0) o dispari (q_1), e lo denota l'espressione regolare $0(00)^*$.



δ	0
q_0	q_1
q_1	q_0

FIGURA 3.25 *DFA che riconosce le stringhe di 0 di lunghezza dispari e la sua matrice di transizione.*

Non basterebbe restare nello stato finale dopo il primo 0: l'automa accetterebbe anche 00, che non sta in $\mathcal{L}$, perché 2 non è della forma $2n + 1$.

A differenza di $a^n b^n$, qui il simbolo è uno solo: non ci sono due parti da confrontare e conta solo la lunghezza totale. Per sapere se è dispari basta una memoria finita. ◇

Esempio. Si consideri il linguaggio

$$\mathcal{L}' = \{0^n 1^{n+1} \mid n \in \mathbb{N}\}.$$

Rispetto all'esempio precedente cambia solo il secondo simbolo, ma è proprio questo a contare: le due parti ora si distinguono e il numero di 1 dipende da quanti 0 sono stati letti, che non è limitato. Per questo $\mathcal{L}'$ non è regolare.

Con il pumping lemma: per ogni $k \in \mathbb{N}$ si sceglie $z = 0^k 1^{k+1}$, che appartiene a $\mathcal{L}'$ e ha lunghezza $|z| = 2k + 1 \geq k$. In ogni suddivisione con $|uv| \leq k$ e $|v| > 0$, v contiene solo 0, quindi

$$z_2 = 0^{k+|v|} 1^{k+1}.$$

In $\mathcal{L}'$ il numero di 1 è il numero di 0 più uno, quindi $z_2 \in \mathcal{L}'$ solo se $k + 1 = k + |v| + 1$, cioè se $|v| = 0$, che è assurdo. Dunque $z_2 \notin \mathcal{L}'$ e $\mathcal{L}'$ non è regolare. ◇

Esempio. Si consideri il linguaggio

$$\mathcal{L}'' = \{0^n 10^{n+1} \mid n \in \mathbb{N}\}.$$

Le stringhe sono fatte quasi solo di 0, come nel linguaggio delle stringhe di lunghezza dispari, ma qui l'1 fa da separatore: le due parti si distinguono e vanno confrontate, quindi $\mathcal{L}''$ non è regolare.

Con il pumping lemma: per ogni $k \in \mathbb{N}$ si sceglie $z = 0^k 10^{k+1}$, che appartiene a $\mathcal{L}''$ e ha lunghezza $|z| = 2k + 2 \geq k$. In ogni suddivisione con $|uv| \leq k$ e $|v| > 0$, v contiene solo zeri che precedono l'1, quindi

$$z_2 = 0^{k+|v|} 10^{k+1}.$$

In $\mathcal{L}''$ gli zeri dopo l'1 sono uno in più di quelli prima, quindi $z_2 \in \mathcal{L}''$ solo se $k + 1 = k + |v| + 1$, cioè se $|v| = 0$, che è assurdo. Dunque $z_2 \notin \mathcal{L}''$ e $\mathcal{L}''$ non è regolare. ◇

Esempio. Sia $m \in \mathbb{N}$ fissato e si consideri il linguaggio, indicato anche con $\mathcal{L}(m)$,

$$\mathcal{L}_m = \{0^m 10^{m+n} \mid n \in \mathbb{N}\}.$$

Poiché $0^m 10^{m+n} = 0^m 10^m 0^n$, il linguaggio è denotato dall'espressione regolare $0^m 10^m 0^*$, quindi è regolare. Qui m non è un input dell'automa, ma un parametro stabilito prima di costruirlo: c'è un automa diverso per ogni m, come quello della Figura 3.26, e contare fino a un numero fissato richiede solo un numero finito di stati.

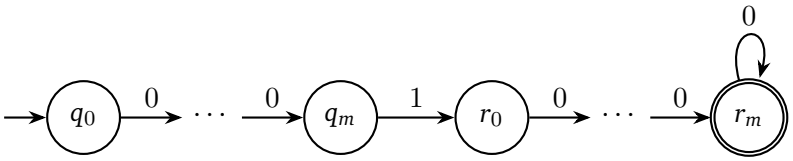


FIGURA 3.26 *DFA che riconosce $\mathcal{L}_m$: lo stato pozzo, non finale, in cui finiscono tutte le altre letture è omesso.*

L'automa legge m zeri, un 1 e altri m zeri. Raggiunto r_m, ogni altro zero non cambia nulla: n non va memorizzato, perché non deve essere confrontato con nessun altro valore. ◇

Esempio. Si consideri ora il linguaggio

$$\mathcal{L} = \{0^m 10^{m+n} \mid n, m \in \mathbb{N}\} = \bigcup_{m \in \mathbb{N}} \mathcal{L}_m.$$

Ogni $\mathcal{L}_m$ è regolare, ma $\mathcal{L}$ non lo è: la chiusura rispetto all'unione (Teorema 6) vale solo per un numero finito di linguaggi. Ora m non è più fissato, quindi bisognerebbe ricordare quanti zeri sono stati letti prima dell'1, un numero che non è limitato.

Con il pumping lemma: per ogni $k \in \mathbb{N}$ si sceglie $z = 0^k 10^k$, che appartiene a $\mathcal{L}$, con $m = k$ e $n = 0$, e ha lunghezza $|z| = 2k + 1 \geq k$. In ogni suddivisione con $|uv| \leq k$ e $|v| > 0$, v contiene solo zeri che precedono l'1, quindi

$$z_2 = 0^{k+|v|} 10^k.$$

Dopo l'1 ci sono k zeri, ma ne servirebbero almeno $k + |v|$, quindi $z_2 \notin \mathcal{L}$ e $\mathcal{L}$ non è regolare.

Qui la scelta di i conta: con $i = 0$ si otterrebbe $0^{k-|v|}10^k$, che appartiene a $\mathcal{L}$, perché togliendo zeri prima dell'1 la condizione resta vera. Serve quindi un $i \geq 2$.

Anche la scelta $z = 0^k10^{2k}$, con $m = n = k$, porta alla stessa conclusione, ma ha un margine di k zeri: $i = 2$ non basta, perché $k + |v| \leq 2k$ e quindi $z_2 \in \mathcal{L}$. Si sceglie allora $i = k + 2$, che può dipendere da k perché nella negazione $\exists i$ viene dopo $\forall k$:

$$z_{k+2} = 0^{k+(k+1)|v|}10^{2k}.$$

Si ha quindi

$$z_{k+2} \in \mathcal{L} \iff 2k \geq k + (k+1)|v| \iff k \geq (k+1)|v|.$$

Poiché $|v| \geq 1$, si ha $(k+1)|v| \geq k + 1 > k$, quindi la condizione è falsa e $z_{k+2} \notin \mathcal{L}$. La strada è valida, ma richiede più conti rispetto a $z = 0^k10^k$, che non ha margine. ◇

4 Linguaggi Context-Free

4.1 Grammatiche libere dal contesto

Finora si sono visti i linguaggi regolari. Il linguaggio $\{a^n b^n \mid n \in \mathbb{N}\}$, però, non è regolare: per descrivere linguaggi di questo tipo si introducono i linguaggi *context-free* (CF).

Definizione (Grammatica libera dal contesto). *Una* grammatica libera dal contesto *(CF) è una quadrupla $\mathcal{G} = \langle V, T, P, S \rangle$, dove:*

- *V è un insieme finito di simboli detti* non terminali*;*
- *T è un insieme finito di simboli detti* terminali*, con $V \cap T = \emptyset$;*
- *P è un insieme finito di* produzioni*, ovvero regole di sostituzione, della forma $A \to \alpha$, dove $A \in V$ è un simbolo non terminale e $\alpha \in (V \cup T)^*$ è una sequenza di simboli in $V \cup T$;*
- *$S \in V$ è il simbolo* iniziale*.*

I simboli di V fanno da segnaposto: indicano una parte della stringa ancora da costruire. I simboli di T si chiamano *terminali* perché, una volta scritti, sono definitivi: il lato sinistro di una produzione è sempre un non terminale, quindi nessuna produzione li riscrive. Il simbolo iniziale S è il non terminale da cui si parte per generare le stringhe, come q_0 in un automa: con le stesse produzioni, un simbolo iniziale diverso può generare un altro linguaggio. La forma delle produzioni determina il tipo di grammatica.

Le produzioni di P si dicono *libere dal contesto* (CF) perché $A \to \alpha$ significa che A si può sostituire con α *indipendentemente* dal suo contesto, ovvero dai simboli presenti prima o dopo A.

Esempio. Le espressioni booleane sono generate dalla grammatica

$$\mathcal{G} = \langle \{E\},\ \{\text{or}, \text{and}, \text{not}, (,), 0, 1\},\ P,\ E \rangle,$$

con $V = \{E\}$, $T = \{\text{or}, \text{and}, \text{not}, (,), 0, 1\}$, simbolo iniziale E e produzioni

$$\begin{aligned} E &\to 0 \mid 1, \\ E &\to (E \text{ or } E), \\ E &\to (E \text{ and } E), \\ E &\to (\text{not } E). \end{aligned}$$

Il simbolo | si legge *oppure*: $E \to 0 \mid 1$ equivale alle due produzioni $E \to 0$ ed $E \to 1$.

La stringa $((1 \text{ and } 0) \text{ or } 0)$ si ottiene partendo da E e sostituendo a ogni passo una sola E con una produzione:

$$\begin{aligned} E &\Rightarrow (E \text{ or } E) && (E \to (E \text{ or } E)) \\ &\Rightarrow ((E \text{ and } E) \text{ or } E) && (E \to (E \text{ and } E) \text{ sulla prima } E) \\ &\Rightarrow ((E \text{ and } E) \text{ or } 0) && (E \to 0 \text{ sull'ultima } E) \\ &\Rightarrow ((1 \text{ and } E) \text{ or } 0) && (E \to 1 \text{ sulla prima } E) \\ &\Rightarrow ((1 \text{ and } 0) \text{ or } 0) && (E \to 0 \text{ sulla } E \text{ rimasta}). \end{aligned}$$

◇

4.2 LINGUAGGIO GENERATO DA $\mathcal{G}$

Una produzione $(A \to \beta) \in P$ dice che A può essere sostituito con $\beta \in (V \cup T)^*$, qualunque cosa ci sia prima e dopo A:

$$\alpha A \gamma \Rightarrow \alpha \beta \gamma, \qquad \alpha, \gamma \in (V \cup T)^*.$$

Le stringhe α e γ sono il contesto di A e possono contenere qualsiasi combinazione di terminali e non terminali: indipendentemente dalla loro natura, A può essere sostituito con β, perché la produzione guarda solo A. È questo il significato di *libera dal contesto*.

DEFINIZIONE (Derivazione immediata). *Se $(A \to \beta) \in P$ e $\alpha, \gamma \in (V \cup T)^*$, si dice che $\alpha\beta\gamma$* deriva immediatamente *da $\alpha A \gamma$, e si scrive $\alpha A \gamma \Rightarrow \alpha\beta\gamma$.*

Una derivazione immediata è quindi l'applicazione di una sola produzione a una sola occorrenza di A: α e γ restano invariate e solo A viene sostituita da β, come mostra la Figura 4.1.

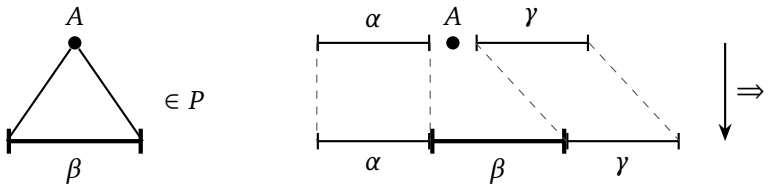


FIGURA 4.1 *A sinistra la produzione $(A \to \beta) \in P$; a destra la derivazione immediata $\alpha A \gamma \Rightarrow \alpha\beta\gamma$, in cui α e γ restano invariate.*

DEFINIZIONE (Derivazione in più passi). *Siano $\alpha_1, \alpha_2, \ldots, \alpha_i \in (V \cup T)^*$ sequenze di simboli in $V \cup T$ tali che da ognuna derivi immediatamente la successiva:*

$$\forall j \leq i - 1.\ \alpha_j \Rightarrow \alpha_{j+1}.$$

Si dice che da α_1 deriva *α_i in $i - 1$ passi, e si scrive*

$$\alpha_1 \overset{\mathcal{G}}{\Rightarrow}_{i-1} \alpha_i.$$

Definizione (Simbolo di generazione). *Il* simbolo di generazione $\overset{\mathcal{G}}{\Rightarrow}$ *è la chiusura transitiva (Sezione 0.2.4) di* $\Rightarrow$*:* $\alpha \overset{\mathcal{G}}{\Rightarrow} \beta$ *indica che* β *si ottiene da* α *con una o più derivazioni immediate nella grammatica* $\mathcal{G}$*.*

Esempio. Nella grammatica delle espressioni booleane, a ogni passo si applica una sola produzione:

$$\begin{aligned} E &\Rightarrow (E \text{ and } E) \\ &\Rightarrow ((E \text{ or } E) \text{ and } E) \\ &\Rightarrow ((E \text{ or } E) \text{ and } 1) \\ &\Rightarrow ((0 \text{ or } E) \text{ and } 1) \\ &\Rightarrow ((0 \text{ or } 1) \text{ and } 1) \in T^*. \end{aligned}$$

I passi sono cinque, quindi si scrive

$$E \overset{\mathcal{G}}{\Rightarrow}_5 ((0 \text{ or } 1) \text{ and } 1).$$

◇

In generale, per indicare un numero qualsiasi di passi si usa il pedice $*$:

$$\alpha \overset{\mathcal{G}}{\Rightarrow}_* \beta \iff \exists i \in \mathbb{N}.\ \alpha \overset{\mathcal{G}}{\Rightarrow}_i \beta.$$

Poiché $\mathbb{N}$ comprende lo 0, vale anche $\alpha \overset{\mathcal{G}}{\Rightarrow}_0 \alpha$: con zero passi ogni stringa deriva da se stessa, ad esempio $E \text{ or } E \overset{\mathcal{G}}{\Rightarrow}_0 E \text{ or } E$. Il caso $i = 0$ è quello della *chiusura riflessiva*, mentre i casi $i \geq 1$ danno la chiusura transitiva, quindi $\overset{\mathcal{G}}{\Rightarrow}_*$ è la chiusura riflessiva e transitiva di $\Rightarrow$.

Definizione (Linguaggio generato da una grammatica). *Sia* $\mathcal{G} = \langle V, T, P, S \rangle$ *una grammatica CF. Il* linguaggio generato *da* $\mathcal{G}$ *è*

$$\mathcal{L}(\mathcal{G}) = \{x \in T^* \mid S \overset{\mathcal{G}}{\Rightarrow}_* x\},$$

ovvero l'insieme di tutte le sequenze di simboli terminali che si possono generare a partire dal simbolo iniziale S*.*

Definizione (Linguaggio CF). *Un linguaggio* $\mathcal{L} \subseteq T^*$ *è un* linguaggio CF *se esiste una grammatica CF* $\mathcal{G}$ *che lo genera, ovvero tale che*

$$\mathcal{L} = \mathcal{L}(\mathcal{G}).$$

Definizione (Grammatiche equivalenti). *Due grammatiche* $\mathcal{G}_1$ *e* $\mathcal{G}_2$ *sono* equivalenti *se generano lo stesso linguaggio, ovvero se* $\mathcal{L}(\mathcal{G}_1) = \mathcal{L}(\mathcal{G}_2)$*.*

Esempio. Il linguaggio $\{0^n 1^n \mid n \in \mathbb{N}\}$ non è regolare, come $\{a^n b^n \mid n \in \mathbb{N}\}$, ma è CF: è generato dalla grammatica con produzioni $S \to 0S1 \mid \varepsilon$. Scrivere solo le produzioni è un'abbreviazione dell'intera quadrupla:

$$S \to 0S1 \mid \varepsilon \quad \equiv \quad \langle \{S\},\ \{0, 1\},\ \{S \to 0S1,\ S \to \varepsilon\},\ S \rangle.$$

Ogni applicazione di $S \to 0S1$ aggiunge uno 0 e un 1 insieme, quindi il loro numero resta sempre uguale, senza bisogno di contarli. Ad esempio,

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011.$$

◇

4.2.1 Come dimostrare che un linguaggio è CF

Per dimostrare che un linguaggio $\mathcal{L}$ è CF si costruisce una grammatica $\mathcal{G}$ e si dimostra che $\mathcal{L} = \mathcal{L}(\mathcal{G})$, mediante la duplice inclusione:

- $\mathcal{L} \subseteq \mathcal{L}(\mathcal{G})$: se $x \in \mathcal{L}$, allora $S \overset{\mathcal{G}}{\Rightarrow}_* x$, cioè esiste una derivazione per x in $\mathcal{G}$. Si dimostra per induzione su $|x|$;
- $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}$: se $S \overset{\mathcal{G}}{\Rightarrow}_i x$, cioè se esiste una derivazione per x in $\mathcal{G}$, allora $x \in \mathcal{L}$. Si dimostra per induzione su i, la lunghezza della derivazione.

Servono entrambe: una grammatica potrebbe generare tutte le stringhe di $\mathcal{L}$ e anche altre che non vi appartengono. Ad esempio, per $\mathcal{L} = \{0^n 1^n \mid n \in \mathbb{N}\}$ la grammatica con produzioni $S \to 0S1 \mid \varepsilon \mid 0$ genera ogni stringa di $\mathcal{L}$, ma anche 0 e 001, che non vi appartengono:

$$S \Rightarrow 0, \qquad\qquad S \Rightarrow 0S1 \Rightarrow 001.$$

Per questa grammatica vale la prima inclusione ma non la seconda.

La grammatica si può scegliere liberamente: la definizione di linguaggio CF chiede solo che ne esista una, quindi basta esibirla, purché generi tutte e sole le stringhe di $\mathcal{L}$. Di solito conviene quella più semplice, perché ha meno casi da dimostrare. Se invece la grammatica è data, bisogna dimostrare che proprio quella genera $\mathcal{L}$.

Esempio. Si dimostra che il linguaggio $\mathcal{L} = \{0^n 1^n \mid n \in \mathbb{N}\}$ è generato dalla grammatica $\mathcal{G}$ con produzioni $S \to 0S1 \mid \varepsilon$, verificando le due inclusioni.

Prima inclusione: $\mathcal{L} \subseteq \mathcal{L}(\mathcal{G})$, per induzione sulla lunghezza di x.

Base. La stringa più corta di $\mathcal{L}$ è ε, e grazie alla produzione $S \to \varepsilon$ si ha $S \Rightarrow \varepsilon$.

Ipotesi induttiva. Per ogni $x \in \mathcal{L}$ con $|x| \leq m$ esiste $S \overset{\mathcal{G}}{\Rightarrow}_* x$.

Passo induttivo. Sia $x \in \mathcal{L}$ con $|x| = m + 1$.

1. Per definizione di $\mathcal{L}$ si ha $x = 0^h 1^h$, con $h \geq 1$ perché $|x| > 0$. Si riscrive x in termini di una stringa più corta x', a cui applicare l'ipotesi induttiva:

   $$x = 0^{h-1}\,01\,1^{h-1}, \qquad\qquad x' = 0^{h-1}1^{h-1} \in \mathcal{L}.$$

   Poiché $|x'| = m - 1$, l'ipotesi deve valere per tutte le stringhe con $|x| \leq m$, e non solo per quelle lunghe m.
2. Per ipotesi induttiva esiste $S \overset{\mathcal{G}}{\Rightarrow}_* x'$. L'unica produzione che elimina S è $S \to \varepsilon$, quindi la derivazione termina necessariamente così:

   $$S \overset{\mathcal{G}}{\Rightarrow}_* 0^{h-1}S1^{h-1} \Rightarrow 0^{h-1}1^{h-1} = x'.$$

3. Ci si ferma prima dell'ultimo passo, perché x' non contiene più S, e al posto di $S \to \varepsilon$ si applicano $S \to 0S1$ e poi $S \to \varepsilon$:

   $$S \overset{\mathcal{G}}{\Rightarrow}_* 0^{h-1}S1^{h-1} \Rightarrow 0^{h-1}\,0S1\,1^{h-1} \Rightarrow 0^{h-1}\,01\,1^{h-1} = x.$$

Seconda inclusione: $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}$, per induzione sulla lunghezza i della derivazione. Poiché $x \in \mathcal{L}(\mathcal{G})$ se e solo se esiste $i \in \mathbb{N}$ tale che $S \overset{\mathcal{G}}{\Rightarrow}_i x$, bisogna dimostrare l'implicazione

$$\forall x \in T^*.\ \left(S \overset{\mathcal{G}}{\Rightarrow}_i x \implies x \in \mathcal{L}\right).$$

Base. La derivazione più corta di una stringa di soli terminali è $S \Rightarrow \varepsilon$, ed $\varepsilon = 0^0 1^0 \in \mathcal{L}$.
Ipotesi induttiva. Ogni stringa di soli terminali generata in $i \geq 1$ passi appartiene a $\mathcal{L}$.

Passo induttivo. Sia $S \overset{\mathcal{G}}{\Rightarrow}_{i+1} x$ con $x \in T^*$. Da questa derivazione se ne ricava una lunga i, per poter applicare l'ipotesi induttiva.

1. Si separano gli ultimi due passi. L'ultimo, l'$(i+1)$-esimo, elimina S, quindi applica $S \to \varepsilon$. Il penultimo, l'i-esimo, porta a una stringa che contiene ancora S, quindi applica $S \to 0S1$, l'unica produzione che la mantiene. Con $x_1 = x_1'0$ e $x_2 = 1x_2'$:

$$\begin{aligned} S \overset{\mathcal{G}}{\Rightarrow}_{i+1} x \quad &\equiv \quad S \overset{\mathcal{G}}{\Rightarrow}_{i} x_1 S x_2 \Rightarrow x_1 x_2 = x \\ &\equiv \quad S \overset{\mathcal{G}}{\Rightarrow}_{i-1} x_1' S x_2' \Rightarrow x_1' \, 0S1 \, x_2' \Rightarrow x_1' \, 01 \, x_2' = x. \end{aligned}$$

2. Saltando il passo con $S \to 0S1$ si ottiene una derivazione lunga i:

$$S \overset{\mathcal{G}}{\Rightarrow}_{i-1} x_1' S x_2' \Rightarrow x_1' x_2'.$$

   Per come è fatta $\mathcal{G}$, x_1' contiene solo 0 e x_2' solo 1.
3. Per ipotesi induttiva $x_1' x_2' \in \mathcal{L}$, quindi esiste h tale che $x_1' = 0^h$ e $x_2' = 1^h$, e

$$x = x_1' \, 01 \, x_2' = 0^h \, 01 \, 1^h = 0^{h+1} 1^{h+1} \in \mathcal{L}.$$

Le due inclusioni danno $\mathcal{L} = \mathcal{L}(\mathcal{G})$, quindi $\mathcal{L}$ è CF. ◇

4.3 Alberi di derivazione

Definizione (Albero di derivazione). *Sia $\mathcal{G} = \langle V, T, P, S \rangle$ una grammatica CF. Un* albero di derivazione *per $\mathcal{G}$ è un albero tale che:*

1. *ogni vertice ha un'etichetta in $V \cup T \cup \{\varepsilon\}$;*
2. *la radice ha etichetta in V (ad esempio il simbolo iniziale $S \in V$);*
3. *ogni vertice interno, cioè non foglia, ha etichetta in V;*
4. *se un vertice n è etichettato con $A \in V$ e $n_1, n_2, \ldots, n_k$, da sinistra a destra, sono i figli di n, etichettati con $X_1, X_2, \ldots, X_k \in V \cup T \cup \{\varepsilon\}$, allora*

$$(A \to X_1 X_2 \cdots X_k) \in P;$$

5. *se un vertice n ha etichetta ε, allora n è una foglia ed è l'unico figlio del vertice padre;*
6. *se A è l'etichetta di un vertice n e $X \in V \cup T$ è l'etichetta di un vertice n_1, allora n_1 è figlio di n se $(A \to X) \in P$.*

La condizione 4 è illustrata nella Figura 4.2: i figli di un vertice, letti da sinistra a destra, formano il lato destro della produzione applicata a quel vertice.

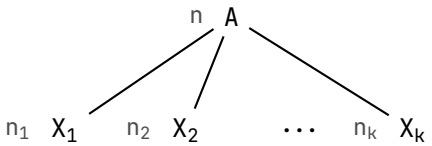


Figura 4.2 *Un vertice n etichettato con A e i suoi figli $n_1, \ldots, n_k$: la produzione $A \to X_1 X_2 \cdots X_k$ appartiene a P.*

Esempio. Nella grammatica delle espressioni booleane, l'albero della Figura 4.3 rappresenta una derivazione di $((0 \text{ or } 1) \text{ and } (\text{not } 0))$. Prendendo le foglie da sinistra a destra si ottiene proprio la stringa generata, che non contiene simboli non terminali: tutti i non terminali sono vertici interni.

La E di sinistra, insieme a tutti i suoi discendenti, forma un *sottoalbero*, evidenziato in grigio nella figura. È a sua volta un albero di derivazione, con radice E, e le sue foglie formano $(0 \text{ or } 1)$.

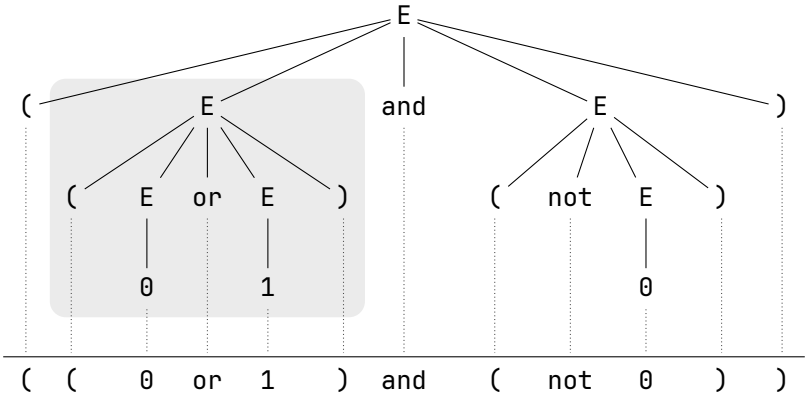


FIGURA 4.3 *Albero di derivazione di* $((0 \text{ or } 1) \text{ and } (\text{not } 0))$*: le foglie, riportate in basso e lette da sinistra a destra, formano la stringa generata. In grigio il sottoalbero con radice nella* E *di sinistra.*

◇

TEOREMA 10 (Derivazioni e alberi). *Sia* $\mathcal{G} = \langle V, T, P, S \rangle$ *una grammatica CF. Allora* $S \overset{\mathcal{G}}{\Rightarrow}_* x$ *se e solo se esiste un albero di derivazione per* $\mathcal{G}$ *con radice* S *la cui sequenza di foglie, letta da sinistra a destra, è uguale a* x*.*

Il teorema è dato senza dimostrazione.

4.4 AMBIGUITÀ

DEFINIZIONE (Grammatica ambigua). *Una grammatica CF* $\mathcal{G}$ *è detta* ambigua *se esiste almeno una stringa generata da* $\mathcal{G}$ *per la quale esiste più di un albero di derivazione.*

Esempio. Si consideri la grammatica con produzioni

$$E \to E + E \mid E * E \mid 0 \mid 1 \mid 2.$$

La stringa $2 * 0 + 1$, che ha la forma $E * E + E$, ha due alberi di derivazione, mostrati nella Figura 4.4. Nel primo la produzione applicata alla radice è $E \to E * E$, e la stringa si legge come $2 * (0 + 1) = 2$; nel secondo è $E \to E + E$, e la stringa si legge come $(2 * 0) + 1 = 1$. I due alberi raggruppano i simboli in modo diverso e danno quindi due valori diversi: la grammatica è ambigua.

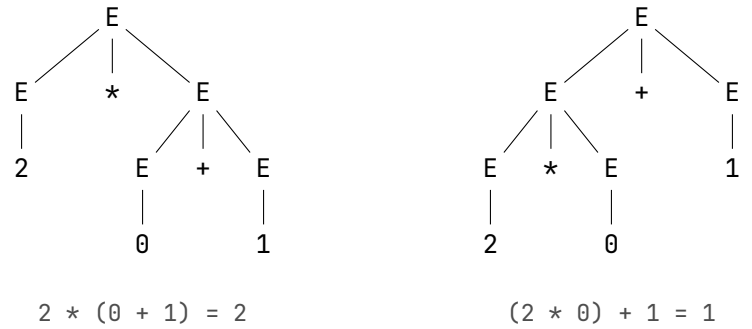


FIGURA 4.4 *I due alberi di derivazione di* $2 * 0 + 1$*: a sinistra la radice applica* $E \to E * E$*, a destra* $E \to E + E$*.*

◇

DEFINIZIONE (Linguaggio inerentemente ambiguo). *Un linguaggio CF* $\mathcal{L}$ *è detto* inerentemente ambiguo *se ogni grammatica CF che lo genera è ambigua:*

$$\forall \mathcal{G}\ \textit{CF}.\ \big(\mathcal{L} = \mathcal{L}(\mathcal{G}) \implies \mathcal{G}\ \textit{è ambigua}\big).$$

In questo caso l'ambiguità è una proprietà del linguaggio e non della singola grammatica: non si può eliminare cambiando grammatica.

4.5 FORME NORMALI

In una grammatica CF ogni produzione ha la forma $A \to \beta$: a sinistra della freccia c'è sempre una sola variabile $A \in V$, a destra una stringa qualsiasi $\beta \in (V \cup T)^*$. Una *forma normale* descrive le produzioni fissando la forma che possono avere; poiché il lato sinistro è già fissato, i vincoli riguardano solo il lato destro β. Le due forme normali principali sono la forma normale di Chomsky e la forma normale di Greibach.

DEFINIZIONE (Forma normale di Chomsky). *Una grammatica CF* $\mathcal{G} = \langle V, T, P, S\rangle$ *è in* forma normale di Chomsky *se ogni sua produzione è di uno dei due tipi*

$$A \to BC \qquad \textit{oppure} \qquad A \to a,$$

con $A, B, C \in V$ *e* $a \in T$*.*

Con produzioni di questi due tipi non si può generare ε: se $\varepsilon \in \mathcal{L}(\mathcal{G})$ si ammette in più la produzione $S \to \varepsilon$ (Sezione 4.5.1).

DEFINIZIONE (Forma normale di Greibach). *Una grammatica CF* $\mathcal{G} = \langle V, T, P, S\rangle$ *è in* forma normale di Greibach *se ogni sua produzione è del tipo*

$$A \to a\gamma,$$

con $A \in V$*,* $a \in T$ *e* $\gamma \in (V \cup T)^*$*.*

Il lato destro di ogni produzione inizia quindi con un terminale.

4.5.1 *Semplificazioni*

Per portare una grammatica CF in forma normale di Chomsky occorre prima semplificarla, imponendo tre condizioni, ciascuna ottenuta eliminando dalla grammatica un tipo di simboli o di produzioni:

1. Eliminazione dei simboli inutili: ogni simbolo, terminale o non terminale, deve essere coinvolto in almeno una derivazione di una stringa di terminali (Sezione 4.6).
2. Eliminazione delle ε-produzioni: non devono esserci *ε-produzioni*, cioè produzioni della forma $A \to \varepsilon$. L'unica deroga riguarda il caso $\varepsilon \in \mathcal{L}(\mathcal{G})$: allora è ammessa la sola produzione $S \to \varepsilon$.
3. Eliminazione delle produzioni unitarie: non devono esserci *produzioni unitarie*, cioè produzioni della forma $A \to B$ con $A, B \in V$.

4.6 Eliminazione dei simboli inutili

In questa sezione $\mathcal{G} = \langle V, T, P, S\rangle$ è una grammatica CF.

Definizione (Simbolo utile). *Un simbolo $X \in V \cup T$ è* utile *se esiste almeno una derivazione*

$$S \overset{\mathcal{G}}{\Rightarrow}_* \alpha X \beta \overset{\mathcal{G}}{\Rightarrow}_* w,$$

con $\alpha, \beta \in (V \cup T)^*$ e $w \in T^*$. Un simbolo che non è utile è detto inutile.

La derivazione ha due parti. La prima, $S \overset{\mathcal{G}}{\Rightarrow}_* \alpha X \beta$, dice che X è *raggiungibile* dal simbolo iniziale S; la seconda, $\alpha X \beta \overset{\mathcal{G}}{\Rightarrow}_* w$, dice che X è coinvolto nella generazione di una sequenza di terminali. Deve terminare l'intera stringa $\alpha X \beta$, non solo X. Un simbolo inutile non contribuisce al linguaggio generato: si può eliminare, insieme alle produzioni che lo contengono, senza cambiare $\mathcal{L}(\mathcal{G})$.

Quindi esiste almeno una derivazione da S a una sequenza w di soli simboli terminali che usa X. L'eliminazione dei simboli inutili avviene in due fasi.

Fase 1. Si eliminano i simboli non terminali che non portano a sequenze di terminali. L'idea è partire dai simboli terminali e procedere all'indietro nelle produzioni, cercando i simboli collegati. Si definisce l'operatore

$$\Gamma(W) = \{A \in V \mid \exists \alpha \in (T \cup W)^*.\ A \to \alpha \in P\},$$

dove $W \subseteq V$ contiene simboli di cui si sa già che portano a terminali: se $\alpha \in (T \cup W)^*$, ogni simbolo di α è un terminale o una variabile di W, quindi anche da A si deriva una stringa di terminali. Iterativamente si costruisce

$$\Gamma^0(\emptyset) = \emptyset, \qquad \Gamma^{i+1}(\emptyset) = \Gamma(\Gamma^i(\emptyset)),$$

dove Γ^i è Γ applicato i volte.

Discesa. Ogni chiamata richiama $\Gamma^{i-1}(\emptyset)$, togliendo un'applicazione di Γ, fino al caso base $\Gamma^0(\emptyset)$:

$$\Gamma^i(\emptyset) = \Gamma(\Gamma^{i-1}(\emptyset)) = \Gamma(\Gamma(\Gamma^{i-2}(\emptyset))) = \cdots = \underbrace{\Gamma(\Gamma(\cdots \Gamma}_{i \text{ volte}}(\Gamma^0(\emptyset)) \cdots)).$$

Risalita. Ogni livello applica Γ all'insieme restituito dal livello sottostante. Il caso base restituisce $\Gamma^0(\emptyset) = \emptyset$, ed è l'unico punto in cui si usa $\emptyset$. Il primo livello parte da $W = \emptyset$ e, poiché $(T \cup \emptyset)^* = T^*$,

$$\Gamma^1(\emptyset) = \Gamma(\emptyset) = \{A \in V \mid \exists \alpha \in T^*.\ A \to \alpha \in P\},$$

cioè le variabili con una produzione fatta di soli terminali. Il secondo livello parte dall'insieme appena restituito, $W = \Gamma^1(\emptyset)$:

$$\Gamma^2(\emptyset) = \Gamma(\Gamma^1(\emptyset)) = \{A \in V \mid \exists \alpha \in (T \cup \Gamma^1(\emptyset))^*.\ A \to \alpha \in P\},$$

e così via fino a $\Gamma^i(\emptyset)$, che è quindi l'insieme dei simboli non terminali da cui si deriva una stringa di terminali con un albero di derivazione di altezza al più i.

Ci si ferma quando $\Gamma^{i+1}(\emptyset) = \Gamma^i(\emptyset)$, il che accade entro $|V|$ passi; le variabili che non stanno nell'ultimo insieme si eliminano, insieme alle produzioni che le contengono.

Fase 2. Si eliminano tutti i simboli mai raggiunti dal simbolo iniziale. L'idea è costruire l'insieme dei simboli raggiunti da S, partendo da S stesso:

$$\Gamma^0 = \{S\}, \qquad \Gamma^{i+1} = \Gamma^i \cup \{X \in V \cup T \mid \exists A \in \Gamma^i.\ A \to \alpha X \beta \in P\},$$

cioè l'insieme dei simboli che da S sono generabili in al più i passi.[1] Ci si ferma quando $\Gamma^{i+1} = \Gamma^i$; i simboli che non stanno nell'ultimo insieme si eliminano, insieme alle produzioni che li contengono.

Esempio. Si consideri la grammatica

$$\mathcal{G} = \langle \{A, B, C, D, E, S\}, \{0, 1, 2, 3\}, P, S \rangle$$

con produzioni

$$\begin{aligned}
&S \to \varepsilon \mid A \mid B \mid D,\\
&A \to 0 \mid 0A0 \mid B \mid E,\\
&B \to 1 \mid 1B1 \mid A \mid \varepsilon,\\
&C \to A \mid B \mid 2,\\
&D \to \varepsilon \mid D,\\
&E \to \varepsilon.
\end{aligned}$$

Fase 1. Si parte dai simboli terminali e si procede all'indietro nelle produzioni:

$$\begin{aligned}
&\Gamma^0(\emptyset) = \emptyset,\\
&\Gamma^1(\emptyset) = \{A, B, C\} && (\text{da } A \to 0,\ B \to 1,\ C \to 2),\\
&\Gamma^2(\emptyset) = \{A, B, C, S\} && (\text{da } S \to A \mid B),\\
&\Gamma^3(\emptyset) = \{A, B, C, S\} = \Gamma^2(\emptyset) && (\text{punto fisso}).
\end{aligned}$$

L'insieme $\Gamma^3(\emptyset)$ contiene tutti i non terminali che raggiungono terminali; D ed E non ci stanno e si eliminano:

$$\begin{aligned}
&\mathcal{G}' = \langle V', T, P', S \rangle,\\
&V' = V \cap \Gamma^3(\emptyset) = \{A, B, C, S\},\\
&P' = P \setminus \{S \to D,\ D \to \varepsilon \mid D,\ E \to \varepsilon,\ A \to E\}.
\end{aligned}$$

[1] Il simbolo iniziale S appartiene a ogni Γ^i.

Fase 2. Le produzioni di $\mathcal{G}'$ sono

$$\begin{aligned} &S \to \varepsilon \mid A \mid B,\\ &A \to B \mid 0A0 \mid 0,\\ &B \to A \mid 1B1 \mid 1 \mid \varepsilon,\\ &C \to A \mid B \mid 2. \end{aligned}$$

Si parte da S e si aggiungono i simboli raggiunti:

$$\begin{aligned} &\Gamma^0 = \{S\},\\ &\Gamma^1 = \{A, B\} \cup \{S\} = \{A, B, S\} && (\text{da } S \to A \mid B),\\ &\Gamma^2 = \{A, B, S, 0, 1\} && (\text{da } A \to 0A0 \mid 0 \text{ e } B \to 1B1 \mid 1),\\ &\Gamma^3 = \Gamma^2 && (\text{punto fisso}). \end{aligned}$$

Si elimina C con le sue produzioni e si eliminano i terminali 2 e 3:

$$\begin{aligned} \mathcal{G}'' &= \langle V'', T'', P'', S\rangle,\\ V'' &= V' \cap \Gamma^3 = \{A, B, S\},\\ T'' &= T \cap \Gamma^3 = \{0, 1\}, \end{aligned}$$

cioè $\mathcal{G}'' = \langle \{A, B, S\}, \{0, 1\}, P'', S\rangle$, con produzioni P''

$$\begin{aligned} &S \to A \mid B \mid \varepsilon,\\ &A \to B \mid 0A0 \mid 0,\\ &B \to A \mid 1B1 \mid 1 \mid \varepsilon. \end{aligned}$$

$\diamond$

4.7 Eliminazione delle ε-produzioni

Si eliminano le produzioni della forma $X \to \varepsilon$ con $X \in V \setminus \{S\}$.

Definizione (Simbolo annullabile). *Un simbolo non terminale $A \in V$ è* annullabile *se da A si può derivare la stringa vuota, cioè*

$$A \overset{\mathcal{G}}{\Rightarrow}_* \varepsilon.$$

Un simbolo può essere annullabile in due modi:

- Direttamente: c'è una produzione $A \to \varepsilon$;
- Indirettamente: c'è una produzione $A \to \alpha$ e tutti i simboli di α sono annullabili, per esempio $A \to B$ con $B \to \varepsilon$, oppure $A \to BC$ con B e C entrambi annullabili.

L'eliminazione dei simboli inutili (Sezione 4.6) non toglie i simboli annullabili: nella Fase 1 spariscono solo le variabili che portano *soltanto* a ε, oppure a niente. Una variabile che porta a ε ma anche a stringhe di terminali è utile, quindi resta.

Passo 1. Si calcolano i simboli annullabili. Si definisce l'operatore

$$\Gamma(W) = \{A \in V \mid \exists A \to \alpha \in P,\ \text{“}\alpha \text{ privata delle variabili in } W \text{ è } \varepsilon\text{”}\},$$

dove W contiene le variabili di cui si sa già che sono annullabili. Da α si cancellano le variabili di W, che possono diventare ε: se non resta niente, ogni simbolo di α era una variabile annullabile, quindi anche A lo è. Se α contiene un terminale, o una variabile che non sta in W, qualcosa resta. In altre parole, la condizione tra virgolette equivale ad $\alpha \in W^*$.

Iterativamente si costruisce Γ^i, l'insieme dei simboli non terminali che generano ε, cioè dei simboli annullabili:

- $\Gamma^0 = \emptyset$ è il caso base;
- Γ^1 contiene le variabili X con una produzione $X \to \varepsilon$;
- Γ^{i+1}: per ogni produzione $X \to \alpha$ si sostituiscono con ε le variabili del lato destro α che stanno in Γ^i. Se resta ε, il lato sinistro X entra in Γ^{i+1}; altrimenti non succede niente;
- ci si ferma quando $\Gamma^{i+1} = \Gamma^i$.

Passo 2. Si modifica la grammatica. Raggiunto il punto fisso, per ogni produzione si applicano tutte le sostituzioni possibili delle variabili annullabili con ε. Se una produzione ha k occorrenze di variabili annullabili, ogni occorrenza è un bit (0 la si tiene, 1 la si toglie) e le combinazioni sono 2^k, come nell'aritmetica binaria. Per esempio, con A e B annullabili, $S \to AbB$ ha $k = 2$ e dà le quattro combinazioni della Tabella 4.1.

A	B	Risultato
0	0	$S \to AbB$ (l'originale)
0	1	$S \to Ab$
1	0	$S \to bB$
1	1	$S \to b$

Tabella 4.1 *Le 2^2 combinazioni per $S \to AbB$ con A e B annullabili: 0 tiene la variabile, 1 la toglie.*

Si aggiungono le produzioni diverse dalle originali, con tre accorgimenti:

- non si aggiunge nessuna $X \to \varepsilon$: da $B \to AA$, la combinazione 11 darebbe $B \to \varepsilon$;
- le combinazioni uguali contano una volta sola: da $B \to AA$, le combinazioni 01 e 10 danno entrambe $B \to A$;
- alla fine si eliminano le ε-produzioni originali; le altre produzioni originali restano.

L'unica eccezione è $S \to \varepsilon$: si ha $\varepsilon \in \mathcal{L}(\mathcal{G})$ se e solo se S è annullabile, cioè se S sta nel punto fisso, e in quel caso togliere $S \to \varepsilon$ cambierebbe il linguaggio. Se S è annullabile senza una produzione $S \to \varepsilon$ scritta, per esempio con $S \to AB$ e A, B annullabili, la produzione $S \to \varepsilon$ va aggiunta, in deroga al primo accorgimento.

Esempio. Si riprende la grammatica $\mathcal{G}''$ dell'esempio precedente, con produzioni

$$\begin{aligned} S &\to A \mid B \mid \varepsilon, \\ A &\to B \mid 0A0 \mid 0, \\ B &\to A \mid 1B1 \mid 1 \mid \varepsilon. \end{aligned}$$

I simboli annullabili sono tre:

- B, per $B \to \varepsilon$;
- A, in modo indiretto: $A \Rightarrow B \Rightarrow \varepsilon$. Non ha una produzione $A \to \varepsilon$, quindi non si vede guardando solo le sue produzioni;

- S, per $S \to \varepsilon$.

  Passo 1. Con la costruzione iterativa:

$$\begin{aligned}
&\Gamma^0 = \emptyset && \text{(caso base)},\\
&\Gamma^1 = \{S, B\} && (S \to \varepsilon,\ B \to \varepsilon),\\
&\Gamma^2 = \{S, B, A\} && (A \to B, \text{ con } B \in \Gamma^1),\\
&\Gamma^3 = \Gamma^2 && \text{(punto fisso)}.
\end{aligned}$$

  Passo 2. Si aggiungono $A \to 00$, da $A \to 0A0$, e $B \to 11$, da $B \to 1B1$; si elimina $B \to \varepsilon$. Da $S \to A$ e $S \to B$ si otterrebbe $S \to \varepsilon$, che c'è già, quindi non si aggiunge; da $A \to B$ e $B \to A$ si otterrebbero $A \to \varepsilon$ e $B \to \varepsilon$, che non si aggiungono per il primo accorgimento. Il risultato è

$$\begin{aligned}
&S \to A \mid B \mid \varepsilon,\\
&A \to B \mid 0A0 \mid 00 \mid 0,\\
&B \to A \mid 1B1 \mid 11 \mid 1,
\end{aligned}$$

cioè

$$\begin{aligned}
P''' &= \big(P'' \setminus \{B \to \varepsilon\}\big) \cup \{A \to 00,\ B \to 11\},\\
\mathcal{G}''' &= \langle \{A, B, S\}, \{0, 1\}, P''', S \rangle.
\end{aligned}$$

◇

4.8 Eliminazione delle produzioni unitarie

Si eliminano le produzioni unitarie, cioè le produzioni della forma $A \to B$ con $A, B \in V$. Una produzione unitaria $X \to Y$ si elimina sostituendo Y con le produzioni di Y: se $Y \to \beta$, allora $X \Rightarrow Y \Rightarrow \beta$, quindi si aggiunge $X \to \beta$.

Esempio. Si riprende la grammatica $\mathcal{G}'''$ dell'esempio precedente, con produzioni

$$\begin{aligned}
&S \to \varepsilon \mid A \mid B,\\
&A \to 0 \mid 00 \mid 0A0 \mid B,\\
&B \to 1 \mid 11 \mid 1B1 \mid A.
\end{aligned}$$

Le produzioni unitarie sono $S \to A$, $S \to B$, $A \to B$ e $B \to A$.

Passo 1. Si elimina $S \to A$. Poiché $S \Rightarrow A \Rightarrow 0$, $S \Rightarrow A \Rightarrow 00$ e così via, in S si sostituisce A con le produzioni di A, cioè $0 \mid 00 \mid 0A0 \mid B$:

$$\begin{aligned}
&S \to \varepsilon \mid 0 \mid 00 \mid 0A0 \mid B,\\
&A \to 0 \mid 00 \mid 0A0 \mid B,\\
&B \to 1 \mid 11 \mid 1B1 \mid A.
\end{aligned}$$

Passo 2. Si elimina $S \to B$, sostituendo B con $1 \mid 11 \mid 1B1$ (le produzioni di A sono già in S, quindi da $B \to A$ non si inserisce $S \to A$):

$$S \to \varepsilon \mid 0 \mid 00 \mid 0A0 \mid 1 \mid 11 \mid 1B1.$$

Passo 3. Si elimina $A \to B$, sostituendo B con $1 \mid 11 \mid 1B1$ (da $B \to A$ si otterrebbe $A \to A$, che non si inserisce):

$$A \to 0 \mid 00 \mid 0A0 \mid 1 \mid 11 \mid 1B1.$$

Passo 4. Si elimina $B \to A$, sostituendo A con le sue produzioni (analogamente ad $A \to A$, non si inserisce $B \to B$). Il risultato è

$$\begin{aligned} S &\to \varepsilon \mid 0 \mid 00 \mid 0A0 \mid 1 \mid 11 \mid 1B1, \\ A &\to 0 \mid 00 \mid 0A0 \mid 1 \mid 11 \mid 1B1, \\ B &\to 1 \mid 11 \mid 1B1 \mid 0 \mid 00 \mid 0A0. \end{aligned}$$

Le tre variabili hanno le stesse produzioni, a parte $S \to \varepsilon$, quindi la grammatica si può scrivere con il solo S:

$$S \to \varepsilon \mid 0 \mid 00 \mid 0S0 \mid 1 \mid 11 \mid 1S1.$$

◇

4.9 Forma normale di Chomsky

Una grammatica CF $\mathcal{G} = \langle V, T, P, S \rangle$ è in forma normale di Chomsky (Sezione 4.5) se ogni produzione è del tipo

$$A \to BC \qquad \text{oppure} \qquad A \to a,$$

con $A, B, C \in V$ e $a \in T$.

Teorema 11 (Forma normale di Chomsky). *Ogni grammatica CF senza simboli inutili, ε-produzioni e produzioni unitarie può essere trasformata in forma normale di Chomsky.*

Esempio. Si riprende la grammatica dell'esempio precedente:

$$S \to \varepsilon \mid 0 \mid 1 \mid 00 \mid 0S0 \mid 11 \mid 1S1.$$

Le produzioni $S \to \varepsilon$, ammessa per la deroga, e $S \to 0 \mid 1$, del tipo $A \to a$, vanno bene; $S \to 00 \mid 0S0 \mid 11 \mid 1S1$ no.

Passo 1. Si aggiunge a V un nuovo simbolo X_1, con la produzione $X_1 \to 0$, che prende il posto di 0 nelle produzioni lunghe:

$$S \to X_1X_1 \mid X_1SX_1 \mid 11 \mid 1S1.$$

La produzione $S \to X_1X_1$ va bene; $S \to X_1SX_1 \mid 11 \mid 1S1$ no.

Passo 2. Si aggiunge un nuovo simbolo X_2, con la produzione $X_2 \to X_1S$, che prende il posto di X_1S:

$$S \to X_1X_1 \mid X_2X_1 \mid 11 \mid 1S1.$$

Le produzioni $S \to X_1X_1 \mid X_2X_1$ vanno bene; $S \to 11 \mid 1S1$ no.

Passo 3. Si aggiunge un nuovo simbolo X_3, con la produzione $X_3 \to 1$, che prende il posto di 1:

$$S \to X_1X_1 \mid X_2X_1 \mid X_3X_3 \mid X_3SX_3.$$

Solo $S \to X_3SX_3$ non va bene.

Passo 4. Si aggiunge un nuovo simbolo X_4, con la produzione $X_4 \to X_3S$, che prende il posto di X_3S:

$$S \to X_1X_1 \mid X_2X_1 \mid X_3X_3 \mid X_4X_3.$$

Ora tutte le produzioni vanno bene. La grammatica in forma normale di Chomsky è $\langle\{S, X_1, X_2, X_3, X_4\}, \{0, 1\}, P, S\rangle$, con produzioni P

$$\begin{aligned} S &\to \varepsilon \mid 0 \mid 1 \mid X_1X_1 \mid X_2X_1 \mid X_3X_3 \mid X_4X_3, \\ X_1 &\to 0, \\ X_2 &\to X_1S, \\ X_3 &\to 1, \\ X_4 &\to X_3S. \end{aligned}$$

◇

4.10 Pumping lemma per i linguaggi CF

Come per i linguaggi regolari (Teorema 9), il pumping lemma per i linguaggi CF fornisce una *condizione necessaria* perché un linguaggio sia CF. Detta Π la proprietà espressa dal lemma, vale

$$\mathcal{L} \text{ CF} \implies \Pi,$$

e, per contrapposizione, se la proprietà Π è falsa il linguaggio non è CF:

$$\neg\Pi \implies \mathcal{L} \text{ non è CF.}$$

La condizione è solo necessaria e non sufficiente: il viceversa, $\Pi \implies \mathcal{L}$ CF, non vale. Il lemma serve quindi a dimostrare che un linguaggio *non* è CF, ma non che lo è.

Teorema 12 (Pumping lemma per i linguaggi CF). *Sia $\mathcal{L}$ un linguaggio CF, cioè esiste una grammatica CF $\mathcal{G} = \langle V, T, P, S\rangle$ tale che $\mathcal{L} = \mathcal{L}(\mathcal{G})$. Allora $\exists k \in \mathbb{N}$ tale che $\forall z \in \mathcal{L}$ con $|z| \geq k$ esiste una suddivisione in cinque parti, $z = uvwxy$, con*

$$|vwx| \leq k, \qquad |vx| > 0,$$

in modo che

$$\forall i \in \mathbb{N}.\ uv^iwx^iy \in \mathcal{L}.$$

In altre parole, ogni stringa abbastanza lunga di $\mathcal{L}$ contiene due parti v e x, non entrambe vuote e comprese in un tratto vwx lungo al più k, che si possono ripetere *lo stesso numero di volte* o togliere insieme, ottenendo sempre stringhe di $\mathcal{L}$. A differenza del caso regolare, le parti da ripetere sono due e il tratto vwx non deve stare all'inizio della stringa. La Figura 4.5 mostra i tre casi:

- $i = 1$: $uv^1wx^1y = uvwxy$ è la stringa z stessa;
- $i > 1$: v e x si ripetono entrambe i volte, per esempio con $i = 2$ si ottiene $uvvwxxy$;
- $i = 0$: v e x si tolgono entrambe e resta uwy.

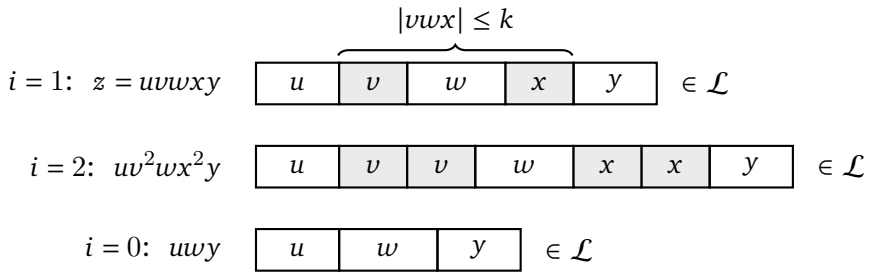


FIGURA 4.5 *Suddivisione $z = uvwxy$ con $|vwx| \le k$ e $|vx| > 0$: per $i = 1$ si ritrova z, per $i > 1$ (qui $i = 2$) le parti v e x, in grigio, si ripetono lo stesso numero di volte, per $i = 0$ si tolgono entrambe. In ogni caso si ottengono stringhe di $\mathcal{L}$.*

La Figura 4.6 mostra la struttura dei quantificatori del lemma, come la Figura 3.19 per il caso regolare. Per ogni stringa $z \in \mathcal{L}$ con $|z| \ge k$ si considerano tutte le suddivisioni $z = uvwxy$: tra queste ne *esiste* almeno una, indicata con $\exists$ nell'intersezione, che rispetta entrambi i vincoli, $|vwx| \le k$ e $|vx| > 0$, e per questa vale $uv^iwx^iy \in \mathcal{L}$ per ogni $i \in \mathbb{N}$.

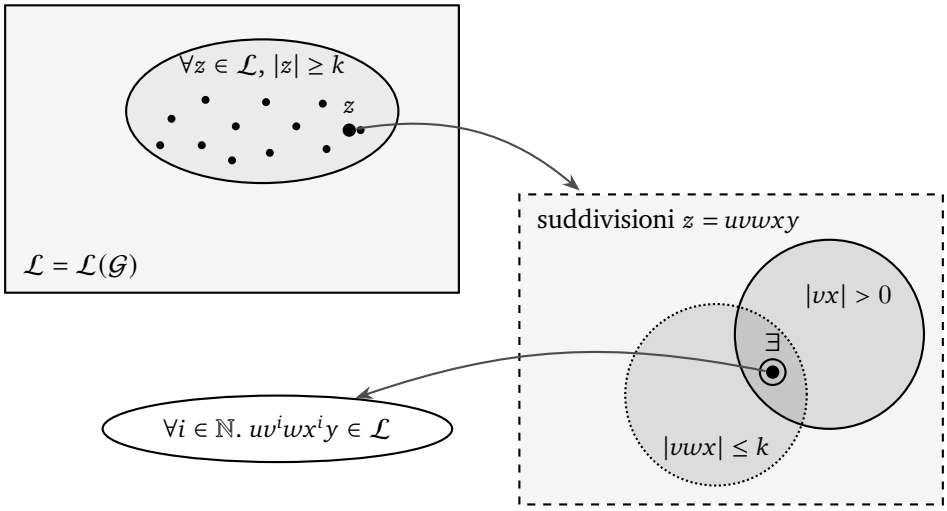


FIGURA 4.6 *Il pumping lemma per i linguaggi CF visto con gli insiemi: per ogni $z \in \mathcal{L}$ con $|z| \ge k$, tra tutte le suddivisioni $z = uvwxy$ (riquadro tratteggiato) ne esiste una ($\exists$) con $|vwx| \le k$ e $|vx| > 0$, nell'intersezione dei due cerchi, per cui $uv^iwx^iy \in \mathcal{L}$ per ogni i.*

Dimostrazione del Teorema 12. Sia $\mathcal{L}$ un linguaggio CF e sia $\mathcal{G} = \langle V, T, P, S\rangle$ una grammatica in forma normale di Chomsky (Teorema 11) tale che $\mathcal{L} = \mathcal{L}(\mathcal{G})$.[2] Sia $n = |V|$ il numero di simboli non terminali: si sceglie

$$k = 2^n.$$

ALTEZZA E FOGLIE. Se la grammatica è in forma normale di Chomsky, l'albero di derivazione è binario: ogni nodo interno ha due figli, per una produzione $A \to BC$, tranne quelli dell'ultimo livello, che hanno un solo figlio, la foglia, per una produzione $A \to a$. L'ultima produzione quindi non moltiplica per 2 il numero dei nodi, e un albero di derivazione alto h ha al più 2^{h-1} foglie.

[2] Si suppone che $\mathcal{G}$ non abbia la produzione $S \to \varepsilon$, cioè che generi $\mathcal{L} \setminus \{\varepsilon\}$: così nessuna variabile genera ε. La stringa vuota non interessa, perché il lemma riguarda solo stringhe con $|z| \ge k \ge 1$.

Sia $z \in \mathcal{L}$ con $|z| \geq k = 2^n$ e si consideri un suo albero di derivazione (Teorema 10). Le foglie, lette da sinistra a destra, formano z, quindi sono $|z| \geq 2^n$. Se h è l'altezza dell'albero,

$$2^{h-1} \geq |z| \geq 2^n \implies h - 1 \geq n \implies h \geq n + 1.$$

UN NON TERMINALE RIPETUTO. Il cammino più lungo dalla radice a una foglia ha $h \geq n + 1$ archi, quindi almeno $n + 2$ nodi. L'ultimo è una foglia, cioè un terminale; gli altri, almeno $n + 1$, sono nodi interni, etichettati con non terminali. Poiché i non terminali sono solo n, nel cammino almeno un non terminale è ripetuto due volte.

Percorrendo il cammino dalla foglia verso la radice, sia A il *primo* non terminale che si ripete, e siano $\hat{A}$ e A^* i due nodi del cammino etichettati con A, dove A^* è il più profondo tra i due, cioè quello più vicino alla foglia. Come mostra la Figura 4.7, z si suddivide in cinque parti:

- w, generata dal sottoalbero di A^*;
- v e x, generate dal sottoalbero di $\hat{A}$, a sinistra e a destra del sottoalbero di A^*;
- u e y, generate dal resto dell'albero, a sinistra e a destra del sottoalbero di $\hat{A}$.

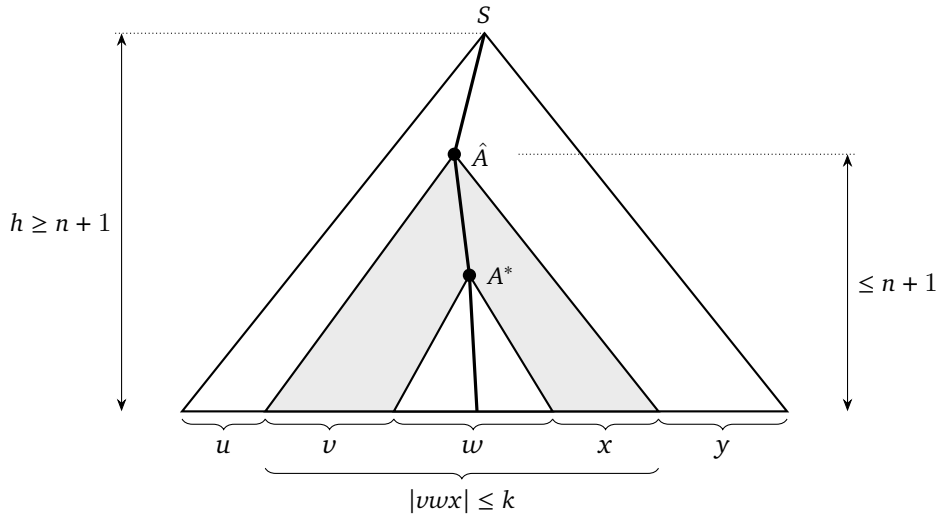


FIGURA 4.7 *Albero di derivazione di z: lungo il cammino più lungo, in grassetto, A è il primo non terminale che si ripete partendo dalla foglia, e A^* è la più profonda delle due occorrenze. Il sottoalbero di A^*, in bianco, genera w; quello di $\hat{A}$, in grigio, genera vwx.*

La suddivisione rispetta i due vincoli del lemma.

- $|vwx| \leq k$. Il tratto del cammino che parte da $\hat{A}$ è il cammino più lungo del suo sottoalbero: se ce ne fosse uno più lungo, lo sarebbe anche nell'albero intero. In questo tratto, sotto $\hat{A}$, i non terminali sono tutti diversi, perché A è la prima ripetizione partendo dalla foglia: sono al più n, più $\hat{A}$ e la foglia. Il sottoalbero di $\hat{A}$ ha quindi altezza al più $n + 1$ e al più $2^{(n+1)-1} = 2^n = k$ foglie, cioè $|vwx| \leq k$.
- $|vx| > 0$. Il nodo $\hat{A}$ ha un discendente, A^*, quindi la sua produzione non è del tipo $A \to a$, ma $A \to BC$. Il nodo A^* sta sotto uno solo dei due figli (Figura 4.8): se sta sotto C, tutto ciò che genera B finisce in v; se sta sotto B, tutto ciò che genera C finisce in x. Poiché non ci sono ε-produzioni, B e C generano stringhe non vuote, quindi $|v| \geq 1$ nel primo caso e $|x| \geq 1$ nel secondo.

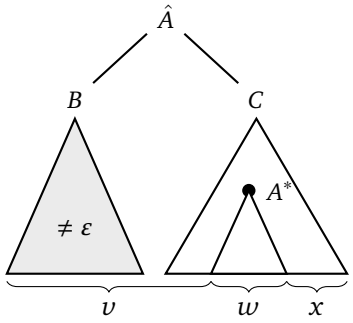


FIGURA 4.8 *Il nodo $\hat{A}$ si espande con $A \to BC$ e A^* sta sotto C: ciò che genera B, non vuoto, finisce in v. Se A^* stesse sotto B, ciò che genera C finirebbe in x.*

POMPARE. I nodi $\hat{A}$ e A^* hanno la stessa etichetta A, quindi i loro sottoalberi sono intercambiabili. Scambiarli significa, alla fine, applicare ad A^* le stesse produzioni applicate ad $\hat{A}$, e viceversa: un sottoalbero registra proprio le produzioni applicate a partire dalla sua radice. Si può fare perché la grammatica è libera dal contesto: una produzione $A \to \beta$ si applica guardando solo la variabile A, non ciò che le sta intorno. Come mostra la Figura 4.9:

- $i \geq 2$: al posto del sottoalbero di A^* si mette una copia di quello di $\hat{A}$, cioè ad A^* si applica di nuovo $A \overset{\mathcal{G}}{\Rightarrow}_* vAx$. Ripetendo la sostituzione,

$$S \overset{\mathcal{G}}{\Rightarrow}_* uAy \overset{\mathcal{G}}{\Rightarrow}_* uvAxy \overset{\mathcal{G}}{\Rightarrow}_* uv^2Ax^2y \overset{\mathcal{G}}{\Rightarrow}_* \cdots \overset{\mathcal{G}}{\Rightarrow}_* uv^iAx^iy \overset{\mathcal{G}}{\Rightarrow}_* uv^iwx^iy;$$

- $i = 1$: è l'albero di partenza, che genera $z = uvwxy$;
- $i = 0$: al posto del sottoalbero di $\hat{A}$ si mette quello di A^*, cioè ad $\hat{A}$ si applica subito $A \overset{\mathcal{G}}{\Rightarrow}_* w$:

$$S \overset{\mathcal{G}}{\Rightarrow}_* uAy \overset{\mathcal{G}}{\Rightarrow}_* uwy.$$

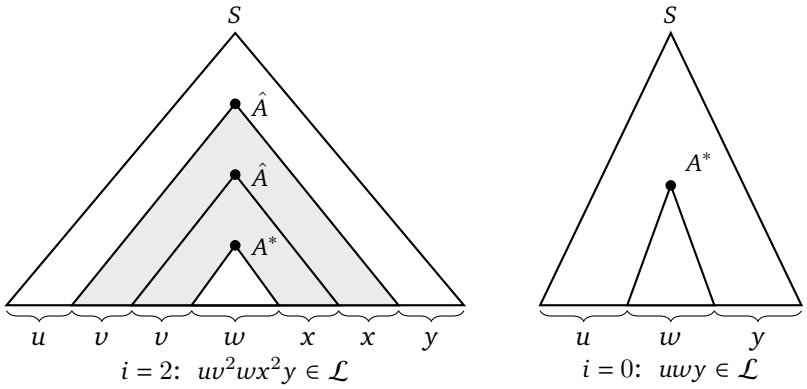


FIGURA 4.9 *A sinistra, al posto del sottoalbero di A^* c'è una copia di quello di $\hat{A}$: v e x compaiono due volte. A destra, al posto del sottoalbero di $\hat{A}$ c'è quello di A^*: v e x spariscono.*

In ogni caso si ottiene un albero di derivazione con radice S e foglie terminali, quindi

$$|vwx| \leq k \ \wedge \ |vx| > 0 \implies \forall i \in \mathbb{N}.\ uv^iwx^iy \in \mathcal{L}. \qquad \square$$

L'utilizzo del pumping lemma per dimostrare che un linguaggio non è CF richiede la negazione della condizione necessaria: se vale $\neg\Pi$, $\mathcal{L}$ non è CF. Negando Π, ogni quantificatore si scambia, $\exists$ con $\forall$ e viceversa, e la formula finale si nega, come mostra la Tabella 4.2.

$\mathcal{L}$ CF $\implies \Pi$		$\neg\Pi \implies \mathcal{L}$ non CF
$\exists k \in \mathbb{N}$	$\rightarrow$	$\forall k \in \mathbb{N}$
$\forall z \in \mathcal{L}$ con $\lvert z\rvert \geq k$	$\rightarrow$	$\exists z \in \mathcal{L}$ con $\lvert z\rvert \geq k$
$\exists z = uvwxy$ con $\lvert vwx\rvert \leq k$ e $\lvert vx\rvert > 0$	$\rightarrow$	$\forall z = uvwxy$ con $\lvert vwx\rvert \leq k$ e $\lvert vx\rvert > 0$
$\forall i \in \mathbb{N}.\ uv^iwx^iy \in \mathcal{L}$	$\rightarrow$	$\exists i \in \mathbb{N}.\ uv^iwx^iy \notin \mathcal{L}$

TABELLA 4.2 *La proprietà Π del pumping lemma e la sua negazione: ogni quantificatore si scambia e l'appartenenza a $\mathcal{L}$ diventa non appartenenza.*

La Figura 4.10 mostra la negazione con gli insiemi, da confrontare con la Figura 4.6. Per ogni k basta trovare *una* stringa $z \in \mathcal{L}$ con $|z| \geq k$; per questa stringa, però, si devono considerare *tutte* le suddivisioni che rispettano i due vincoli, cioè tutti i punti dell'intersezione, e per ciascuna trovare un i che faccia uscire da $\mathcal{L}$.

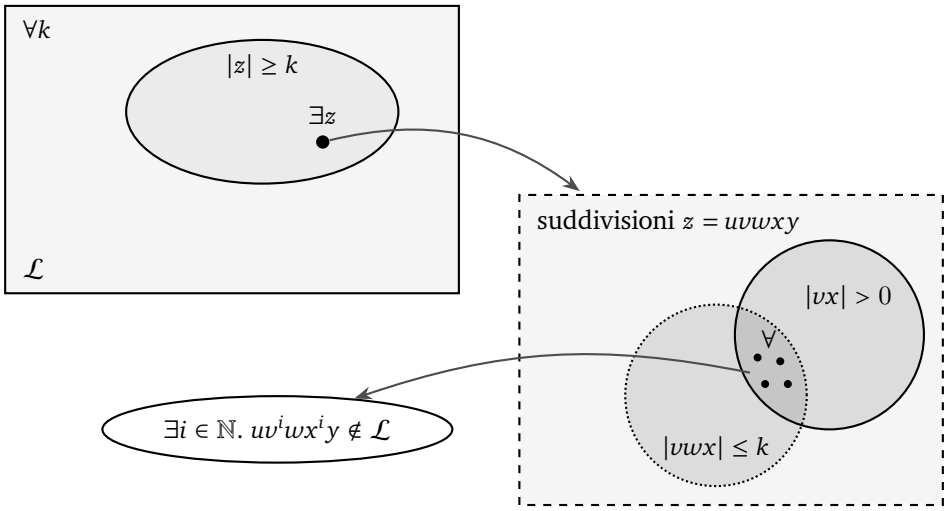


FIGURA 4.10 *La negazione del pumping lemma vista con gli insiemi: per ogni k esiste una $z \in \mathcal{L}$ con $|z| \geq k$ tale che, per tutte le sue suddivisioni con $|vwx| \leq k$ e $|vx| > 0$, esiste un i con $uv^iwx^iy \notin \mathcal{L}$.*

Esempio. Si consideri il linguaggio

$$\mathcal{L} = \{a^n b^n c^n \mid n \in \mathbb{N}\}.$$

Si dimostra che $\mathcal{L}$ non è CF verificando la negazione della condizione necessaria, un quantificatore alla volta.

- $\forall k \in \mathbb{N}$: si sceglie un valore generico k, sul quale non si può porre alcuna condizione.
- $\exists z \in \mathcal{L}$ con $|z| \geq k$: si sceglie
$$z = a^k b^k c^k,$$
che sta in $\mathcal{L}$ e ha $|z| = 3k \geq k$.
- $\forall z = uvwxy$ con $|vwx| \leq k$ e $|vx| > 0$: si devono considerare *tutte* le suddivisioni che rispettano i vincoli. Ogni blocco di z è lungo k, quindi il tratto vwx non può

toccare tutti e tre i blocchi: se contenesse una a e una c, conterrebbe anche tutto b^k, e sarebbe $|vwx| \geq k + 2 > k$. La Figura 4.11 mostra i casi possibili, con v e x entrambe in a^k, in b^k o in c^k, oppure in due blocchi consecutivi.[3]

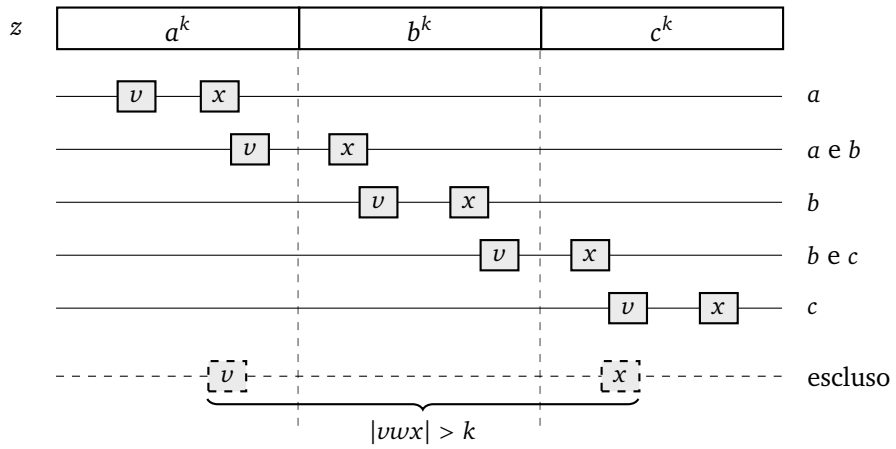


Figura 4.11 *Le posizioni possibili di v e x in $z = a^k b^k c^k$, con a destra i blocchi toccati. Poiché $|vwx| \leq k$, il tratto vwx tocca al più due blocchi consecutivi; l'ultima riga, tratteggiata, è esclusa, perché vwx conterrebbe tutto b^k.*

- $\exists i \in \mathbb{N}.\ uv^i wx^i y \notin \mathcal{L}$: si sceglie $i = 0$. Togliendo v e x i simboli restano nell'ordine a, b, c, quindi $uwy = a^p b^q c^r$ per qualche p, q, r. Poiché vwx tocca al più due blocchi, in vx manca almeno uno dei tre simboli, che in uwy compare ancora k volte; poiché $|vx| > 0$, invece, almeno un altro simbolo compare meno di k volte. I tre numeri p, q, r non sono tutti uguali, quindi $uwy \notin \mathcal{L}$.

Il ragionamento vale per ogni k e per ogni suddivisione che rispetta i vincoli, quindi la negazione della condizione necessaria è verificata: $\mathcal{L}$ non è CF. ◇

4.11 Proprietà dei linguaggi CF

Come i linguaggi regolari (Teorema 6), anche i linguaggi CF godono di proprietà di chiusura: sono chiusi rispetto all'unione, alla concatenazione e alla chiusura di Kleene.

Teorema 13 (Chiusura dei linguaggi CF). *Siano $\mathcal{L}_1$ e $\mathcal{L}_2$ linguaggi CF. Allora*

$$\mathcal{L}_1 \cup \mathcal{L}_2, \qquad \mathcal{L}_1 \cdot \mathcal{L}_2, \qquad \mathcal{L}_1^*$$

sono CF.

Le proprietà di chiusura servono anche nella direzione opposta: se un linguaggio si scrive come unione, concatenazione o chiusura di Kleene di linguaggi più semplici, basta dimostrare che questi sono CF, studiandoli separatamente. Può essere utile ricordare che i linguaggi regolari sono anche CF.

[3] v o x potrebbero anche stare a cavallo tra due blocchi, per esempio $v = ab$: il ragionamento che segue vale lo stesso, perché usa solo il fatto che vwx tocca al più due blocchi.

Dimostrazione del Teorema 13. Si dà solo un abbozzo della dimostrazione.[4] Poiché $\mathcal{L}_1$ e $\mathcal{L}_2$ sono CF, per definizione esistono due grammatiche CF

$$\mathcal{G}_1 = \langle V_1, T_1, P_1, S_1 \rangle, \qquad \mathcal{G}_2 = \langle V_2, T_2, P_2, S_2 \rangle$$

tali che $\mathcal{L}_1 = \mathcal{L}(\mathcal{G}_1)$ e $\mathcal{L}_2 = \mathcal{L}(\mathcal{G}_2)$. Si suppone $V_1 \cap V_2 = \emptyset$: le due grammatiche non hanno variabili in comune. Non è una restrizione, perché basta rinominare le variabili di $\mathcal{G}_2$, e rinominarle non cambia il linguaggio generato. Serve perché altrimenti, unendo le produzioni, una variabile comune avrebbe sia le produzioni di $\mathcal{G}_1$ sia quelle di $\mathcal{G}_2$, e una derivazione partita in $\mathcal{G}_1$ potrebbe proseguire con le produzioni di $\mathcal{G}_2$.

Unione. Sia S un simbolo nuovo, che non compare in nessuna delle due grammatiche. Si costruisce

$$\mathcal{G}_\cup = \langle V_1 \cup V_2 \cup \{S\},\ T_1 \cup T_2,\ P_\cup,\ S \rangle, \qquad P_\cup = \{S \to S_1 \mid S_2\} \cup P_1 \cup P_2.$$

Il primo passo di ogni derivazione sceglie tra S_1 e S_2; poiché le variabili sono disgiunte, da S_1 si usano poi solo le produzioni di P_1 e da S_2 solo quelle di P_2. Quindi $\mathcal{L}(\mathcal{G}_\cup) = \mathcal{L}_1 \cup \mathcal{L}_2$.

Concatenazione. Con S simbolo nuovo come prima, si costruisce

$$\mathcal{G}_\cdot = \langle V_1 \cup V_2 \cup \{S\},\ T_1 \cup T_2,\ P_\cdot,\ S \rangle, \qquad P_\cdot = P_1 \cup P_2 \cup \{S \to S_1 S_2\}.$$

Ogni derivazione inizia con $S \Rightarrow S_1 S_2$: da S_1 si deriva, con le sole produzioni di P_1, una stringa di $\mathcal{L}_1$, e da S_2, con quelle di P_2, una stringa di $\mathcal{L}_2$. Quindi $\mathcal{L}(\mathcal{G}_\cdot) = \mathcal{L}_1 \cdot \mathcal{L}_2$.

Chiusura di Kleene. Con S simbolo nuovo, si costruisce

$$\mathcal{G}_* = \langle V_1 \cup \{S\},\ T_1,\ P_*,\ S \rangle, \qquad P_* = P_1 \cup \{S \to \varepsilon \mid S_1 S\}.$$

Con $S \to S_1 S$ il simbolo S si richiama da solo, mettendo ogni volta un S_1 davanti a sé; con $S \to \varepsilon$ la catena si chiude. Usando n volte la prima produzione e poi la seconda,

$$S \Rightarrow S_1 S \Rightarrow S_1 S_1 S \Rightarrow \cdots \Rightarrow S_1^n S \Rightarrow S_1^n,$$

e ogni S_1 deriva una stringa di $\mathcal{L}_1$, quindi si ottengono le stringhe di $\mathcal{L}_1^n$. Con $n = 0$ si usa subito $S \to \varepsilon$ e si ottiene ε, che appartiene sempre a $\mathcal{L}_1^*$ come unico elemento di $\mathcal{L}_1^0$. Poiché n è arbitrario, $\mathcal{L}(\mathcal{G}_*) = \bigcup_{n \geq 0} \mathcal{L}_1^n = \mathcal{L}_1^*$. □

Come per i linguaggi regolari, la chiusura rispetto all'unione vale solo per un numero *finito* di linguaggi: anche se ogni $\mathcal{L}_i$ è CF, l'unione di infiniti linguaggi CF può non esserlo, e va studiata caso per caso. In simboli,

$$\left(\forall i \in \mathbb{N}.\ \mathcal{L}_i \text{ CF}\right) \not\Rightarrow \bigcup_{i \in \mathbb{N}} \mathcal{L}_i \text{ CF}.$$

Teorema 14 (Non chiusura rispetto all'intersezione). *I linguaggi CF* non *sono chiusi rispetto all'intersezione: esistono due linguaggi CF $\mathcal{L}_1$ e $\mathcal{L}_2$ tali che $\mathcal{L}_1 \cap \mathcal{L}_2$ non è CF.*

Dimostrazione del Teorema 14. Si fornisce un caso in cui $\mathcal{L}_1$ e $\mathcal{L}_2$ sono CF, ma $\mathcal{L}_1 \cap \mathcal{L}_2$ non lo è.

Il linguaggio $\mathcal{L}_1$. Si consideri

$$\mathcal{L}_1 = \{a^i b^i c^j \mid i, j \in \mathbb{N}\}$$

[4] La dimostrazione non è completa: si costruisce la grammatica, senza dimostrare formalmente che genera il linguaggio voluto.

che ha tante a quante b, mentre il numero di c è libero. È generato dalla grammatica $\mathcal{G}_1$ con produzioni

$$S \to RC, \qquad R \to aRb \mid \varepsilon, \qquad C \to cC \mid \varepsilon,$$

dove R genera $a^i b^i$ e C genera c^j, per esempio

$$S \Rightarrow RC \Rightarrow aRbC \Rightarrow aaRbbC \Rightarrow aabbC \Rightarrow aabbcC \Rightarrow aabbccC \Rightarrow aabbcc.$$

Si ha $\mathcal{L}(\mathcal{G}_1) = \mathcal{L}_1$, come si dimostra con il metodo della Sezione 4.2.1, quindi $\mathcal{L}_1$ è CF.

Il linguaggio $\mathcal{L}_2$. Si consideri

$$\mathcal{L}_2 = \{a^j b^i c^i \mid i, j \in \mathbb{N}\}$$

che ha tante b quante c, mentre il numero di a è libero. È generato dalla grammatica $\mathcal{G}_2$ con produzioni

$$S \to AR, \qquad A \to aA \mid \varepsilon, \qquad R \to bRc \mid \varepsilon,$$

dove A genera a^j e R genera $b^i c^i$, per esempio

$$S \Rightarrow AR \Rightarrow aAR \Rightarrow aaAR \Rightarrow aaaAR \Rightarrow aaaR \Rightarrow aaabRc \Rightarrow aaabbRcc \Rightarrow aaabbcc.$$

Si ha $\mathcal{L}(\mathcal{G}_2) = \mathcal{L}_2$, quindi anche $\mathcal{L}_2$ è CF.

L'intersezione. Entrambi i linguaggi contengono solo stringhe della forma $a^m b^n c^l$, quindi anche $\mathcal{L}_1 \cap \mathcal{L}_2$. Una tale stringa appartiene all'intersezione se e solo se appartiene a entrambi i linguaggi:

$$a^m b^n c^l \in \mathcal{L}_1 \cap \mathcal{L}_2 \iff \underbrace{m = n}_{\mathcal{L}_1} \wedge \underbrace{n = l}_{\mathcal{L}_2} \iff m = n = l,$$

dove l'ultimo passo segue per transitività. Quindi

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \{a^n b^n c^n \mid n \in \mathbb{N}\},$$

che non è CF, come visto con il pumping lemma nella Sezione 4.10. Due linguaggi CF hanno quindi un'intersezione che non è CF. □

4.12 Proprietà di decisione

Come per i linguaggi regolari (Teorema 7), alcuni problemi sui linguaggi CF sono decidibili: per ciascuno esiste un algoritmo che termina sempre con la risposta corretta.

Teorema 15 (Problemi decidibili sui linguaggi CF). *Sia $\mathcal{G} = \langle V, T, P, S\rangle$ una grammatica CF. I seguenti problemi sono decidibili:*

(a) Vacuità*: stabilire se $\mathcal{L}(\mathcal{G}) = \emptyset$;*
(b) Finitezza*: stabilire se $\mathcal{L}(\mathcal{G})$ è finito;*
(c) Infinità*: stabilire se $\mathcal{L}(\mathcal{G})$ è infinito;*
(d) Appartenenza*: data $x \in T^*$, stabilire se $x \in \mathcal{L}(\mathcal{G})$.*

Dimostrazione del Teorema 15. Punto (a). Si usa l'operatore Γ della Fase 1 dell'eliminazione dei simboli inutili (Sezione 4.6), che in al più $|V|$ passi calcola l'insieme $\Gamma^{|V|}(\emptyset)$ dei non terminali da cui si deriva una stringa di soli terminali. Allora

$$\mathcal{L}(\mathcal{G}) \neq \emptyset \iff S \in \Gamma^{|V|}(\emptyset).$$

Se S sta nel punto fisso, da S si deriva una stringa di terminali, che appartiene a $\mathcal{L}(\mathcal{G})$; altrimenti ogni derivazione da S lascia sempre qualche non terminale, e $\mathcal{L}(\mathcal{G}) = \emptyset$. Qui ε conta come stringa di terminali, poiché $\varepsilon \in (T \cup W)^*$: con la sola produzione $S \to \varepsilon$ si ha $\mathcal{L}(\mathcal{G}) = \{\varepsilon\} \neq \emptyset$.

Per gli altri punti si porta $\mathcal{G}$ in forma normale di Chomsky (Teorema 11), con le semplificazioni della Sezione 4.5.1, che sono tutte algoritmiche. Sia $n = |V|$ il numero di non terminali della grammatica ottenuta.

Punto (d). Se $x = \varepsilon$, basta controllare se c'è la produzione $S \to \varepsilon$. Se invece $|x| = m \geq 1$, ogni derivazione di x ha esattamente $2m - 1$ passi. Infatti una produzione $A \to BC$ trasforma un non terminale in due, quindi allunga la stringa di 1, mentre $A \to a$ trasforma un non terminale in un terminale e non cambia la lunghezza. Partendo da S, lungo 1, per arrivare a x, lunga m:

- servono esattamente $m - 1$ applicazioni di $A \to BC$, le sole che allungano la stringa;
- servono esattamente m applicazioni di $A \to a$, una per ogni terminale di x.

Le derivazioni di $2m - 1$ passi sono in numero finito, perché a ogni passo si sceglie tra un numero finito di non terminali e di produzioni: si provano tutte e si controlla se una di esse produce x.

Punto (b). Sia $k = 2^n$ la costante del pumping lemma (Teorema 12). Vale

$$\mathcal{L}(\mathcal{G}) \text{ infinito} \iff \exists z \in \mathcal{L}(\mathcal{G}).\ k \leq |z| < 2k.$$

($\Leftarrow$) Poiché $|z| \geq k$, z si può pompare: $uv^i wx^i y \in \mathcal{L}(\mathcal{G})$ per ogni $i \in \mathbb{N}$, e poiché $|vx| > 0$ queste stringhe hanno lunghezze tutte diverse. Quindi $\mathcal{L}(\mathcal{G})$ è infinito.

($\Rightarrow$) Se $\mathcal{L}(\mathcal{G})$ è infinito, contiene stringhe lunghe almeno k, perché quelle più corte sono in numero finito. Sia z la più corta tra queste. Se fosse $|z| \geq 2k$, pompando con $i = 0$ si otterrebbe $uwy \in \mathcal{L}(\mathcal{G})$ con

$$|uwy| = |z| - |vx| \geq |z| - k \geq k,$$

poiché $|vx| \leq |vwx| \leq k$. Ma $|uwy| < |z|$, perché $|vx| > 0$, e questo contraddice la scelta di z come la più corta: quindi $|z| < 2k$.

Il limite superiore $2k$ serve a rendere la ricerca finita: l'algoritmo conosce solo la grammatica, non la lunghezza massima delle stringhe di $\mathcal{L}(\mathcal{G})$, e senza un limite, se $\mathcal{L}(\mathcal{G})$ fosse finito, non saprebbe mai quando fermarsi. Il valore $2k$ viene dal pumping verso il basso: con $i = 0$ si tolgono da 1 a k simboli, perché $0 < |vx| \leq k$. Una stringa lunga almeno k, accorciata così un gradino alla volta, non può scavalcare la finestra delle lunghezze da k a $2k - 1$, che contiene esattamente k valori, quanti ne toglie al massimo un gradino; una finestra più stretta, invece, potrebbe essere scavalcata.

Le stringhe con $k \leq |z| < 2k$ sono in numero finito, e per ciascuna l'appartenenza a $\mathcal{L}(\mathcal{G})$ si decide con il punto (d). Quindi si può stabilire se $\mathcal{L}(\mathcal{G})$ è infinito, e di conseguenza se è finito.

Punto (c). È il complemento del punto (b): se si sa decidere se $\mathcal{L}(\mathcal{G})$ è finito, basta dare la risposta opposta. □

Come per i linguaggi regolari, in tutti i casi il problema si riduce a controllare un numero finito di casi: i non terminali della grammatica, le derivazioni di lunghezza fissata, le stringhe di lunghezza limitata.

5 Esercizi

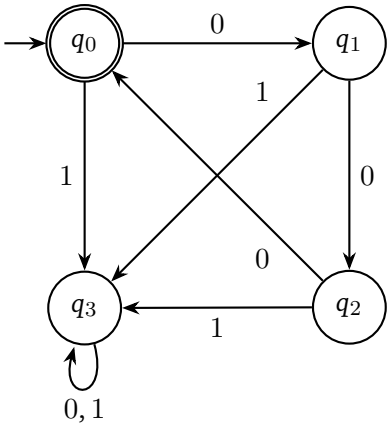
5.1 Automi a stati finiti deterministici

Gli esercizi seguenti dimostrano che un linguaggio è regolare costruendo un DFA che lo riconosce, con il metodo della Sezione 3.4.1.

Esercizio 1

Si dimostri che il linguaggio $\mathcal{L} = \{0^{3n} \mid n \geq 0\}$ *sull'alfabeto* $\Sigma = \{0, 1\}$ *è regolare.*

Svolgimento. Il linguaggio contiene le stringhe formate solo da zeri, in numero multiplo di 3. Lo riconosce il DFA $\mathcal{M} = \langle \{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_0\} \rangle$ della Figura 5.1: gli stati q_0, q_1 e q_2 contano gli zeri letti modulo 3, mentre q_3 è lo stato in cui si finisce, senza più uscirne, appena si legge un 1.



δ	0	1
q_0	q_1	q_3
q_1	q_2	q_3
q_2	q_0	q_3
q_3	q_3	q_3

Figura 5.1 *DFA che riconosce* $\{0^{3n} \mid n \geq 0\}$ *e la sua matrice di transizione.*

Bisogna dimostrare che $\mathcal{L}(\mathcal{M}) = \mathcal{L}$, cioè

$$\begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F}, \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F}. \end{cases}$$

Come nell'esempio della Sezione 3.4.1, la seconda condizione non basta per il passo induttivo, perché non dice in quale dei tre stati non finali si trovi l'automa. La si scorpora

quindi secondo il significato degli stati, ottenendo la proprietà più forte

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \implies \hat{\delta}(q_0, x) = q_0, & (1) \\ x \notin \mathcal{L} \wedge x = 0^{3i+1} \implies \hat{\delta}(q_0, x) = q_1, & (2) \\ x \notin \mathcal{L} \wedge x = 0^{3i+2} \implies \hat{\delta}(q_0, x) = q_2, & (3) \\ x \notin \mathcal{L} \wedge \text{“}x \text{ contiene almeno un } 1\text{”} \implies \hat{\delta}(q_0, x) = q_3, & (4) \end{cases}$$

con $i \geq 0$. Le quattro condizioni coprono tutte le stringhe e si escludono a vicenda: una stringa o contiene almeno un 1, oppure è della forma 0^k, e in tal caso k diviso per 3 dà resto 0, 1 o 2.

Per $|x| \leq 2$ esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$, quindi si sceglie questa come base:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \in \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 & \text{(caso 1 di } \Psi) \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_1 & \text{(caso 2 di } \Psi) \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_3 & \text{(caso 4 di } \Psi) \\ |x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_2 & \text{(caso 3 di } \Psi) \\ & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_3 & \text{(caso 4 di } \Psi) \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_3 & \text{(caso 4 di } \Psi) \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_3 & \text{(caso 4 di } \Psi) \end{array}\right\} \text{Base}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $|x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle quattro condizioni. Si distingue un caso per ciascuna di esse.

1. $x' \in \mathcal{L}$, cioè $x' = 0^{3i}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   1a. $x = x'0 = 0^{3i+1}$, quindi $x \notin \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_0, 0) = q_1.$$
   1b. $x = x'1$ contiene un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_0, 1) = q_3.$$
2. $x' \notin \mathcal{L}$ e $x' = 0^{3i+1}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   2a. $x = x'0 = 0^{3i+2}$, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_1, 0) = q_2.$$
   2b. $x = x'1$ contiene un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_1, 1) = q_3.$$
3. $x' \notin \mathcal{L}$ e $x' = 0^{3i+2}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$.
   3a. $x = x'0 = 0^{3i+3} = 0^{3(i+1)}$, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_2, 0) = q_0.$$

3b. $x = x'1$ contiene un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):

$$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_2, 1) = q_3.$$

4. $x' \notin \mathcal{L}$ e x' contiene almeno un 1: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_3$. Qualunque sia l'ultimo simbolo $a \in \{0, 1\}$, anche $x = x'a$ contiene almeno un 1, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):

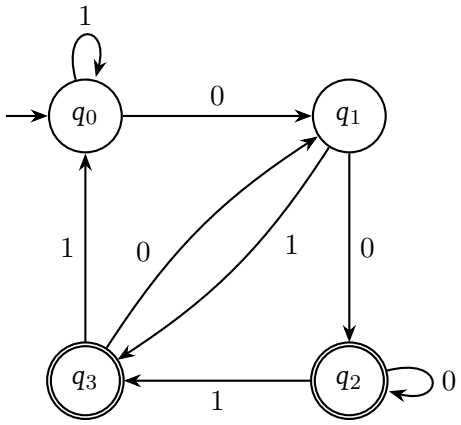
$$\hat{\delta}(q_0, x'a) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), a) \stackrel{\text{Hp}}{=} \delta(q_3, a) = q_3.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$, quindi $\Psi(x)$ vale anche per $|x| = n + 1$ e, per induzione, per ogni $x \in \Sigma^*$. In particolare, se $x \in \mathcal{L}$ allora $\hat{\delta}(q_0, x) = q_0 \in \mathcal{F}$; se $x \notin \mathcal{L}$ allora $\hat{\delta}(q_0, x) \in \{q_1, q_2, q_3\}$, e nessuno di questi stati è finale. Quindi $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.

Esercizio 2

Si dimostri che il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid \exists v \in \Sigma^*,\ a \in \Sigma.\ x = v0a\}$ sull'alfabeto $\Sigma = \{0, 1\}$, formato dalle stringhe il cui penultimo simbolo è 0, è regolare.

SVOLGIMENTO. Lo riconosce il DFA $\mathcal{M} = \langle\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_2, q_3\}\rangle$ della Figura 5.2. Ogni stato ricorda gli ultimi due simboli letti: q_0 corrisponde a 11, q_1 a 10, q_2 a 00 e q_3 a 01. Gli stati finali sono quelli in cui il penultimo simbolo è 0, cioè q_2 e q_3.



δ	0	1
q_0	q_1	q_0
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_1	q_0

FIGURA 5.2 *DFA che riconosce le stringhe con penultimo simbolo 0 e la sua matrice di transizione.*

Le stringhe con meno di due simboli non hanno un penultimo simbolo, quindi non appartengono a $\mathcal{L}$. Per le stringhe con almeno due simboli si scorpora la tesi secondo gli ultimi due simboli letti, ottenendo

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \wedge x = v00 \implies \hat{\delta}(q_0, x) = q_2, & (1) \\ x \in \mathcal{L} \wedge x = v01 \implies \hat{\delta}(q_0, x) = q_3, & (2) \\ x \notin \mathcal{L} \wedge x = v10 \implies \hat{\delta}(q_0, x) = q_1, & (3) \\ x \notin \mathcal{L} \wedge x = v11 \implies \hat{\delta}(q_0, x) = q_0, & (4) \end{cases}$$

con $v \in \Sigma^*$. Per $|x| \geq 2$ le quattro condizioni coprono tutte le stringhe e si escludono a vicenda, perché dipendono solo dagli ultimi due simboli.

La prima lunghezza per cui esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$ è $|x| = 2$, quindi la base comprende tutte le stringhe con $|x| \leq 2$. Quelle più corte non soddisfano nessuna condizione di Ψ e si verificano direttamente sulla tesi:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} & & \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_1 \notin \mathcal{F} & & \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_0 \notin \mathcal{F} & & \\ |x| = 2 & x = 00 \in \mathcal{L} & \hat{\delta}(q_0, 00) = q_2 & \text{(caso 1 di } \Psi) & \\ & x = 01 \in \mathcal{L} & \hat{\delta}(q_0, 01) = q_3 & \text{(caso 2 di } \Psi) & \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 & \text{(caso 3 di } \Psi) & \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_0 & \text{(caso 4 di } \Psi) & \end{array}\right\} \text{Base}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $2 \leq |x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n \geq 2$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle quattro condizioni. Gli ultimi due simboli di x sono l'ultimo simbolo di x' seguito da quello appena letto.

1. $x' \in \mathcal{L}$ e $x' = v00$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$.
   1a. $x = x'0 = (v0)00$, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_2, 0) = q_2.$$
   1b. $x = x'1 = (v0)01$, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_2, 1) = q_3.$$
2. $x' \in \mathcal{L}$ e $x' = v01$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_3$.
   2a. $x = x'0 = (v0)10$, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_3, 0) = q_1.$$
   2b. $x = x'1 = (v0)11$, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_3, 1) = q_0.$$
3. $x' \notin \mathcal{L}$ e $x' = v10$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   3a. $x = x'0 = (v1)00$, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_1, 0) = q_2.$$
   3b. $x = x'1 = (v1)01$, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_1, 1) = q_3.$$
4. $x' \notin \mathcal{L}$ e $x' = v11$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   4a. $x = x'0 = (v1)10$, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'0) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \overset{\text{Hp}}{=} \delta(q_0, 0) = q_1.$$

4b. $x = x'1 = (v1)11$, quindi $x \notin \mathcal{L}$ (caso 4 di Ψ):

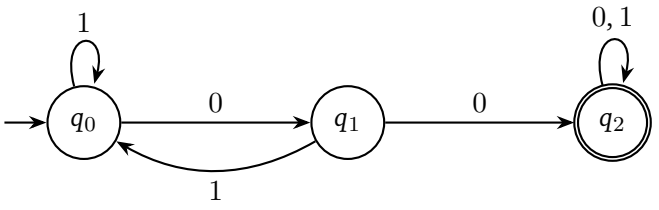
$$\hat{\delta}(q_0, x'1) \overset{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \overset{\text{Hp}}{=} \delta(q_0, 1) = q_0.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$, quindi $\Psi(x)$ vale per ogni x con $|x| \geq 2$. Ne segue la tesi: se $x \in \mathcal{L}$, allora $|x| \geq 2$ e x termina con 00 o 01, quindi $\hat{\delta}(q_0, x) \in \{q_2, q_3\} = \mathcal{F}$; se $x \notin \mathcal{L}$, allora x ha meno di due simboli, e la base mostra che $\hat{\delta}(q_0, x) \notin \mathcal{F}$, oppure termina con 10 o 11, e $\hat{\delta}(q_0, x) \in \{q_0, q_1\}$, stati non finali. Quindi $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.

Esercizio 3

Si dimostri che il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid$ "x contiene almeno due 0 consecutivi"$\}$ sull'alfabeto $\Sigma = \{0, 1\}$ è regolare.

SVOLGIMENTO. Lo riconosce il DFA $\mathcal{M} = \langle \{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\} \rangle$ della Figura 5.3. Finché non ha letto due 0 consecutivi l'automa ricorda soltanto l'ultimo simbolo: è in q_1 se l'ultimo simbolo è 0 e in q_0 altrimenti. Appena legge il secondo 0 consecutivo passa in q_2, da cui non esce più.



δ	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

FIGURA 5.3 *DFA che riconosce le stringhe con almeno due 0 consecutivi e la sua matrice di transizione.*

Si scorpora la tesi secondo il significato degli stati, ottenendo

$$\Psi(x) = \begin{cases} x \notin \mathcal{L} \wedge x = w0 \implies \hat{\delta}(q_0, x) = q_1, & (1) \\ x \in \mathcal{L} \implies \hat{\delta}(q_0, x) = q_2, & (2) \\ x \notin \mathcal{L} \wedge x = w1 \implies \hat{\delta}(q_0, x) = q_0, & (3) \end{cases}$$

con $w \in \Sigma^*$. Le tre condizioni si escludono a vicenda e coprono tutte le stringhe tranne ε, che non ha un ultimo simbolo e si verifica a parte.

La prima lunghezza per cui esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$ è $|x| = 2$, quindi la base comprende tutte le stringhe con $|x| \leq 2$:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} & & \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_1 & \text{(caso 1 di } \Psi) & \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_0 & \text{(caso 3 di } \Psi) & \\ |x| = 2 & x = 00 \in \mathcal{L} & \hat{\delta}(q_0, 00) = q_2 & \text{(caso 2 di } \Psi) & \\ & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_0 & \text{(caso 3 di } \Psi) & \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 & \text{(caso 1 di } \Psi) & \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_0 & \text{(caso 3 di } \Psi) & \end{array}\right\} \text{BASE}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $|x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n \geq 2$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle tre condizioni.

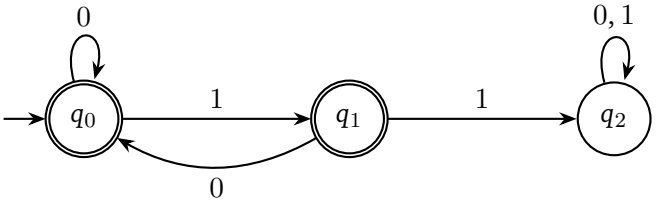
1. $x' \notin \mathcal{L}$ e $x' = w0$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   1a. $x = x'0 = w00$ contiene due 0 consecutivi, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_1, 0) = q_2.$$
   1b. $x = x'1 = w01$: aggiungere un 1 non crea due 0 consecutivi, quindi $x \notin \mathcal{L}$ e x termina con 1 (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_1, 1) = q_0.$$
2. $x' \in \mathcal{L}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$. Qualunque sia l'ultimo simbolo $a \in \{0, 1\}$, $x = x'a$ contiene ancora i due 0 consecutivi di x', quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'a) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), a) \stackrel{\text{Hp}}{=} \delta(q_2, a) = q_2.$$
3. $x' \notin \mathcal{L}$ e $x' = w1$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   3a. $x = x'0 = w10$: poiché x' termina con 1, aggiungere uno 0 non crea due 0 consecutivi, quindi $x \notin \mathcal{L}$ e x termina con 0 (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_0, 0) = q_1.$$
   3b. $x = x'1 = w11$: aggiungere un 1 non crea due 0 consecutivi, quindi $x \notin \mathcal{L}$ e x termina con 1 (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_0, 1) = q_0.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$. Quindi $\hat{\delta}(q_0, x) = q_2 \in \mathcal{F}$ se $x \in \mathcal{L}$, mentre se $x \notin \mathcal{L}$ lo stato raggiunto è q_0 o q_1, che non sono finali: $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.

Esercizio 4

Si dimostri che il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid$ "x non contiene due 1 consecutivi"$\}$ sull'alfabeto $\Sigma = \{0, 1\}$ è regolare.

Svolgimento. Lo riconosce il DFA $\mathcal{M} = \langle \{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_0, q_1\} \rangle$ della Figura 5.4. Finché non ha letto due 1 consecutivi l'automa ricorda soltanto l'ultimo simbolo: è in q_1 se l'ultimo simbolo è 1 e in q_0 altrimenti. Appena legge il secondo 1 consecutivo passa in q_2, da cui non esce più e che non è finale.



δ	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_2	q_2

Figura 5.4 *DFA che riconosce le stringhe senza due 1 consecutivi e la sua matrice di transizione.*

Si scorpora la tesi secondo il significato degli stati, ottenendo

$$\Psi(x) = \begin{cases} x \in \mathcal{L} \wedge x = w0 \implies \hat{\delta}(q_0, x) = q_0, & (1) \\ x \in \mathcal{L} \wedge x = w1 \implies \hat{\delta}(q_0, x) = q_1, & (2) \\ x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) = q_2, & (3) \end{cases}$$

con $w \in \Sigma^*$. Anche qui le tre condizioni si escludono a vicenda e coprono tutte le stringhe tranne ε, che si verifica a parte.

La prima lunghezza per cui esistono sia stringhe in $\mathcal{L}$ sia stringhe non in $\mathcal{L}$ è $|x| = 2$, quindi la base comprende tutte le stringhe con $|x| \leq 2$:

$$\left.\begin{array}{lllll} |x| = 0 & x = \varepsilon \in \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \in \mathcal{F} & & \\ |x| = 1 & x = 0 \in \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 & \text{(caso 1 di } \Psi) & \\ & x = 1 \in \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 & \text{(caso 2 di } \Psi) & \\ |x| = 2 & x = 00 \in \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 & \text{(caso 1 di } \Psi) & \\ & x = 01 \in \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 & \text{(caso 2 di } \Psi) & \\ & x = 10 \in \mathcal{L} & \hat{\delta}(q_0, 10) = q_0 & \text{(caso 1 di } \Psi) & \\ & x = 11 \notin \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 & \text{(caso 3 di } \Psi) & \end{array}\right\} \text{Base}$$

Passo induttivo. Si suppone che $\Psi(x)$ valga per ogni $x \in \Sigma^*$ con $|x| \leq n$, per un certo $n \geq 2$, e sia x una stringa con $|x| = n + 1$. Allora $x = x'0$ oppure $x = x'1$, con $|x'| = n \geq 2$: per ipotesi induttiva vale $\Psi(x')$, e x' soddisfa esattamente una delle tre condizioni.

1. $x' \in \mathcal{L}$ e $x' = w0$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_0$.
   1a. $x = x'0 = w00$: aggiungere uno 0 non crea due 1 consecutivi, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_0, 0) = q_0.$$
   1b. $x = x'1 = w01$: poiché x' termina con 0, aggiungere un 1 non crea due 1 consecutivi, quindi $x \in \mathcal{L}$ (caso 2 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_0, 1) = q_1.$$
2. $x' \in \mathcal{L}$ e $x' = w1$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_1$.
   2a. $x = x'0 = w10$: aggiungere uno 0 non crea due 1 consecutivi, quindi $x \in \mathcal{L}$ (caso 1 di Ψ):
   $$\hat{\delta}(q_0, x'0) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 0) \stackrel{\text{Hp}}{=} \delta(q_1, 0) = q_0.$$
   2b. $x = x'1 = w11$ termina con due 1 consecutivi, quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'1) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), 1) \stackrel{\text{Hp}}{=} \delta(q_1, 1) = q_2.$$
3. $x' \notin \mathcal{L}$: per ipotesi induttiva $\hat{\delta}(q_0, x') = q_2$. Qualunque sia l'ultimo simbolo $a \in \{0, 1\}$, $x = x'a$ contiene ancora i due 1 consecutivi di x', quindi $x \notin \mathcal{L}$ (caso 3 di Ψ):
   $$\hat{\delta}(q_0, x'a) \stackrel{\text{def}}{=} \delta(\hat{\delta}(q_0, x'), a) \stackrel{\text{Hp}}{=} \delta(q_2, a) = q_2.$$

In tutti i casi lo stato raggiunto da x è quello previsto da $\Psi(x)$. Quindi se $x \in \mathcal{L}$ lo stato raggiunto è q_0 o q_1, entrambi finali, mentre se $x \notin \mathcal{L}$ è $q_2 \notin \mathcal{F}$: $\mathcal{L}(\mathcal{M}) = \mathcal{L}$.